



PROYECTO DE INFRAESTRUCTURA PaaS PARA OPENPADI

Proyecto de Fin de Ciclo
2º ASIR 2024-2025

Felipe Robles López



Contenido

Preguntas rápidas para entender qué es OpenPaDi y qué abarca este proyecto	4
¿Qué es OpenPaDi?	4
¿Cubre este proyecto la totalidad de OpenPaDi?	4
¿Habrá una aplicación web en este proyecto?	4
1. Contextualización	5
1.1. Descripción del proyecto	5
1.2. Modelo de Negocio	6
1.1.1. Descripción externa	8
1.2.2. Descripción interna.....	9
1.2.3. Viabilidad y expansión del negocio.....	12
1.3. Descripción y objetivos del proyecto	14
1.3.1. Estado actual de la problemática y soluciones existentes	14
1.3.2. Objetivo general	14
1.3.3. Objetivos específicos.....	15
1.3.4. Contribución a la necesidad detectada.....	16
1.3.5. Oportunidad y Modelo de Financiación de la Plataforma OpenPaDi	17
2. ANÁLISIS	18
2.1. Requisitos de la solución técnica.....	18
2.2. Análisis de Riesgos e Interesados	22
2.2.1. Identificación de Interesados Clave	22
2.2.2. Identificación, Evaluación y Mitigación de Riesgos para la Infraestructura PaaS.....	23
3. Planificación temporal y de recursos de las distintas fases	28
3.1. Identificación de fases.....	28
3.2. Identificación de recursos	33
3.2.1. Recursos Personales	33
3.2.2. Recursos Materiales.....	34
3.3. Diagrama de Gant.....	37
4. DISEÑO DE LA SOLUCIÓN	38
4.1. Resumen de la Solución Técnica Propuesta	38
4.2. Diseño de la Red.....	40
4.2.1. Consideraciones sobre la Infraestructura Física de Red Subyacente	41

4.2.2. Arquitectura de Red Lógica Virtualizada sobre Proxmox VE.....	43
4.2.1. Comunicaciones externas	47
4.2.2. Comunicaciones Internas y Segmentación de Red Lógica	48
4.2.3. Integración de Redes y Dispositivos Virtualizados	49
4.2.4. Simulación de la red en Proxmox	50
4.3. Servicios de Red	62
4.3.1. Servicio de Base de Datos Relacional (PostgreSQL).....	63
4.3.2. Servicio de Almacenamiento de Objetos (MinIO)	67
4.3.3. Servicio de Autenticación y Autorización Centralizada (Keycloak)	72
4.3.4. Servicio Firewall/Router (OPNsense)	80
4.3.5. Servicio DHCP (Gestionado por OPNsense).....	91
4.3.6. Servicio DNS (Gestionado por OPNsense y K3s CoreDNS)	98
4.3.7. Servicio de Orquestación de Contenedores (K3s).....	103
4.3.8. Servicio de Ingress y Gestión de Certificados (Traefik Proxy y Cert-Manager)	113
4.3.9. Servidor Web Frontend (Nginx + Svelte en K3s).....	120
4.3.10. Servidor API Backend (FastAPI en K3s)	128
4.4. Base de Datos	139
4.4.1. Modelo Entidad/Relación de la Aplicación OpenPaDi	140
4.4.2. Uso de la Base de Datos por Keycloak	142
4.5. Software y paquetes de Software	143
4.6. Infraestructura Hardware	145
4.6.1. Servidores e distribución de servicios	145
4.6.2. Estaciones de trabajo	152
4.7. Seguridad.....	152
5. PRESUPUESTO DA SOLUCIÓN.....	155
5.1. Desglose de Costos:.....	155
6. PROPUESTA DE MEJORAS	157
6.1. Ampliación y Escalabilidad de la Infraestructura	157
6.2. Incorporación de Nuevos Servicios y Funcionalidades	158
6.3. Implantación de Tecnologías Emergentes	159
6.4. Otras Mejoras y Elementos Pendientes	159
7. CONCLUSIONES.....	159
7.1. Logros Principales:.....	160

7.2. Desafíos y Aprendizajes:	163
7.3. Impacto y Valor:.....	164
8. BIBLIOGRAFÍA, ENLACES Y RECURSOS	166
8.1. Documentación Oficial de Tecnologías.....	166
8.2. Documentación Académica y Guías de Referencia	167
8.3. Repositorio del Proyecto	167

Preguntas rápidas para entender qué es OpenPaDi y qué abarca este proyecto

¿Qué es OpenPaDi?

OpenPaDi es una plataforma web pensada para facilitar la investigación en Humanidades, creando un espacio centralizado y abierto para el intercambio de transcripciones digitalizadas de fuentes históricas manuscritas.

Conceptualmente se puede dividir en dos partes:

- La **aplicación**, que incluye la interfaz web y las funcionalidades para que los usuarios finales puedan subir, consultar y descargar documentos, entre otras.
- La **Infraestructura** (PaaS), que comprende el conjunto de sistemas, redes y servicios (bases de datos, sistemas de almacenamiento de objetos, autenticación, orquestación de contenedores...) necesarios para que la aplicación funcione eficientemente, de modo seguro y escalable.

¿Cubre este proyecto la totalidad de OpenPaDi?

NO. Este proyecto se encargará del desarrollo e implementación de la Infraestructura PaaS.

¿Habrá una aplicación web en este proyecto?

Aunque desarrollar una aplicación web completa para OpenPaDi no es el objetivo de este trabajo, **SÍ** que se ha creado una aplicación, extremadamente básica, que cumple con las funcionalidades mínimas para probar que la Infraestructura desarrollada cumple con las necesidades de lo que sería la aplicación final y aquella le ofrece a esta una base sólida para su desarrollo total. Este prototipo de aplicación se encuentra alojado en el siguiente repositorio de GitHub, junto a los archivos de configuración más importantes de los servicios de la infraestructura: <https://github.com/robleslf/OpenPaDi>

1. Contextualización

1.1. Descripción del proyecto

El presente proyecto aborda el diseño e implementación de la infraestructura de sistemas y redes para OpenPaDi, una plataforma web concebida para la investigación en Humanidades. Aunque se describirá el concepto y se implementarán las funcionalidades más básicas de la aplicación web –que servirán para demostrar la armonía y el correcto funcionamiento de los componentes clave de la infraestructura–, el foco principal de este trabajo radica en los aspectos de la Administración de Sistemas Informáticos en Red necesarios para soportar dicha aplicación, y no tanto en el desarrollo de esta.

Esta plataforma nace con el propósito de solucionar un problema presente en el campo de las Humanidades. Una de las tareas más importantes en cualquier investigación en este ámbito es la búsqueda de fuentes, ya que determinan tanto la calidad como la profundidad de cualquier estudio que se haga. En la mayoría de los casos, sin embargo, esta se convierte en un trabajo extremadamente tedioso, en el que quien pretende realizar el estudio se embarca en una auténtica búsqueda a ciegas, pues estas fuentes son textos realizados en escrituras ilegibles para alguien ajeno al mundo de la Paleografía y la Diplomática

OpenPaDi pretende ser la solución a esto, ofreciendo al investigador una plataforma centralizada y abierta cuyo objetivo principal es facilitar el intercambio de transcripciones digitalizadas, de modo que:

- El investigador no requiera tener conocimientos en Paleografía y Diplomática para utilizar las fuentes.
- Aun teniendo dichos conocimientos, este no necesite gastar tiempo de su investigación en la transcripción de dichas fuentes y pueda centrarse en lo que realmente desea.
- Se evite transcribir fuentes que, una vez transcritas, sean desechadas porque no era lo que se buscaba.
- Se solventa el problema, al menos en parte, de la desorientación en la búsqueda de fuentes, al ofrecerse una clasificación de estas y un sistema de búsqueda basado en el contexto, fechas, lugar, etc. de estas.

Se trata, por tanto, de una herramienta digital que ofrece subida y descarga, consulta, así como un sistema clasificatorio que facilite la búsqueda y el filtrado de las diferentes fuentes a partir de criterios como la fecha, el lugar o los personajes o temas tratados en ellas, todo ello en un entorno digital seguro, y cuyo objetivo principal es la simplificación y mejora en la eficiencia de las investigaciones.

Está destinada, de este modo, a investigadores, estudiantes, paleógrafos, diplomáticos, historiadores, filólogos, así como a cualquier persona interesada en el acceso a fuentes históricas digitalizadas, sea cual sea su motivo, sin el requisito de poseer conocimientos técnicos en la transcripción de documentos antiguos.

Serán los propios transcriptores quienes suban digitalizados los documentos correspondientes a las fuentes transcritas, fomentando el libre intercambio de conocimiento.

Cómo requisitos básicos, la plataforma debe cumplir con:

- La existencia de un repositorio digital fácilmente accesible, de modo seguro, donde se permita el almacenamiento y la gestión de los documentos digitalizados correspondientes a las transcripciones realizadas.
- Una Base de Datos que permita un sistema de búsqueda que permita la localización de las fuentes deseadas de forma eficiente y eficaz.
- Control de accesos, cifrado de datos y demás mecanismos de seguridad.

1.2. Modelo de Negocio

La plataforma OpenPaDi, una iniciativa para la transcripción colaborativa y consulta de textos históricos, requiere para su operación una infraestructura tecnológica avanzada, concebida como una Plataforma como Servicio (PaaS).

Este desarrollo técnico es llevado a cabo por Muy Buenas Soluciones S.L., una empresa especializada en el diseño, implementación y gestión de soluciones de infraestructura tecnológica avanzada, incluyendo sistemas virtualizados de alto rendimiento y arquitecturas distribuidas seguras. Muy Buenas Soluciones S.L. ha sido seleccionada por la Fundación Patrimonio Digital (promotora de OpenPaDi) como su

socio tecnológico para asegurar que la aplicación OpenPaDi se despliegue sobre una base de infraestructura escalable, segura y con un alto rendimiento y disponibilidad.

Descripción de la empresa y su modelo de negocio:

Muy Buenas Soluciones S.L. es una firma de consultoría e ingeniería tecnológica enfocada en proveer soluciones de infraestructura personalizadas y plataformas PaaS gestionadas. Su objetivo es llevar a cabo la transformación digital de sus clientes, entre los que se incluyen desde instituciones culturales y académicas hasta empresas más tradicionales, facilitándoles el acceso a tecnologías actuales como la orquestación de contenedores, la virtualización avanzada y la configuración de redes seguras.

El modelo de negocio de Muy Buenas Soluciones S.L. se sustenta en la generación de ingresos a través de tres líneas principales de servicio:

- **Proyectos de Implementación de PaaS a Medida**, consistentes en el diseño y la construcción de plataformas de infraestructura personalizadas, adaptadas a los requisitos específicos de las aplicaciones y cargas de trabajo de cada cliente. El proyecto actual para la infraestructura de OpenPaDi ejemplifica este tipo de servicio.
- **Servicios Gestionados de Plataforma y Operaciones Continuas:** Una vez desplegada la PaaS, se ofrecen contratos de soporte técnico proactivo, mantenimiento evolutivo de la plataforma, monitorización 24/7, gestión integral de la seguridad y optimización continua del rendimiento.
- **Consultoría Estratégica en Arquitecturas Modernas de Infraestructura, Seguridad y DevOps**, brindando asesoramiento a organizaciones para guiar su adopción de arquitecturas, fortalecer su postura de seguridad y optimizar sus ciclos de desarrollo y operaciones TI.

Ubicación de la empresa:

La sede central de Muy Buenas Soluciones S.L. se encuentra ubicada en el Parque Tecnológico de León. Esta localización proporciona un entorno dinámico y favorable para la actividad tecnológica y la innovación, con acceso a infraestructuras

de comunicaciones de primer nivel, un ecosistema empresarial tecnológico consolidado y proximidad a talento cualificado.

Si bien la dirección y las operaciones principales se coordinan desde esta sede central, la empresa trabaja con un modelo que combina trabajo presencial con una amplia capacidad de trabajo remoto. Las infraestructuras PaaS para los clientes se despliegan según sus preferencias y requisitos: en sus propias instalaciones, sobre proveedores de nube pública, o en entornos de virtualización controlados para desarrollo y pruebas, como es el caso de este proyecto.

Posible previsión de crecimiento de la empresa:

Muy Buenas Soluciones S.L. se proyecta hacia un crecimiento sostenido, fundamentado en la creciente demanda del mercado por soluciones PaaS especializadas, la modernización de infraestructuras TI y la adopción de tecnologías de orquestación de contenedores y seguridad en entornos virtualizados y distribuidos.

Las líneas de expansión previstas incluyen:

- **Ampliación y Diversificación de la Base de Clientes:** Continuar con la penetración en sectores industriales y geográficos tradicionales que busquen transformar sus operaciones TI, y explorar activamente la entrada en nuevos nichos de mercado, como el de las Humanidades Digitales y el sector cultural.
- **PaaS Avanzado:** Ofrecer PaaS especializadas (datos, IA/ML) y reforzar los servicios de ciberseguridad.
- **Talento:** Fomentar un equipo de alta cualificación en tecnologías de infraestructura.

La colaboración con la Fundación Patrimonio Digital para la infraestructura de OpenPaDi representa la capacidad para entregar soluciones PaaS de alta complejidad, al mismo tiempo que consolida la entrada estratégica de la empresa en el campo de la tecnología aplicada a las Humanidades.

1.1.1. Descripción externa

Desde la perspectiva de sus potenciales clientes, Muy Buenas Soluciones S.L. oferta Plataformas como Servicio (PaaS) gestionada y a medida. Este producto permite a las organizaciones desplegar y operar sus aplicaciones web con eficiencia,

seguridad y escalabilidad, liberándolas de la complejidad de la gestión directa de la infraestructura.

Las principales capacidades que ofrecen incluyen:

- Virtualización y Orquestación automatizada de aplicaciones contenerizadas.
- Gestión avanzada de la red y la seguridad perimetral e interna.
- Provisión de servicios de *backend* esenciales y seguros (bases de datos, almacenamiento de objetos, autenticación).
- Sistemas integrales de monitorización del rendimiento y gestión de registros.
- Soporte técnico experto y consultoría para la optimización de aplicaciones sobre la plataforma.

Este servicio está dirigido a organizaciones de diversos sectores que buscan modernizar sus aplicaciones, mejorar la agilidad de sus operaciones TI y delegar la administración de infraestructuras complejas a un socio tecnológico especializado.

1.2.2. Descripción interna

Desde el punto de vista interno, Muy Buenas Soluciones S.L. se organiza para ofrecer servicios especializados de consultoría e implementación de Plataformas como Servicio (PaaS). La estructura departamental se ha diseñado para cubrir todas las fases del ciclo de vida de dichas soluciones de infraestructura.

Departamentos Principales:

1. Dirección General:

Encabezada por el CEO, define la estrategia global de la empresa, las relaciones institucionales y la toma de decisiones finales. Cuenta con el apoyo del CTO (Director de Tecnología) para la estrategia tecnológica, la innovación en las soluciones PaaS y la supervisión técnica general.

2. Departamento de Arquitectura y Consultoría de Soluciones PaaS:

Responsable de la fase inicial de interacción con el cliente y el diseño de las soluciones. Sus funciones incluyen el análisis de requisitos, el diseño de arquitecturas de plataforma PaaS a medida, la selección de los enfoques tecnológicos y la elaboración de las propuestas técnicas.

3. Departamento de Implementación de Sistemas e Infraestructura de Red:

Encargado de la construcción física y lógica de la infraestructura base. Esto incluye el despliegue y configuración de plataformas de virtualización, la instalación y configuración de sistemas operativos, y el diseño e implementación de la infraestructura de red y sus sistemas de seguridad.

4. Departamento de Orquestación y Servicios de Plataforma (PaaS Core):

Dedicado al despliegue, configuración y gestión de los clústeres de orquestación de contenedores. También se encarga de la integración y administración de los servicios gestionados que componen el núcleo de la PaaS, como las bases de datos, los sistemas de almacenamiento de objetos y las herramientas de autenticación.

5. Departamento de Operaciones, Seguridad y Fiabilidad de Plataformas:

Garantiza el funcionamiento, la seguridad y la resiliencia de las plataformas PaaS en producción. Sus responsabilidades incluyen la monitorización del rendimiento y la disponibilidad, la gestión de incidentes, la implementación y prueba de planes de *backup* y recuperación ante desastres, la optimización continua del rendimiento, y la aplicación de políticas y herramientas de seguridad en toda la plataforma.

6. Departamento Comercial y de Marketing:

Responsable de la estrategia de ventas, la captación de nuevos clientes para los servicios PaaS, la promoción de la empresa y la gestión de la comunicación y relaciones públicas.

7. Departamento de Administración y Recursos Humanos:

Gestiona los aspectos financieros, contables, legales y administrativos de la empresa. Además, se encarga de todos los procesos relacionados con el personal: selección, contratación, formación, desarrollo profesional y cultura organizacional.

A continuación, se presenta una estimación conceptual de la plantilla de Muy Buenas Soluciones S.L.:

Puesto	Departamento	N.º Empleados	Responsabilidades Principales
CEO (Director General)	Dirección General	1	Liderazgo estratégico, gestión global de la empresa, relaciones institucionales.
CTO (Director de Tecnología)	Dirección General	1	Estrategia tecnológica, innovación, supervisión técnica de soluciones PaaS, definición de arquitecturas de referencia.
Arquitecto de Soluciones PaaS	Arquitectura y Consultoría de Soluciones PaaS	2	Diseño de soluciones PaaS a medida para clientes, análisis de requisitos, liderazgo técnico en la fase de preventa y diseño.
Técnico Superior de Sistemas	Implementación de Sistemas e Infraestructura de Red	3	Instalación, configuración y mantenimiento de servidores (virtualización), sistemas operativos. Soporte técnico de la infraestructura base.
Técnico Especialista en Redes	Implementación de Sistemas e Infraestructura de Red	2	Diseño, implementación y gestión de la infraestructura de red (segmentación, routing, switching) y firewalls.
Especialista en Orquestación (Contenedores)	Orquestación y Servicios de Plataforma	3	Despliegue, configuración y gestión de clústeres de orquestación, y servicios PaaS (BBDD, almacenamiento).
Ingeniero de DevOps/Automatización	Orquestación y Servicios de Plataforma / Operaciones	1	Desarrollo de herramientas de automatización (IaC), integración de CI/CD,

			optimización de flujos de despliegue.
Ingeniero SRE (Fiabilidad del Sitio)	Operaciones, Seguridad y Fiabilidad de Plataformas	1	Monitorización avanzada, gestión de la disponibilidad, optimización del rendimiento, automatización de operaciones, gestión de incidentes.
Ingeniero de Seguridad (SecOps)	Operaciones, Seguridad y Fiabilidad de Plataformas	1	Implementación de políticas de seguridad, gestión de vulnerabilidades, respuesta a incidentes de seguridad, auditorías, y aseguramiento de la PaaS.
Consultor Comercial	Comercial y de Marketing	2	Desarrollo de negocio, gestión de la relación con clientes, venta de servicios PaaS y consultoría.
Especialista en Marketing y Comunicación	Comercial y de Marketing	1	Creación y ejecución de estrategias de marketing, gestión de la marca, comunicación externa.
Responsable de Administración y Finanzas	Administración y Recursos Humanos	1	Gestión financiera, contabilidad, legal y administrativa.
Especialista en Recursos Humanos	Administración y Recursos Humanos	1	Procesos de selección, contratación, formación, desarrollo del personal y cultura organizacional.

1.2.3. Viabilidad y expansión del negocio

La viabilidad de Muy Buenas Soluciones S.L. se fundamenta en la creciente demanda de modernización de infraestructuras TI y su especialización en soluciones PaaS personalizadas. El modelo de ingresos combina proyectos de implementación

con servicios gestionados recurrentes, asegurando un flujo financiero estable. Las probabilidades de éxito son altas, impulsadas por la adopción de tecnologías de contenedores y DevOps, aunque se enfrentan desafíos como la rápida evolución tecnológica y la competencia.

Plan y Previsión de Crecimiento de Muy Buenas Soluciones S.L.:

Corto Plazo (hasta 1 año):

- Consolidar la oferta PaaS base y metodologías, utilizando proyectos como OpenPaDi para generar casos de éxito y captar clientes iniciales, especialmente en nichos como Humanidades Digitales.

Medio Plazo (1-3 años):

- Intensificar la automatización en la entrega PaaS (Infraestructura como Código).
- Desarrollar ofertas PaaS especializadas para diferentes sectores o cargas de trabajo.
- Expandir la base de clientes y aumentar los ingresos por servicios gestionados.

Largo Plazo (3 años en adelante):

- Posicionarse como un referente en soluciones PaaS personalizadas y gestionadas.
- Fomentar la innovación continua en tecnologías de infraestructura y modelos de servicio para asegurar la sostenibilidad y el liderazgo en el mercado.

La viabilidad de Muy Buenas Soluciones S.L. también contempla el cumplimiento de sus obligaciones fiscales (IVA, Impuesto de Sociedades) y laborales (cotizaciones a la Seguridad Social, IRPF de empleados), así como la implementación de un plan básico de prevención de riesgos laborales acorde a su actividad de oficina y desarrollo tecnológico. Se explorará activamente la solicitud de ayudas y subvenciones destinadas a PYMES tecnológicas y proyectos de innovación para optimizar la inversión.

1.3. Descripción y objetivos del proyecto

El presente proyecto se centra en la creación de la infraestructura técnica para OpenPaDi, una plataforma concebida para abordar una necesidad fundamental en el campo de las Humanidades: la simplificación del acceso y trabajo con fuentes históricas manuscritas. Actualmente, la investigación se ve a menudo obstaculizada por la dificultad para localizar fuentes pertinentes y por la barrera que suponen las escrituras antiguas, que exigen conocimientos especializados en Paleografía y Diplomática. Este proceso no solo consume un tiempo valioso, sino que también puede llevar a esfuerzos de transcripción infructuosos si las fuentes finalmente no son relevantes. OpenPaDi nace con la motivación de paliar estos desafíos.

1.3.1. Estado actual de la problemática y soluciones existentes

La dificultad para acceder y trabajar con fuentes históricas transcritas es un desafío conocido en las Humanidades Digitales. Herramientas actuales como Transkribus o eScriptorium son muy útiles para el proceso de transcripción asistida por inteligencia artificial y el trabajo individual o en pequeños grupos sobre imágenes de documentos.

Sin embargo, estas soluciones no se centran prioritariamente en ofrecer un ecosistema abierto y centralizado para compartir, consultar y colaborar sobre un gran volumen de transcripciones ya completas o avanzadas. OpenPaDi busca cubrir este espacio, diferenciándose por ofrecer:

- Un repositorio centralizado y abierto de transcripciones de calidad.
- Un sistema de búsqueda avanzada por metadatos detallados sobre las transcripciones.
- Una interfaz diseñada para la colaboración en la revisión y mejora de transcripciones.
- Un espacio comunitario para la discusión y el apoyo entre usuarios.

A partir de este análisis, se definen los objetivos del proyecto OpenPaDi.

1.3.2. Objetivo general

El objetivo general del proyecto es diseñar, implementar y gestionar una infraestructura de sistemas y redes robusta y moderna, sobre la cual se desplegará y operará la plataforma web OpenPaDi. El objetivo es ofrecer un entorno colaborativo,

abierto, seguro y escalable que facilite y agilice la búsqueda y el análisis de fuentes históricas manuscritas, reduciendo los tiempos de investigación y democratizando el acceso al patrimonio histórico escrito.

1.3.3. Objetivos específicos

Para alcanzar el objetivo general del proyecto, se plantean los siguientes objetivos específicos, abarcando tanto la funcionalidad de la aplicación OpenPaDi como la infraestructura que la soporta:

A. Objetivos Funcionales de la Aplicación OpenPaDi (Soportados por la Infraestructura):

- **Gestión Integral de Transcripciones:** Permitir a los usuarios la subida, almacenamiento seguro, consulta y descarga de documentos históricos digitalizados y sus transcripciones textuales, facilitando así el acceso primario al contenido.
- **Descubrimiento Eficiente de Fuentes:** Implementar una base que permita implementar a la aplicación sistema de búsqueda y filtrado basado en metadatos descriptivos, para que los usuarios puedan localizar con precisión las transcripciones relevantes para su investigación.
- **Acceso Seguro mediante Autenticación:** Integrar un sistema robusto de gestión de identidades que controle el acceso a la plataforma y sus recursos, garantizando que solo usuarios autorizados puedan operar según sus permisos.

B. Objetivos Técnicos de la Infraestructura PaaS:

- **Adopción de Estándares y Código Abierto:** Construir la infraestructura prioritariamente con software libre y tecnologías estándar, para asegurar la transparencia, flexibilidad, interoperabilidad y sostenibilidad a largo plazo de la solución.
- **Arquitectura Modular y Orientada a Servicios:** Diseñar la plataforma con componentes independientes y bien definidos (servicios/microservicios), para facilitar el desarrollo, despliegue, escalado y mantenimiento individualizado de cada funcionalidad.

- **Alta Disponibilidad y Resiliencia mediante Redundancia:** Implementar redundancia en los componentes críticos de la infraestructura (red, orquestación, almacenamiento, servicios) y mecanismos de conmutación por error, con el fin de minimizar los puntos únicos de fallo y asegurar la continuidad del servicio ante incidentes.
- **Orquestación Automatizada de Contenedores:** Utilizar un sistema de orquestación avanzado para la gestión automatizada del ciclo de vida de las aplicaciones contenerizadas, optimizando el uso de recursos, el escalado y la auto-reparación.
- **Red Segura, Segmentada y Controlada:** Establecer una infraestructura de red con segmentación lógica (zonas aisladas) y un perímetro de seguridad robusto (firewall), para proteger los activos de información y controlar el flujo de datos.
- **Exposición Segura de Servicios Web:** Garantizar que el acceso externo a la aplicación se realice a través de un punto de entrada único y seguro, que gestione el tráfico cifrado (TLS) y los certificados digitales de forma eficiente.
- **Almacenamiento de Datos Persistente, Seguro y Fiable:** Proveer soluciones de almacenamiento (relacional para metadatos, objetos para archivos) que garanticen la integridad, confidencialidad, disponibilidad y durabilidad de toda la información gestionada por OpenPaDi, incluyendo capacidades de respaldo y redundancia.
- **Plan Efectivo de Continuidad del Negocio (Backup y Recuperación):** Definir, implementar y validar una estrategia de copias de seguridad automatizadas y un plan de recuperación ante desastres, para asegurar la capacidad de restaurar los servicios y datos críticos en caso de fallo mayor.
- **Soporte Funcional a la Aplicación OpenPaDi:** Asegurar que la PaaS desarrollada pueda alojar y servir de manera eficiente la interfaz de usuario y las funcionalidades básicas de la aplicación OpenPaDi, validando así la idoneidad de la infraestructura.

1.3.4. Contribución a la necesidad detectada

OpenPaDi contribuye significativamente a solventar la problemática descrita en el ámbito de la investigación histórica.

Principalmente, democratiza el acceso a las fuentes al permitir que investigadores y el público general puedan consultar transcripciones sin necesidad de poseer conocimientos especializados en Paleografía o Diplomática.

Además, optimiza el valioso tiempo de los investigadores, ya que facilita el acceso directo a transcripciones existentes. Esto reduce considerablemente el tiempo que de otra manera se dedicaría a la transcripción manual y a la laboriosa tarea de evaluar la pertinencia de cada fuente antes de su estudio en profundidad.

Finalmente, al concebirse como una plataforma abierta, OpenPaDi fomenta la mejora continua de la calidad de las transcripciones. Esto se logra al promover la revisión por pares y las aportaciones colaborativas de la comunidad de usuarios, quienes pueden enriquecer y corregir el material existente.

1.3.5. Oportunidad y Modelo de Financiación de la Plataforma OpenPaDi

La plataforma OpenPaDi, al ser un proyecto con un fuerte componente de servicio a la comunidad académica y cultural, presenta una oportunidad significativa para la colaboración y el avance en las Humanidades Digitales.

Su modelo de financiación se concibe de forma mixta para asegurar su sostenibilidad y crecimiento:

- **Acceso Básico Gratuito:** Las funcionalidades esenciales de consulta, subida de transcripciones y participación comunitaria se ofrecerán de forma gratuita para fomentar la máxima adopción y el intercambio de conocimiento.
- **Servicios de Valor Añadido para Instituciones:** Se podrían explorar modelos de suscripción o acuerdos de colaboración con instituciones (universidades, archivos, centros de investigación) para ofrecerles funcionalidades avanzadas, soporte prioritario o servicios de integración personalizados.
- **Financiación mediante Subvenciones y Patrocinios:** Se buscará activamente el apoyo de organismos públicos y privados, fundaciones e instituciones interesadas en la preservación del patrimonio documental y la promoción de la investigación en Humanidades, para asegurar la continuidad y evolución del proyecto.

Este enfoque permitiría mantener el carácter abierto y accesible de OpenPaDi para la comunidad global, mientras se obtienen los recursos necesarios para su mantenimiento, desarrollo y expansión a través del apoyo institucional.

2. ANÁLISIS

2.1. Requisitos de la solución técnica

Estos son los diferentes requisitos necesarios para la solución técnica de OpenPaDi:

Requisito 1: Fundamentación en Software de Código Abierto y Estándares Consolidados.

Como principio base, la plataforma se construirá utilizando predominantemente software libre y de código abierto, o tecnologías que se adhieran a estándares industriales reconocidos, con el fin de priorizar la transparencia, la flexibilidad para la adaptación, la interoperabilidad y el acceso a un amplio soporte comunitario, sentando las bases para una solución fiable y personalizable.

Requisito 2: Diseño Arquitectónico Modular y Orientado a Servicios.

Sobre la base del software abierto, la arquitectura general de la plataforma deberá ser modular. Se priorizará un diseño orientado a servicios o microservicios, de modo que se facilite el despliegue independiente, la escalabilidad modular y la resiliencia de los distintos componentes funcionales.

Requisito 3: Diseño para la Alta Disponibilidad y Redundancia.

La plataforma PaaS debe ser diseñada e implementada para ofrecer altos niveles de disponibilidad del servicio. Esto se logrará mediante la aplicación de redundancia en los niveles críticos de la infraestructura: componentes de red, nodos del clúster de orquestación, e instancias de servicios de backend (bases de datos, almacenamiento), junto con mecanismos de balanceo de carga. El sistema debe ser capaz de tolerar fallos en componentes individuales sin una interrupción significativa del servicio.

Requisito 4: Implementación mediante Contenerización y Orquestación Avanzada.

Con el fin de materializar la modularidad y facilitar la gestión del ciclo de vida de los servicios, así como soportar la redundancia y el escalado, se requiere el uso de tecnologías de contenerización. Estos contenedores serán gestionados por un sistema de orquestación robusto, capaz de automatizar despliegues, escalado, balanceo de carga interno, auto-reparación y descubrimiento de servicios.

Requisito 5: Infraestructura de Red Segura, Segmentada y con Acceso Controlado.

La plataforma de orquestación y todos sus servicios asociados operarán sobre una infraestructura de red diseñada con la seguridad como prioridad. Esto implica una segmentación lógica estricta para aislar componentes críticos y la implementación firewall que controle y filtre todo el tráfico entre segmentos y con el exterior, considerando también la redundancia en los enlaces de red y dispositivos clave.

Requisito 6: Exposición Segura y Eficiente de los Servicios Web.

El acceso externo a los servicios web de la aplicación OpenPaDi se gestionará a través de un punto de entrada único, seguro y potencialmente redundante, como un proxy inverso o controlador de ingreso. Este sistema será responsable de la terminación de conexiones cifradas (TLS), la gestión automatizada de certificados digitales y el enrutamiento inteligente del tráfico.

Requisito 7: Gestión Centralizada y Segura de Identidades y Accesos.

Para proteger tanto la aplicación como la propia PaaS, se implementará un sistema centralizado de gestión de identidades. Este permitirá la autenticación robusta de usuarios y servicios.

Requisito 8: Soluciones de Almacenamiento Persistente, Fiable, Seguro y Redundante para Datos.

La plataforma debe proveer soluciones de almacenamiento diferenciadas para los datos de OpenPaDi: un sistema de base de datos relacional para información estructurada y un sistema de almacenamiento de objetos para archivos de gran

tamaño. Ambos deben incluir mecanismos de seguridad de acceso, capacidades de respaldo y opciones de redundancia o replicación para asegurar la durabilidad y disponibilidad de los datos.

Requisito 9: Estrategia Robusta de Copias de Seguridad y Plan de Recuperación.

Se definirá e implementará una estrategia completa de copias de seguridad para todos los datos críticos y configuraciones esenciales de la PaaS y la aplicación OpenPaDi. Esto incluirá la automatización de los *backups*, su almacenamiento seguro y un plan documentado y probado (en la medida de lo posible) para la recuperación de los servicios ante diversos escenarios de fallo, considerando la restauración en entornos redundantes.

Requisito 10: Capacidad para Soportar una Interfaz de Usuario Funcional.

Finalmente, la PaaS debe demostrar su capacidad para alojar y servir eficientemente la interfaz de usuario de la aplicación web, permitiendo una interacción fluida para las funcionalidades básicas de la aplicación, incluso bajo escenarios de alta disponibilidad

Por tanto, podemos hacernos una idea de cómo sería OpenPaDi a nivel conceptual:



2.2. Análisis de Riesgos e Interesados

Este análisis identifica a las partes interesadas, involucradas o afectadas por el diseño, implementación y operación de la infraestructura PaaS de OpenPaDi. Asimismo, se evalúan los principales riesgos asociados a la provisión de este servicio.

2.2.1. Identificación de Interesados Clave

La siguiente tabla detalla los principales interesados y su relación con el proyecto de infraestructura PaaS:

Interesado	Rol y Relación con el Proyecto PaaS	Intereses y Expectativas Principales sobre la PaaS
Fundación Patrimonio Digital (Cliente)	Entidad que contrata a Muy Buenas Soluciones S.L. para el desarrollo y (potencialmente) gestión de la PaaS sobre la que operará su aplicación OpenPaDi.	Una plataforma estable, segura, de alto rendimiento y escalable para OpenPaDi. Cumplimiento de plazos y presupuesto. Soporte técnico eficaz. Buen retorno de la inversión.
Equipo Técnico de Muy Buenas Soluciones S.L. (Ejecutor del Proyecto)	Responsables del diseño, implementación, gestión y mantenimiento de la PaaS.	Entrega exitosa de una solución técnica robusta y que cumpla los requisitos. Adquisición de experiencia. Satisfacción del cliente. Cumplimiento de objetivos del proyecto. Viabilidad técnica y operativa.
Usuarios Finales de OpenPaDi (Investigadores, académicos, comunidad)	Utilizarán la aplicación OpenPaDi, cuya experiencia depende de la calidad de la PaaS subyacente.	Acceso rápido, fiable y seguro a la aplicación OpenPaDi. Que la aplicación funcione sin interrupciones y con buen rendimiento. Protección de sus datos.
Instituciones Culturales y Académicas (Universidades, Archivos, Bibliotecas, Centros de Investigación)	Potenciales adoptantes y beneficiarios de la aplicación OpenPaDi; posibles futuros clientes para servicios PaaS similares de Muy Buenas Soluciones S.L.	Disponibilidad de herramientas digitales innovadoras como OpenPaDi. Infraestructuras fiables que soporten la investigación y la difusión del patrimonio. Soluciones tecnológicas adaptadas a sus necesidades.
Dirección de Muy Buenas Soluciones S.L.	Responsables de la estrategia, rentabilidad y reputación de la empresa.	Éxito del proyecto como caso de estudio y referencia. Rentabilidad del contrato. Fortalecimiento de la posición

		de la empresa en el mercado PaaS.
Proveedores de Tecnologías Base (Software Open Source, Hardware)	Entidades o comunidades que desarrollan el software o fabrican el hardware utilizado para construir la PaaS.	Uso adecuado y ético de sus tecnologías. Contribuciones a la comunidad (si se realizan). Feedback sobre el uso de sus productos.
Organismos Reguladores	Entidades gubernamentales o sectoriales que establecen normativas de obligado cumplimiento (ej. protección de datos, seguridad).	Cumplimiento de la PaaS y de la aplicación alojada con todas las leyes y regulaciones aplicables.

2.2.2. Identificación, Evaluación y Mitigación de Riesgos para la Infraestructura PaaS

La provisión de un servicio PaaS implica diversos riesgos que deben ser gestionados constantemente. A continuación, se presenta una tabla con los principales riesgos identificados para la infraestructura de OpenPaDi, su impacto potencial, probabilidad estimada y las estrategias de mitigación propuestas por Muy Buenas Soluciones S.L.

Para la evaluación de los riesgos, se han definido los siguientes niveles:

Niveles de Impacto:

- **Crítico:** Consecuencias severas que podrían llevar a la interrupción total y prolongada del servicio PaaS, pérdida irreparable de datos del cliente, graves incumplimientos legales o regulatorios con posibles sanciones significativas, o un daño reputacional extremo para Muy Buenas Soluciones S.L. que amenace la viabilidad del contrato o de la empresa.
- **Alto:** Consecuencias significativas que causarían una degradación importante o interrupción parcial del servicio PaaS, pérdida de datos recuperable con esfuerzo considerable, incumplimientos normativos con posibles sanciones moderadas, o un impacto negativo notable en la satisfacción del cliente y la reputación de Muy Buenas Soluciones S.L.
- **Medio:** Consecuencias moderadas que podrían generar una degradación menor o interrupciones breves del servicio PaaS, dificultades operativas

temporales, o un impacto limitado en funcionalidades específicas de la plataforma, resolubles sin comprometer el servicio global a largo plazo.

- **Bajo:** Consecuencias leves con un impacto mínimo o nulo en la operatividad del servicio PaaS, inconvenientes menores o problemas fácilmente subsanables que no afectan de manera relevante la funcionalidad principal ni la percepción del cliente.

Niveles de Probabilidad:

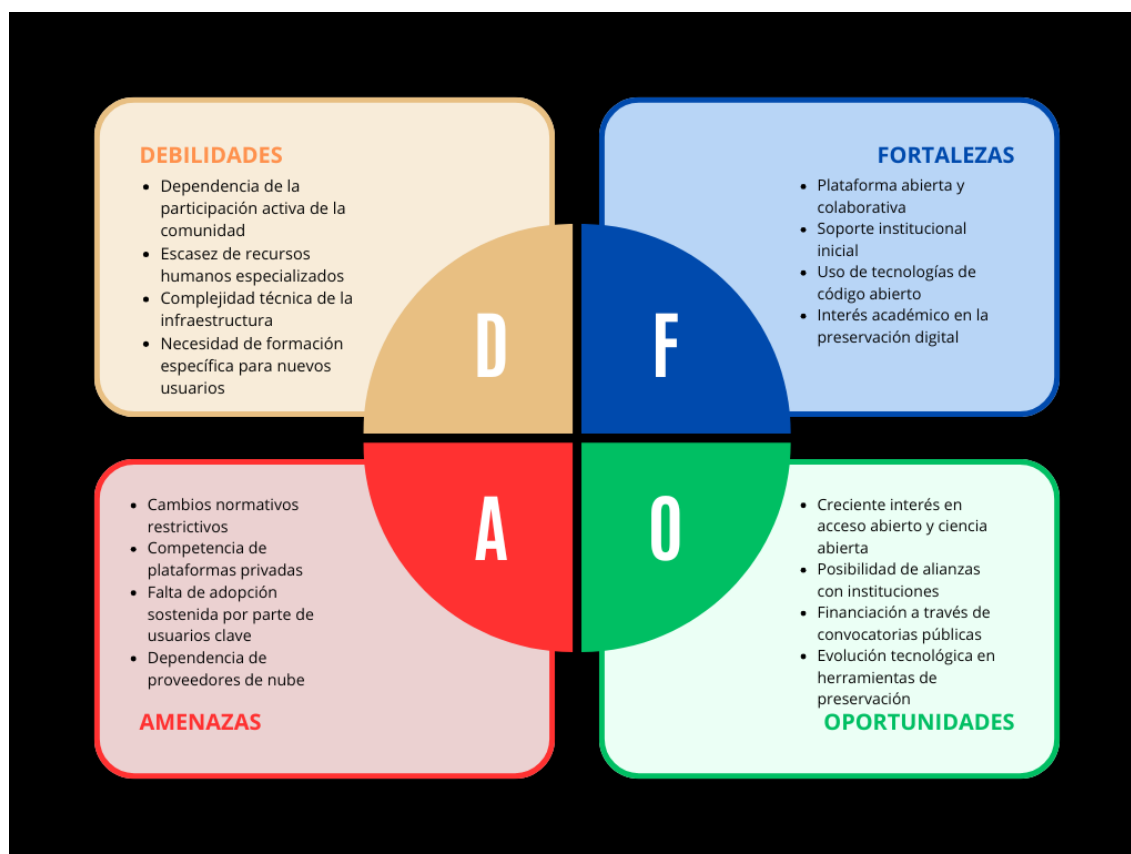
- **Alta:** Es muy probable que el riesgo se materialice durante el ciclo de vida del proyecto o la operación del servicio si no se aplican medidas de mitigación.
- **Media:** Existe una posibilidad razonable de que el riesgo ocurra; no es seguro, pero tampoco improbable.
- **Baja:** Es poco probable que el riesgo se materialice, aunque no imposible; podría ocurrir en circunstancias excepcionales.

Riesgo para la Infraestructura PaaS	Impacto	Probabilidad	Estrategias de Mitigación Propuestas
Fallos de seguridad en la PaaS comprometiendo datos o servicios de OpenPaDi.	Crítico	Medio	Diseño de red segmentada con firewalls; cifrado de datos en tránsito y reposo; gestión de y accesos; monitorización de seguridad continua; plan de respuesta a incidentes; auditorías de seguridad.
Baja disponibilidad del servicio PaaS (caídas frecuentes o prolongadas) debido a fallos de hardware, software o configuración.	Crítico	Medio	Diseño para alta disponibilidad: redundancia en componentes críticos (nodos de orquestación, servicios de backend, enlaces de red, dispositivos de seguridad); mecanismos de

			conmutación por error (failover) automáticos; pruebas periódicas de failover; monitorización proactiva de la salud del sistema.
Rendimiento deficiente de la PaaS que degrade la experiencia de usuario de la aplicación OpenPaDi	Alto	Medio	Planificación de capacidad y pruebas de carga/estrés; monitorización continua del rendimiento de todos los componentes; optimización de configuraciones de servicios (BBDD, almacenamiento, orquestador); diseño escalable (horizontal y vertical) de la infraestructura.
Pérdida de datos críticos de OpenPaDi (bases de datos, almacenamiento de objetos) alojados en la PaaS.	Crítico	Bajo	Estrategia de backup robusta: copias de seguridad automatizadas y frecuentes; almacenamiento de backups en ubicaciones física o lógicamente separadas y seguras; pruebas periódicas de restauración; políticas de retención de datos.
Dificultad para escalar la PaaS para soportar el crecimiento en usuarios o datos de la aplicación OpenPaDi.	Medio	Medio	Política de gestión de parches y actualizaciones; selección de software con soporte a largo plazo (LTS) y comunidad activa; vigilancia tecnológica continua; plan de migración o sustitución

			para componentes que lleguen al final de su vida útil.
Incumplimiento de normativas de protección de datos	Crítico	Bajo	Diseño de la PaaS aplicando principios de privacidad por defecto y desde el diseño; implementación de medidas técnicas y organizativas para el cumplimiento; asesoramiento legal si es necesario; formación del personal.
Pérdida del cliente principal (Fundación Patrimonio Digital) debido a insatisfacción con el servicio PaaS o cambios en su estrategia.	Alto	Bajo	Mantener una comunicación fluida y transparente con el cliente; cumplimiento estricto de los Acuerdos de Nivel de Servicio; ofrecer un soporte técnico de alta calidad; flexibilidad para adaptar la PaaS a necesidades evolutivas del cliente.
Complejidad en la gestión y operación de la PaaS que supere las capacidades del equipo técnico de Muy Buenas Soluciones S.L.	Medio	Medio	Inversión en formación continua del equipo; documentación exhaustiva de la arquitectura y procedimientos operativos; fomento de la automatización (Infraestructura como Código); uso de herramientas de gestión y monitorización que simplifiquen la operación.

Tras este análisis (interesados y riesgos), se ha realizado un análisis DAFO (Debilidades, Amenazas, Fortalezas y Oportunidades), cuyo objetivo principal es el de ofrecer una visión global a nivel estratégico de OpenPaDi. Con él podemos identificar los principales elementos internos y externos que pueden influir, positiva o negativamente, en el desarrollo de nuestro proyecto, sirviéndonos como base para la toma de decisiones futuras:



3. Planificación temporal y de recursos de las distintas fases

3.1. Identificación de fases

El éxito de OpenPaDi dependerá en gran medida de una planificación rigurosamente secuenciada. En este apartado se detallarán las fases de implantación del sistema, estableciendo objetivos claros y tareas concretas, todo ello acompañado de sus duraciones estimadas.

Se acompaña cada tarea con una serie de herramientas que, en todo caso, no serán definitivas y pueden ser remplazadas por otras similares que se adapten mejor a las necesidades que en esta fase no hayan sido tenidas en cuenta.

Fase 1: Diseño y Configuración de la Plataforma de Virtualización y Red Base	
Objetivo	Establecer la infraestructura de virtualización fundamental y la segmentación de red inicial.
Tareas clave	• Elegir, instalar y configurar el hipervisor. • Diseñar la topología de red, incluyendo la definición de redes virtuales (VLANs). • Configurar el enrutamiento básico entre VLANs y el acceso a la red externa. • Preparar plantillas de máquinas virtuales base.
Herramientas a considerar	• Hipervisores: Proxmox VE, VMware. • Sistemas Operativos para VMs: Ubuntu Server, Debian.
Duración aproximada	25 horas

Fase 2: Implementación de la Seguridad Perimetral y Control de Acceso a la Red

Objetivo	Asegurar el perímetro de la red y controlar el flujo de tráfico entre las zonas definidas.
Tareas clave	• Desplegar y configurar un dispositivo firewall virtualizado. • Implementar reglas de firewall detalladas para controlar el tráfico entre VLANs e Internet. • Configurar servicios de red esenciales (DHCP, DNS) para las VLANs.
Herramientas a considerar	• Firewalls: OPNsense, pfSense, VyOS.
Duración aproximada	20 horas

Fase 3: Despliegue y Configuración de la Plataforma de Orquestación de Contenedores

Objetivo	Crear un entorno robusto y escalable para la gestión de aplicaciones contenerizadas.
Tareas clave	• Desplegar los nodos del clúster de orquestación sobre la plataforma de virtualización. • Configurar la red del clúster y el almacenamiento persistente para los contenedores. • Instalar y configurar el controlador de ingreso para la

	gestión del tráfico entrante a las aplicaciones. • Implementar un sistema para la gestión automatizada de certificados TLS.
Herramientas a considerar	• Orquestadores: Kubernetes (distribuciones como K3s, RKE2, o vanilla), Docker Swarm, HashiCorp Nomad. • Soluciones de Almacenamiento para Contenedores: Longhorn, Rook (Ceph), OpenEBS, o drivers CSI para almacenamiento local/NFS. • Ingress Controllers: Traefik, Nginx Ingress, HAProxy Ingress. • Gestores de Certificados: Cert-Manager.
Duración aproximada	30 horas

Fase 4: Implementación de Servicios de Backend Fundamentales de la PaaS

Objetivo	Desplegar y asegurar los servicios de persistencia de datos y autenticación necesarios para las aplicaciones.
Tareas clave	• Desplegar y configurar un sistema de gestión de bases de datos relacionales, incluyendo

	la configuración de seguridad, acceso y backups iniciales. • Desplegar y configurar un sistema de almacenamiento de objetos, asegurando su acceso y creando los repositorios necesarios. • Desplegar y configurar un servicio de gestión de identidad y acceso centralizado, integrándolo con la base de datos persistente.
Herramientas a considerar	• Bases de Datos Relacionales: PostgreSQL, MySQL, MariaDB. • Almacenamiento de Objetos: MinIO, Ceph RGW, OpenIO. • Servicios de Identidad: Keycloak, Dex, o soluciones personalizadas con OAuth2/OIDC
Duración aproximada	25 horas

Fase 5: Despliegue y Validación de la Aplicación de Prueba (OpenPaDi) sobre la PaaS

Objetivo	Desplegar los componentes de la aplicación OpenPaDi en la PaaS para validar la funcionalidad de la infraestructura y sus servicios.
-----------------	---

Tareas clave	• Adaptar o crear los manifiestos de despliegue para el frontend y backend/API. • Configurar la aplicación para utilizar los servicios de backend (base de datos, almacenamiento de objetos, autenticación) provistos por la PaaS. • Realizar pruebas funcionales básicas para asegurar la correcta comunicación y operación de la aplicación sobre la nueva infraestructura.
Herramientas a considerar	• Lenguajes/Frameworks Backend: Python (FastAPI), Node.js, Go. • Lenguajes/Frameworks Frontend: JavaScript (Svelte, React, Vue). • Contenerización: Docker.
Duración aproximada	15 horas

Fase 6: Establecimiento de la Estrategia de Backup, Recuperación y Hardening Final

Objetivo	Asegurar la resiliencia de los datos y la plataforma, y aplicar medidas de seguridad finales.
Tareas clave	• Implementar y automatizar los procedimientos de backup para datos críticos (bases de datos,

	almacenamiento de objetos) y configuraciones de la plataforma (orquestrador, VMs). • Documentar y probar (simulacro) un plan básico de recuperación ante desastres.
Herramientas a considerar	• Backup de VMs: Proxmox Backup Server, scripts personalizados. • Backup de Datos de Aplicación: pg_dump/pg_basebackup (PostgreSQL), mc mirror (MinIO), Velero (para K8s), Restic, BorgBackup. • Herramientas de Hardening: Lynis, OpenSCAP, guías CIS.
Duración aproximada	5 horas

3.2. Identificación de recursos

Para la puesta en marcha del proyecto de infraestructura PaaS para OpenPaDi, se requiere una combinación de recursos personales cualificados y recursos materiales (hardware y software). A continuación, se detallan los principales recursos identificados:

3.2.1. Recursos Personales

La ejecución de este proyecto será llevada a cabo por un equipo técnico compacto y altamente cualificado, donde los miembros asumen múltiples responsabilidades a lo largo de las diferentes fases. Para el desarrollo de este proyecto se estima la participación de dos personas, que serán seleccionadas de entre los recursos humanos de Muy Buenas Soluciones S.L. definidos anteriormente:

Perfil	Nº de personas	Fases de participación y rol
--------	----------------	------------------------------

Líder Técnico del Proyecto y Arquitecto de la PaaS	1	Todas las fases: Liderazgo técnico, diseño arquitectónico global de la PaaS, selección de tecnologías, supervisión y ejecución de la implementación (virtualización, red, orquestación, servicios backend), configuración de monitorización, backups y seguridad. Elaboración de documentación.
Técnico de Infraestructura y Soporte	1	Principalmente en Fases 1 y 2: Soporte en la instalación y configuración de la infraestructura base (VMs, red inicial, firewall). Asistencia en tareas de mantenimiento y pruebas bajo la dirección del Líder Técnico.

3.2.2. Recursos Materiales

Hardware Necesario:

- Servidores Físicos para Virtualización:
 - Número: Un mínimo de tres servidores físicos host para la plataforma de virtualización. Esta configuración permite la creación de un clúster de virtualización que soporte la migración en vivo de máquinas virtuales y la tolerancia a fallos a nivel de host (si un servidor físico falla, las VMs pueden reiniciarse o migrarse a los otros hosts).
 - Especificaciones por Servidor (Mínimas Recomendadas):
 - CPU: 2 Procesadores multi-núcleo modernos (16-32 núcleos físicos totales por servidor) con soporte para virtualización por hardware (Intel VT-x/AMD-V).
 - RAM: Mínimo 128 GB de RAM ECC por servidor, idealmente 256 GB o más, para alojar múltiples máquinas virtuales con cargas de trabajo diversas.

- Almacenamiento Local (para SO del Hipervisor y VMs críticas): Discos SSD de alta velocidad y fiabilidad en configuración RAID 1 o similar para el sistema operativo del hipervisor y, opcionalmente, para VMs que requieran latencia mínima.
 - Conectividad de Red: Múltiples interfaces de red de alta velocidad (ej. 4 x 10GbE o superior por servidor), configuradas para redundancia y agregación de enlaces para el tráfico de datos, gestión y almacenamiento.
 - Fuentes de Alimentación Redundantes.
 - Interfaz de Gestión Remota (BMC/IPMI).
- Sistema de Almacenamiento Centralizado (SAN/NAS de Alto Rendimiento):
 - Propósito: Para el almacenamiento principal de las máquinas virtuales y, potencialmente, para los datos de los servicios de almacenamiento de objetos y backups.
 - Características: Conectividad redundante, múltiples controladoras para HA, capacidad de expansión, y discos SSD o una combinación híbrida (SSD/HDD) para equilibrar rendimiento y capacidad.
- Equipamiento de Red Físico:
 - Switches Gestionables L2/L3:
 - Número: Mínimo dos switches de núcleo/agregación para redundancia. Cada servidor físico se conectaría a ambos switches.
 - Características: Soporte para VLANs (802.1Q), agregación de enlaces (LACP) y capacidad de conmutación adecuada para el tráfico previsto. Puertos de alta velocidad (10GbE o superior) para los servidores y enlaces inter-switch.
 - Routers/Firewalls Físicos Perimetrales (Opcional, si no se virtualiza):
 - Número: Idealmente dos dispositivos en clúster HA para la conexión a Internet y la protección perimetral principal.

- Alternativa: Virtualizar esta función en la plataforma de hipervisión, utilizando VMs dedicadas para el firewall, lo que requiere que los servidores host tengan interfaces de red dedicadas para la conexión WAN y las diferentes VLANs.
- Sistema de Alimentación Ininterrumpida (SAI/UPS):
 - Unidades SAI con capacidad suficiente para soportar todo el equipamiento crítico (servidores, almacenamiento, switches) durante cortes de energía, permitiendo un apagado ordenado o la continuidad con generadores.

Software:

La selección de software se priorizará hacia soluciones de código abierto, robustas y con amplio soporte comunitario, para asegurar la flexibilidad, transparencia y sostenibilidad de la plataforma. Las categorías de software necesarias son:

- Plataforma de Virtualización de Servidores.
- Sistemas Operativos para Servidores Virtuales.
- Software de Firewall/Router (virtualizado).
- Plataforma de Orquestación de Contenedores.
- Motor de Contenerización.
- Controlador de Ingreso / Proxy Inverso.
- Sistema de Gestión de Certificados TLS.
- Sistema de Gestión de Identidad y Acceso Centralizado.
- Sistema de Gestión de Bases de Datos Relacionales (SGBDR).
- Sistema de Almacenamiento de Objetos.

3.3. Diagrama de Gant

En el siguiente diagrama de Gantt se ilustra la distribución de horas para cada una de las fases descritas en el apartado anterior a lo largo de los días de desarrollo del proyecto:

	MAYO																															JUNIO														
	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15									
FASE 1	2	5	3	3	3	4					1		2	2																																
FASE 2					1		2	2	5		2	2																3	3																	
FASE 3							2	2		3	1	1	3	2	5	5							1	1	1	1	1	1																		
FASE 4																		3	2	2	4	3	3	3	1	1	2	1																		
FASE 5																						5	3	2	2	1	1	1																		
FASE 6																																1	1	1	1	1										

Diagrama de Gantt para la implementación de las fases del proyecto OpenPaDi

4. DISEÑO DE LA SOLUCIÓN

4.1. Resumen de la Solución Técnica Propuesta

El presente capítulo detalla el diseño técnico de la infraestructura integral de sistemas y redes que se implementará para dar soporte a la aplicación web colaborativa OpenPaDi. La solución se materializa como una Plataforma como Servicio (PaaS) privada, diseñada con un fuerte enfoque en la seguridad, la alta disponibilidad mediante redundancia, la escalabilidad y la eficiencia operativa. Esta PaaS se construirá sobre la plataforma de virtualización Proxmox VE, que permitirá gestionar de forma centralizada y eficiente todos los componentes virtualizados.

La arquitectura de la solución se fundamenta en los siguientes componentes y tecnologías clave:

1. Plataforma de Virtualización y Red Base:

La infraestructura se desplegará sobre Proxmox VE, configurado idealmente en un clúster para la alta disponibilidad de las máquinas virtuales. Sobre esta base, se establecerá una arquitectura de red lógica segmentada mediante Redes Virtuales de Área Local (VLANs). La seguridad perimetral y el enrutamiento inter-VLAN serán gestionados por una máquina virtual dedicada ejecutando OPNsense. Se implementarán servicios de red esenciales como DHCP y DNS de forma centralizada y con redundancia.

2. Orquestación de Contenedores y Servicios de Aplicación:

Un clúster K3s (distribución ligera de Kubernetes), desplegado sobre máquinas virtuales en Proxmox, orquestrará los contenedores Docker que alojan los servicios de la aplicación OpenPaDi. Esto incluye:

- El Frontend Svelte, servido por Nginx.
- El Backend API FastAPI.
- El servicio de autenticación centralizada Keycloak. El clúster K3s se diseñará con nodos master y worker para permitir la escalabilidad y resiliencia de estos servicios.

3. Gestión de Tráfico Entrante y Seguridad de Aplicación:

El tráfico HTTP/HTTPS destinado a la aplicación OpenPaDi y a Keycloak será gestionado por Traefik Proxy como Ingress Controller, integrado con K3s.

Traefik se encargará de la terminación TLS/SSL (utilizando certificados gestionados por Cert-Manager) y del enrutamiento inteligente de las peticiones a los servicios correspondientes. Este componente residirá en una Zona Desmilitarizada (DMZ) virtual para mayor seguridad.

4. Servicios de Backend Persistentes:

Los datos críticos de la aplicación y de los servicios de la plataforma se gestionarán mediante:

- Un servidor de base de datos PostgreSQL, ejecutándose en una máquina virtual dedicada, para almacenar los metadatos de OpenPaDi y la configuración de Keycloak. Se implementarán mecanismos de replicación para alta disponibilidad.
- Un servidor de almacenamiento de objetos MinIO, también en una máquina virtual dedicada o en configuración de clúster, para guardar los documentos históricos digitalizados y otros archivos. Se configurará para redundancia de datos.

5. Monitorización, Logging y Continuidad del Negocio:

Aunque no se implementarán directamente los servicios de monitorización, ya que escapan de la magnitud del proyecto, se creará una VLAN dedicada a servicios de este tipo para una futura implementación. Además, se implementará una estrategia robusta de copias de seguridad automatizadas para las VMs, bases de datos, almacenamiento de objetos y configuraciones del clúster K3s, junto con un plan de recuperación ante desastres.

Justificación del Enfoque de Infraestructura

La arquitectura técnica propuesta se ha seleccionado estratégicamente para cumplir con los requisitos de una aplicación web colaborativa moderna, priorizando la seguridad, resiliencia, escalabilidad y eficiencia operativa dentro de un entorno de PaaS privada controlada.

La elección de Proxmox VE como base de virtualización ofrece una gestión centralizada y eficiente de los recursos, con la posibilidad de clúster para alta disponibilidad a nivel de VM. Sobre esta, la segmentación de red con VLANs y un

firewall dedicado (OPNsense) establece un perímetro de seguridad robusto y control detallado del tráfico.

La orquestación de los servicios de aplicación (frontend, backend, autenticación) mediante K3s y contenedores Docker se justifica por su capacidad para ofrecer despliegues ágiles, auto-reparación, escalado horizontal y una gestión de ciclo de vida simplificada. K3s proporciona la potencia de Kubernetes sin su complejidad total, ideal para este alcance.

Traefik Proxy como Ingress Controller con Cert-Manager se elige por su gestión dinámica del tráfico, terminación SSL/TLS automatizada y su integración nativa con Kubernetes, simplificando la exposición segura de los servicios. Su ubicación en una DMZ virtual refuerza la seguridad.

Para los datos persistentes, la elección de PostgreSQL con replicación y MinIO para almacenamiento de objetos garantiza la integridad, disponibilidad y capacidad de recuperación de los datos críticos, utilizando soluciones de código abierto probadas y escalables.

Este enfoque integral, que combina virtualización, orquestación de contenedores y servicios de *backend* resilientes, proporciona el equilibrio óptimo entre control, agilidad y fiabilidad para OpenPaDi.

4.2. Diseño de la Red

El diseño de la red constituye un pilar esencial para la infraestructura PaaS de OpenPaDi, definiendo la estructura de comunicación, la segmentación de seguridad y la base para el rendimiento y la alta disponibilidad de todos los servicios. La solución de red se implementará sobre la plataforma de virtualización Proxmox VE, aprovechando sus capacidades para crear una topología de red virtualizada compleja y eficiente, la cual se apoya conceptualmente en una infraestructura física subyacente robusta.

4.2.1. Consideraciones sobre la Infraestructura Física de Red Subyacente

Para un despliegue de producción con los niveles de alta disponibilidad y rendimiento deseados, la plataforma Proxmox VE se asentaría sobre una infraestructura física que incluiría:

- **Servidores Físicos en Clúster:** Múltiples servidores físicos (mínimo tres) interconectados, formando un clúster Proxmox VE.
- **Switches Físicos Redundantes:** Al menos dos switches físicos gestionables conectados mediante enlaces redundantes y, potencialmente, agregados (LACP).
- **Almacenamiento de Red Compartido:** Un sistema de almacenamiento (SAN, NAS o SDS como Ceph) accesible por todos los hosts Proxmox, con conectividad de red redundante y de alto rendimiento, esencial para la alta disponibilidad de las máquinas virtuales.
- **Conectividad WAN Múltiple:** Idealmente, conexiones a dos Proveedores de Servicios de Internet (ISPs) distintos para garantizar la continuidad del acceso externo.

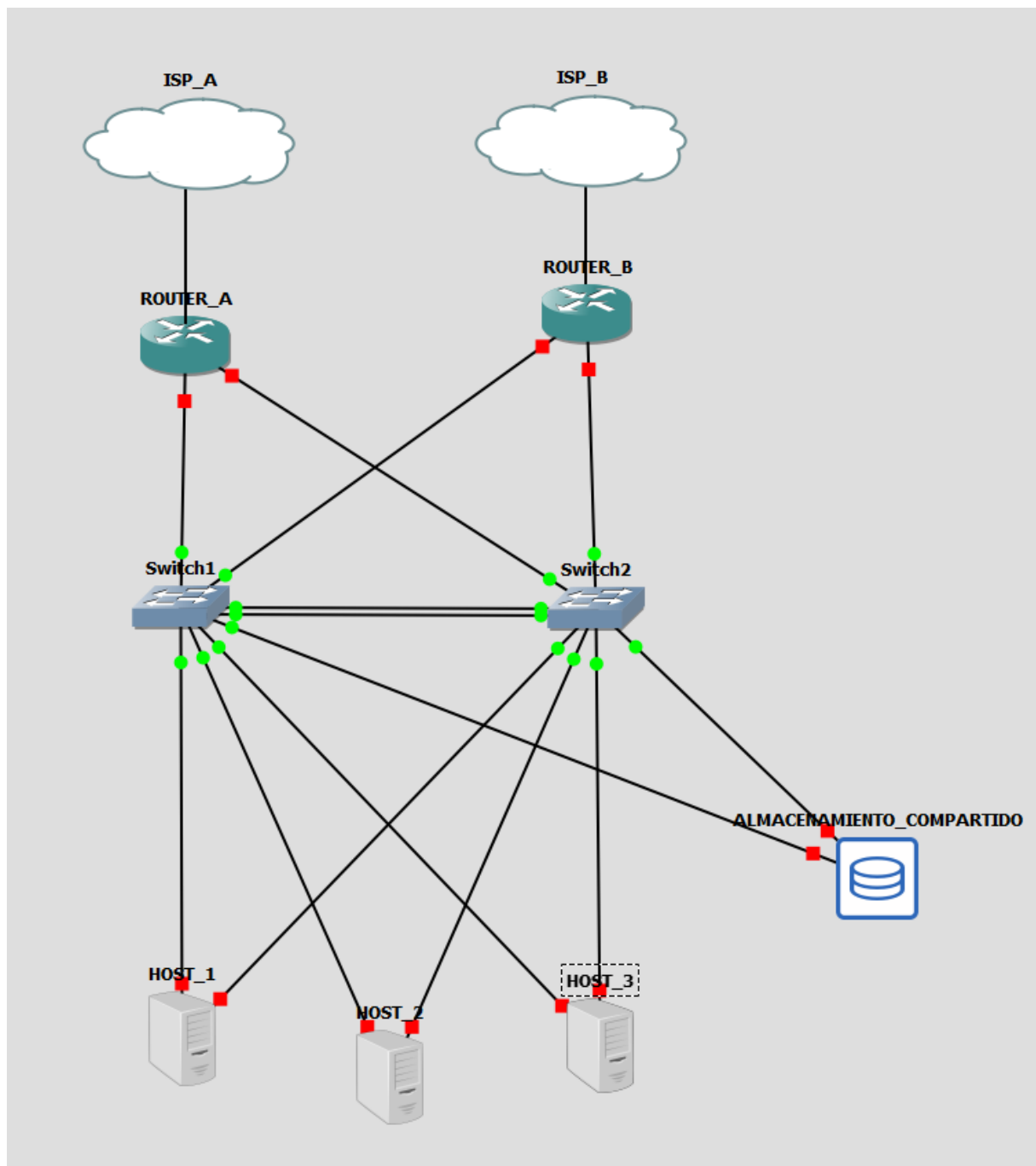


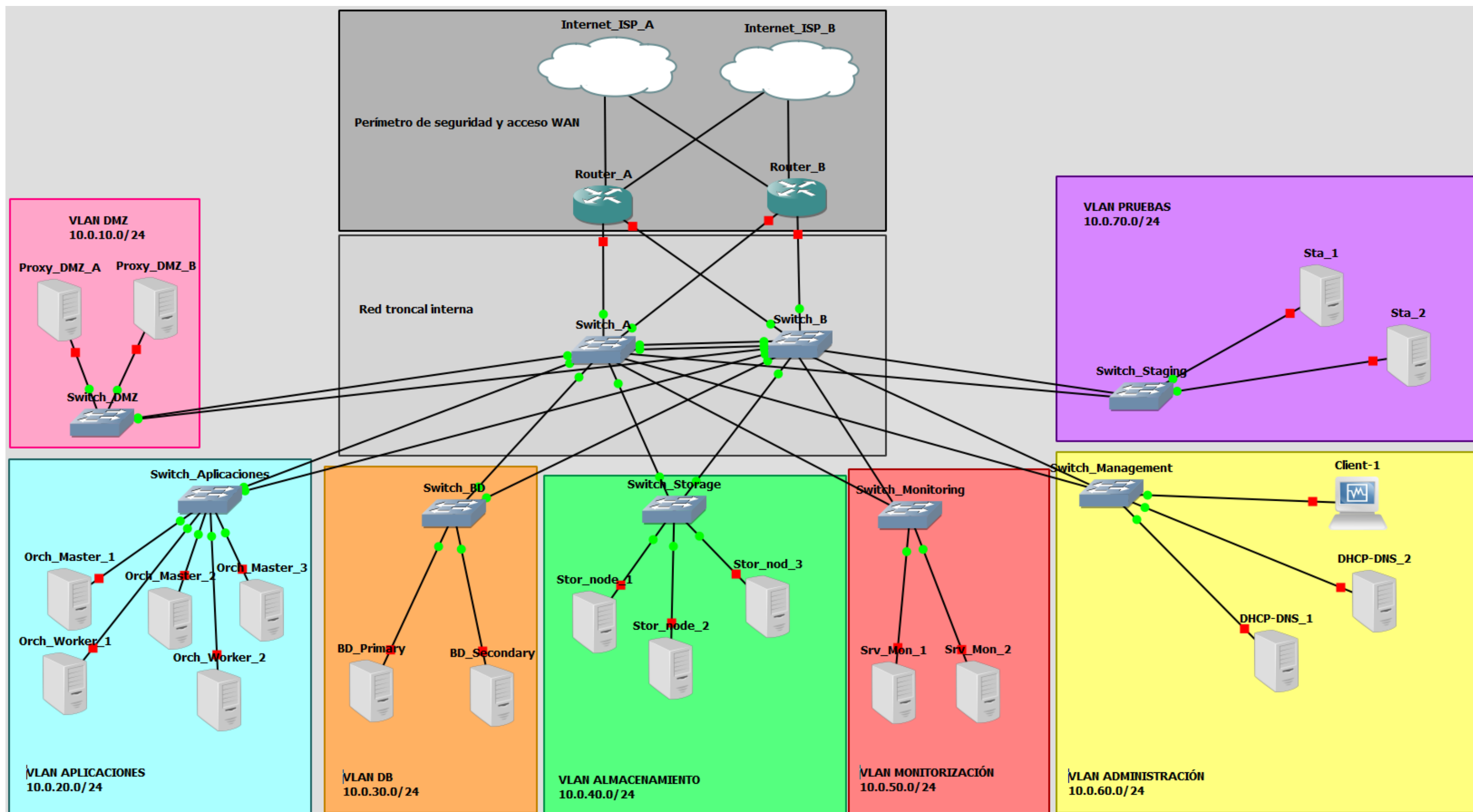
Diagrama de la red física; la alta disponibilidad se consigue a partir de dos proveedores de red y dos routers, así como dos switches conectados a su vez entre sí y un clúster con tres servidores compartiendo almacenamiento con redundancia de discos

Esta capa física proporciona la resiliencia fundamental sobre la que se construye el entorno virtualizado.

4.2.2. Arquitectura de Red Lógica Virtualizada sobre Proxmox VE

Dentro del entorno Proxmox VE, que estará instalado en el clúster de servidores físicos del diagrama anterior, se implementará la arquitectura de red lógica detallada a continuación. Esta arquitectura utiliza bridges de red virtual en Proxmox VE para la creación de Redes Virtuales de Área Local (VLANs) segmentadas, y una máquina virtual dedicada ejecutando OPNsense actuará como el firewall/router central, gestionando la seguridad perimetral y el enrutamiento entre todas las VLANs internas.

A continuación, se presenta el diagrama que representa gráficamente esta topología de red lógica virtualizada, mostrando la interconexión de las VLANs a través del firewall OPNsense virtualizado y la ubicación de las máquinas virtuales de servicio principales:



El anterior diagrama ilustra la topología de red lógica y la distribución de los servicios virtualizados que componen la infraestructura PaaS para OpenPaDi, tal como se implementará sobre la plataforma de virtualización Proxmox VE. Este diseño prioriza la segmentación, la seguridad y la alta disponibilidad de los servicios.

Conectividad Externa y Perímetro de Seguridad Virtualizado

- Internet (Internet_ISP_A e Internet_ISP_B): Representan las conexiones redundantes a la red externa. En el entorno Proxmox, esto se logra mediante interfaces de red del host Proxmox configuradas de modo que permita que la máquina virtual del firewall acceda a Internet.
- Firewalls/Routers Perimetrales Virtualizados (Router_A y Router_B): Estos dos routers simbolizan las máquinas virtuales (VMs) que ejecutarán el software de firewall/router avanzado (OPNsense).
 - Función Principal: Actúan como el borde de la red virtualizada, gestionando todo el tráfico entre Internet y las VLANs internas. Son responsables de la seguridad perimetral (filtrado de paquetes, NAT, prevención de intrusiones...), así como el enrutamiento entre VLANs.
 - Alta Disponibilidad: La presencia de dos instancias (Router_A y Router_B) representa un diseño para alta disponibilidad. Cada una se conectaría a una fuente de Internet y a ambos switches de núcleo virtuales para resiliencia.

Núcleo de Red Virtualizado / Distribución (Zona Gris Claro Intermedia):

- Switches de Núcleo Virtuales (Switch_A y Switch_B): Estos representan la funcionalidad de conmutación y agregación proporcionada por los bridges de Linux dentro de Proxmox VE. No son VMs de switches separadas, sino que representan la infraestructura de red del hipervisor que interconecta las diferentes VLANs lógicas hacia las VMs firewall.

VLANs de Acceso y Servicios Virtualizados:

Cada una de estas zonas representa una Red Virtual de Área Local (VLAN) lógica, implementada en Proxmox VE.

- **VLAN DMZ (10.0.10.0/24):**
 - El Switch_DMZ representa el bridge de Proxmox para esta VLAN.
 - Aloja las VMs Proxy_DMZ_A y Proxy_DMZ_B, que ejecutarán el Ingress Controller (Traefik Proxy) en configuración redundante. Reciben tráfico de los firewalls (Router_A/Router_B) y lo dirigen a la VLAN de Aplicaciones.
- **VLAN Aplicaciones (10.0.20.0/24):**
 - El Switch_Aplicaciones representa el bridge de Proxmox para esta VLAN.
 - Aloja las VMs que constituyen los nodos del clúster K3s. Los contenedores de OpenPaDi (Frontend, API, Keycloak) se ejecutarán dentro de este clúster.
- **VLAN DB (10.0.30.0/24):**
 - El Switch_BD representa el bridge de Proxmox para esta VLAN.
 - Aloja las VMs BD_Primary y BD_Secondary para el servicio de base de datos PostgreSQL con replicación para HA.
- **VLAN Almacenamiento (10.0.40.0/24):**
 - El Switch_Storage es el bridge de Proxmox para esta VLAN.
 - Aloja las VMs Stor_node_1, 2 y 3 para el clúster de almacenamiento de objetos MinIO.
- **VLAN Monitorización (10.0.50.0/24):**
 - El Switch_Monitoring es el bridge de Proxmox para esta VLAN.
 - Aloja las VMs Srv_Mon_1 y Srv_Mon_2 para la pila de monitorización (Prometheus, Grafana, Loki) en HA.
- **VLAN Administración (10.0.60.0/24):**
 - El Switch_Management es el bridge de Proxmox para esta VLAN.
 - Aloja la VM Client-1 (Estación de Administración) y las VMs DHCP-DNS_1 y DHCP-DNS_2 para los servicios de red DHCP y DNS internos en HA¹.
- **VLAN Pruebas (10.0.70.0/24):**

¹ No obstante, el propio OPNsense ofrece servicios DHCP y DNS que, como se verá más adelante y por motivos de centralización y simplificación de estos servicios, serán configurados en dicha VM.

- El Switch_Staging es el bridge de Proxmox para esta VLAN.
- Aloja las VMs Sta_1 y Sta_2 para un entorno de pruebas aislado.

Flujo de Tráfico y Comunicación entre las diferentes VLANs:

Todo el tráfico que necesite pasar entre diferentes VLANs será enrutado a través de la VM OPNsense (Router_A/Router_B en el diagrama). Las VMs dentro de la misma VLAN se comunican directamente a través de su bridge Proxmox correspondiente. El firewall aplicará las políticas de seguridad para permitir o denegar estos flujos inter-VLAN.

Consideraciones sobre la Alta Disponibilidad en el Entorno Virtualizado:

La alta disponibilidad se busca en múltiples niveles:

- **Redundancia de Conexión de Switches de Acceso:** Cada "switch de acceso" (bridge Proxmox de VLAN) tiene uplinks conceptuales a ambos "switches de núcleo" (la infraestructura de red de Proxmox que conecta a los firewalls), lo que se traduce en cómo se configuran los bridges y las interfaces del firewall virtualizado.
- **Servicios en HA dentro de las VMs:** Duplicación de VMs para proxies, monitorización, DHCP/DNS; clústeres para K3s y MinIO; replicación para PostgreSQL.
- **HA de la Plataforma de Virtualización:** En un entorno de producción real, los servidores físicos que alojan Proxmox están en clúster con almacenamiento compartido, permitiendo la migración o reinicio automático de VMs en caso de fallo de un host físico. Para este proyecto, y debido a las limitaciones técnicas y económicas, esta capa de HA física se simula o se asume, y la HA se centra en los servicios dentro del entorno virtualizado.

4.2.1. Comunicaciones externas

La plataforma requiere conectividad a redes externas, principalmente Internet, para:

- Permitir el acceso de los usuarios finales a la aplicación OpenPaDi.
- Facilitar la descarga de software, imágenes de contenedores y actualizaciones para los sistemas y servicios de la PaaS.

Soluciones para la Conexión:

En el entorno de virtualización Proxmox VE, la conexión de las máquinas virtuales (incluida la VM OPNsense) a la red externa se gestiona a través de las interfaces de red físicas del servidor host Proxmox.

Para este proyecto, se ha escogido la implementación de la conexión WAN de la VM mencionada (OPNsense) en modo puente a través de una interfaz de red física del host Proxmox. Los motivos de esta elección son:

- **Control y Seguridad:** Este enfoque otorga a la VM OPNsense el control total sobre la conexión a Internet, permitiendo aplicar todas sus funcionalidades de firewall, NAT, y gestión de tráfico directamente sobre el enlace WAN, como si fuera un dispositivo físico perimetral.
- **Aislamiento:** Separa la gestión de la red externa de la propia configuración de red del host Proxmox, lo que es una práctica más segura y modular.
- **Flexibilidad:** Permite una configuración de red más realista y compleja, incluyendo la posibilidad de manejar IPs públicas directamente en el firewall virtualizado si el entorno lo permitiera (aunque en el presente proyecto se usará una IP privada con NAT en el router doméstico).

Para la alta disponibilidad de la conexión a Internet, como se representó en el diagrama conceptual de producción, se requerirían dos interfaces WAN en el clúster de firewalls virtuales, cada una conectada (a través de bridges Proxmox y NICs físicas distintas del host) a proveedores de Internet diferentes. En la implementación simplificada de este proyecto se configurará una única interfaz WAN, pero el diseño de la PaaS considera la capacidad de expansión hacia dicha redundancia WAN.

4.2.2. Comunicaciones Internas y Segmentación de Red Lógica

Para implementar la solución de forma segura y organizada dentro del entorno Proxmox VE, se establece una arquitectura de comunicaciones internas basada en la segmentación lógica mediante Redes Virtuales de Área Local (VLANs). Esta estrategia, ya descrita en la descripción general del diseño de red (apartado 4.2), es crucial para aislar el tráfico, aplicar políticas de seguridad específicas y gestionar eficientemente los flujos de datos entre los distintos componentes de la plataforma.

Como se dijo anteriormente, se ha optado por implementar esta segmentación utilizando bridges de red virtual dedicados en Proxmox VE para cada VLAN lógica, con la máquina virtual firewall (OPNsense) actuando como el enrutador inter-VLAN y punto de control de seguridad para todas las comunicaciones entre estos segmentos. Entre los motivos principales podemos enumerar:

- **Aislamiento y Seguridad:** Proporciona un fuerte aislamiento entre las diferentes zonas funcionales (DMZ, Aplicaciones, Bases de Datos, Almacenamiento, Gestión, Monitorización, Pruebas), como se detalla en los requisitos (2.1) y en el diseño general de red (4.2).
- **Control detallado:** Permite definir políticas de firewall específicas en la VM OPNsense para cada flujo de comunicación inter-VLAN.
- **Flexibilidad y Escalabilidad:** Facilita la adición o modificación de segmentos de red y servicios de forma modular sin impactar en otras zonas.
- **Gestión Simplificada:** Aunque se crean múltiples segmentos, la gestión del enrutamiento y la seguridad se centraliza en la VM OPNsense, lo que simplifica la administración general en un entorno virtualizado.

4.2.3. Integración de Redes y Dispositivos Virtualizados

La integración de las diferentes redes virtuales (VLANs) y los dispositivos virtualizados (máquinas virtuales) que componen la infraestructura PaaS de OpenPaDi se realiza dentro del entorno Proxmox VE, siguiendo la topología de red lógica presentada en la Figura del apartado 4.2.2.

Los aspectos clave de esta integración son:

- **Interconexión de VLANs:** Todas las VLANs lógicas (DMZ, Aplicaciones, Bases de Datos, Almacenamiento, Gestión, Monitorización, Pruebas), implementadas como bridges de red virtual en Proxmox VE, se interconectan a través de la máquina virtual firewall/router (OPNsense). Esta VM dispone de interfaces de red virtuales conectadas a cada uno de dichos bridges, actuando como el gateway y punto de enrutamiento central para todo el tráfico inter-VLAN.
- **Conexión de Dispositivos (VMs) a sus VLANs:** Cada máquina virtual de servicio (proxies, nodos K3s, servidores de bases de datos, etc.) se asigna a

su VLAN correspondiente mediante la conexión de su interfaz de red virtual al bridge de Proxmox VE designado para esa VLAN.

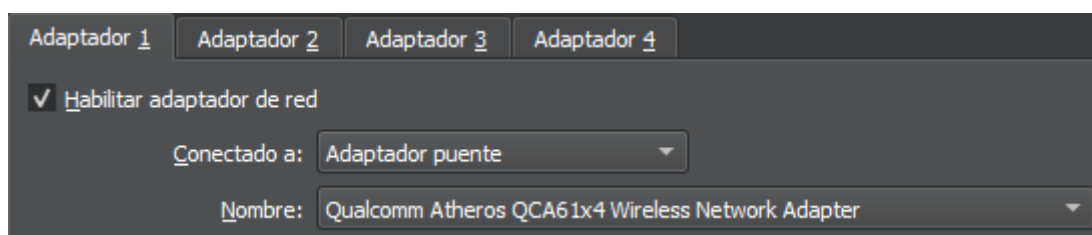
- **Políticas de Seguridad:** La comunicación permitida entre dispositivos de diferentes VLANs está estrictamente definida por las reglas de firewall configuradas en la VM OPNsense, como se detallará en el apartado correspondiente al diseño del firewall.

Esta arquitectura asegura que todos los componentes estén correctamente segmentados, pero puedan a su vez comunicarse de forma controlada y segura según las necesidades de la plataforma, tal como se visualiza en el diagrama de red lógica.

4.2.4. Simulación de la red en Proxmox

Para materializar la topología de red lógica descrita anteriormente, se ha procedido a su simulación y configuración dentro de un entorno Proxmox VE. Esta implementación práctica permite validar el diseño de segmentación y observar el comportamiento de los flujos de red entre los distintos componentes de la PaaS OpenPaDi.

Para llevar a cabo esta simulación, se ha configurado una instancia de Proxmox VE (en este caso, virtualizada en VirtualBox). La conectividad externa (WAN) para el firewall OPNsense se gestiona a través de un adaptador de red del host Proxmox configurado en modo puente. Las redes internas (VLANs) se implementan mediante Linux Bridges (vbrX) dentro de Proxmox, actuando como conmutadores virtuales para cada segmento de red.



Adaptador 1
Adaptador 2
Adaptador 3
Adaptador 4

☒ Habilitar adaptador de red

Conectado a: Red interna

Nombre: vlan-trunk-pve

Adaptadores de red de la VM de Proxmox en esta simulación

Configuración de Redes (Linux Bridges) en Proxmox VE:

En Proxmox, cada VLAN lógica de nuestro diseño se implementa como un Linux Bridge dedicado (nombrados vmbr10 a vmbr70 en este caso). Estos bridges actúan como conmutadores virtuales a los que se conectan las interfaces de red de las máquinas virtuales, aislando el tráfico de cada segmento. Cada uno de ellos sustituiría a los switches de acceso representados en el diagrama de red mencionado. La siguiente captura muestra la configuración de estos bridges en la interfaz de Proxmox:

Crear Revertir Editar Eliminar Aplicar configuración									
Nombre ↑	Tipo	Activo	Inicio a...	Consci...	Puertos/Es...	Modo de B...	CIDR	Puerta de enlace	Comentario
enp0s3	Dispositivo de ...	Sí	No	No					
enp0s8	Dispositivo de ...	Sí	No	No					
vmbr0	Linux Bridge	Sí	Sí	No	enp0s3		192.168.1.138/24	192.168.1.1	
vmbr1	Linux Bridge	Sí	Sí	Sí	enp0s8				
vmbr10	Linux Bridge	Sí	Sí	No					VLAN DMZ 10.0.10.0/24
vmbr20	Linux Bridge	Sí	Sí	No					VLAN APLICACIONES 10.0.20.0/24
vmbr30	Linux Bridge	Sí	Sí	No					VLAN DB 10.0.30.0/24
vmbr40	Linux Bridge	Sí	Sí	No					VLAN ALMACENAMIENTO 10.0.40.0/24
vmbr50	Linux Bridge	Sí	Sí	No					VLAN MONITORIZACIÓN 10.0.50.0/24
vmbr60	Linux Bridge	Sí	Sí	No					VLAN ADMINISTRACIÓN 10.0.60.0/24
vmbr70	Linux Bridge	Sí	Sí	No					VLAN PRUEBAS 10.0.70.0/24

Configuración de los vmbr en Proxmox; cada uno de ellos sería la materialización de los switches de acceso del diagrama de red lógica del apartado 4.2.2.

- **vmbr0**: Conectado a la red física (enp0s3), proporciona el acceso WAN para la VM del firewall (OPNsense).

Editar: Linux Bridge

Nombre:	vmbr0	Inicio automático:	☒
IPv4/CIDR:	192.168.1.138/24	Consciente de VLAN:	☐
Puerta de enlace (IPv4):	192.168.1.1	Puertos de puente:	enp0s3
IPv6/CIDR:		Comentario:	
Puerta de enlace (IPv6):			

Avanzado ☐
Aceptar

Configuración de vmbr0; tiene como puerto de enlace enp0s3, que corresponde al adaptador 1 de la VM de Proxmox, conectado en modo puente

- **vmbr1:** Conectado a la interfaz física enp0s8, que representa la "red interna" principal del host Proxmox. En esta arquitectura, enp0s8 actúa como el "enlace troncal" físico subyacente. El bridge vmbr1 sirve como el conducto principal para el tráfico de gestión del host Proxmox y es la base sobre la cual (o a través de la cual) se gestiona el tráfico de las VLANs internas (vmbr10 a vmbr70) que son segmentadas y enrutadas por la VM OPNsense.

Editar: Linux Bridge

Nombre:	vmbr1	Inicio automático:	☒
IPv4/CIDR:		Consciente de VLAN:	☒
Puerta de enlace (IPv4):		Puertos de puente:	enp0s8
IPv6/CIDR:		Comentario:	
Puerta de enlace (IPv6):			

Avanzado ☐
Aceptar

Configuración de vmbr1; tiene como puerto de enlace enp0s8, que corresponde al adaptador 2 de la VM de Proxmox, conectado en red interna

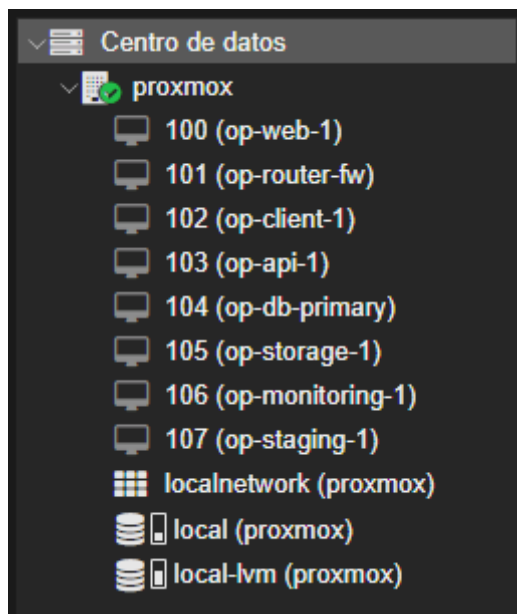
Para el enfoque que vamos a dar a esta simulación, este bridge no tendrá utilidad, pero podría utilizarse si se utilizase el etiquetado de VLANS estando todas ellas conectadas a este vmbr.

- **vmbr10** (VLAN DMZ - 10.0.10.0/24): Segmento para los proxies inversos (Ingress Controller).
- **vmbr20** (VLAN APLICACIONES - 10.0.20.0/24): Segmento para los nodos del clúster K3s que alojarán los servicios de la aplicación.
- **vmbr30** (VLAN DB - 10.0.30.0/24): Segmento para los servidores de base de datos.
- **vmbr40** (VLAN ALMACENAMIENTO - 10.0.40.0/24): Segmento para los servidores de almacenamiento de objetos.
- **vmbr50** (VLAN MONITORIZACIÓN - 10.0.50.0/24): Segmento para los servicios de monitorización y logging.
- **vmbr60** (VLAN ADMINISTRACIÓN - 10.0.60.0/24): Segmento para la VM de cliente/administración y servicios de red internos (DHCP/DNS si fueran necesarios).
- **vmbr70** (VLAN PRUEBAS - 10.0.70.0/24): Segmento aislado para el entorno de staging/pruebas.

En el siguiente apartado se mostrará como cada uno de los vmbr mencionados se conecta con la VM de OPNsense.

Máquinas Virtuales Desplegadas para la Simulación:

Sobre esta infraestructura de red virtualizada, se han desplegado las siguientes máquinas virtuales (VMs), cada una cumpliendo un rol específico dentro de la PaaS OpenPaDi y conectada al bridge (VLAN) correspondiente. La siguiente captura muestra el listado de VMs en Proxmox:



- **101 (op-router-fw):** Ejecuta OPNsense. Actúa como firewall perimetral y router inter-VLAN. Conectada a vmbr0 (WAN) y a todos los bridges de VLAN internas (vmbr10 a vmbr70). Para alta disponibilidad, en producción se configuraría un clúster HA de OPNsense. Se encarga de gestionar el tráfico entre VLANs y las reglas de firewall.



Para ello, por supuesto, tiene que tener los correspondientes adaptadores de red conectados a todos los *vmbr* anteriormente mencionados:

Maquina virtual 101 (op-router-fw) en el nodo proxmox		Ninguna etiqueta
Resumen	Agregar Eliminar Editar Acción de disco Revertir	
Consola	Memoria	2.00 GiB
Hardware	Procesadores	2 (1 sockets, 2 cores) [x86-64-v2-AES]
Cloud-Init	BIOS	SeaBIOS
Opciones	Pantalla	Por defecto
Historial de tareas	Maquina	q35
Monitor	Controlador SCSI	VirtIO SCSI single
Respaldo	Dispositivo CD/DVD (scsi0)	none,media=cdrrom
Replicación	Disco duro (virtio0)	local-lvm:vm-101-disk-1,iosthread=1,size=20G
Snapshots	Dispositivo de red (net0)	virtio=BC:24:11:D5:DA:AE,bridge=vmbr0
Cortafuego	Dispositivo de red (net1)	virtio=BC:24:11:67:25:D8,bridge=vmbr10
Permisos	Dispositivo de red (net2)	virtio=BC:24:11:E9:E3:01,bridge=vmbr20
	Dispositivo de red (net3)	virtio=BC:24:11:8F:6A:2B,bridge=vmbr30
	Dispositivo de red (net4)	virtio=BC:24:11:BF:44:9D,bridge=vmbr40
	Dispositivo de red (net5)	virtio=BC:24:11:94:8E:FA,bridge=vmbr50
	Dispositivo de red (net6)	virtio=BC:24:11:84:2D:F9,bridge=vmbr60
	Dispositivo de red (net7)	virtio=BC:24:11:35:E9:CE,bridge=vmbr70
	Disco sin usar 0	local-lvm:vm-101-disk-0

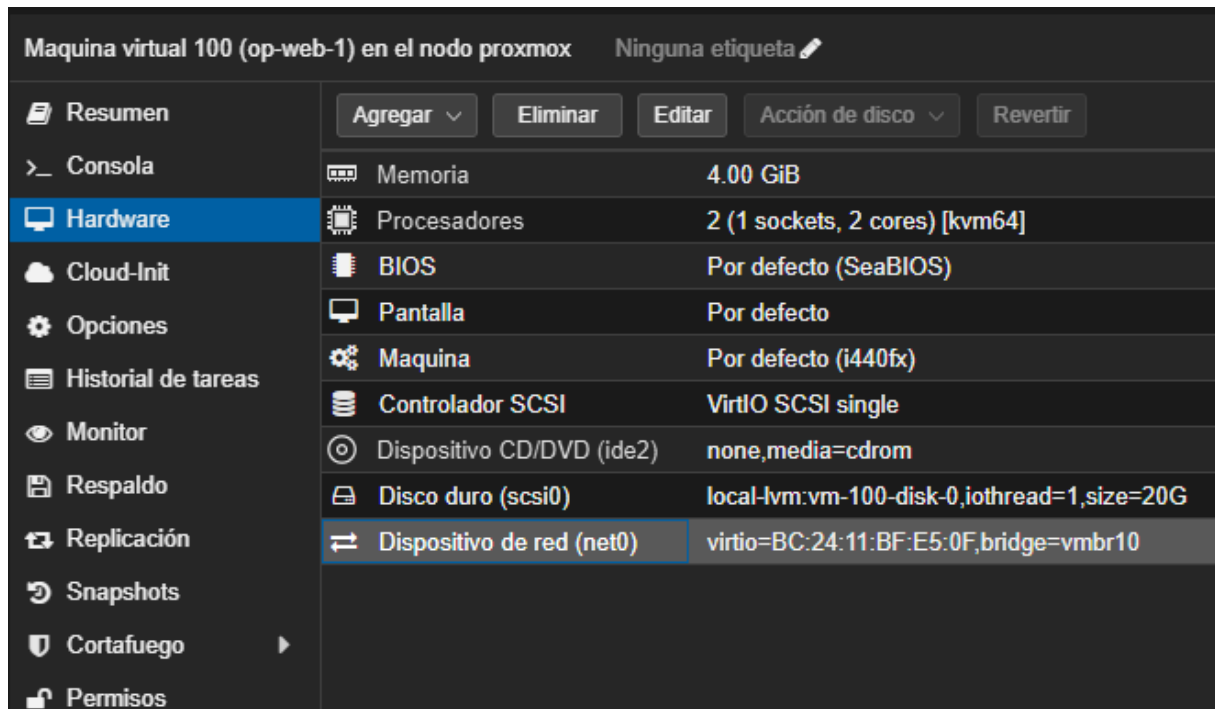
- **100 (op-web-1):** Representa el nodo Maestro del clúster K3s. Este nodo es crucial para la orquestación de contenedores.

Servicios contenerizados alojados (gestionados por K3s en este nodo):

- Traefik Ingress Controller: Gestiona el tráfico entrante HTTPS, realiza la terminación TLS (con certificados de Cert-Manager) y enruta a los servicios. Su exposición externa se realiza a través de op-router-fw desde la VLAN DMZ (vmbr10).
- Cert-Manager: Gestiona los certificados TLS para los servicios.
- Keycloak: Servicio de autenticación centralizada (expuesto como auth.openpadi.local), utilizando PostgreSQL externo para persistencia.
- Frontend Svelte de OpenPaDi: Interfaz de usuario de la aplicación (expuesta como openpadi.local).
- Conexión de Red de la VM: Primariamente a vmbr20 (VLAN Aplicaciones) para la comunicación del plano de control de K3s y el

acceso a servicios internos. La exposición de Traefik se maneja a través del enrutamiento y NAT de OPNsense hacia la IP del servicio Traefik en K3s. Para alta disponibilidad del máster K3s, se requerirían múltiples nodos maestros.

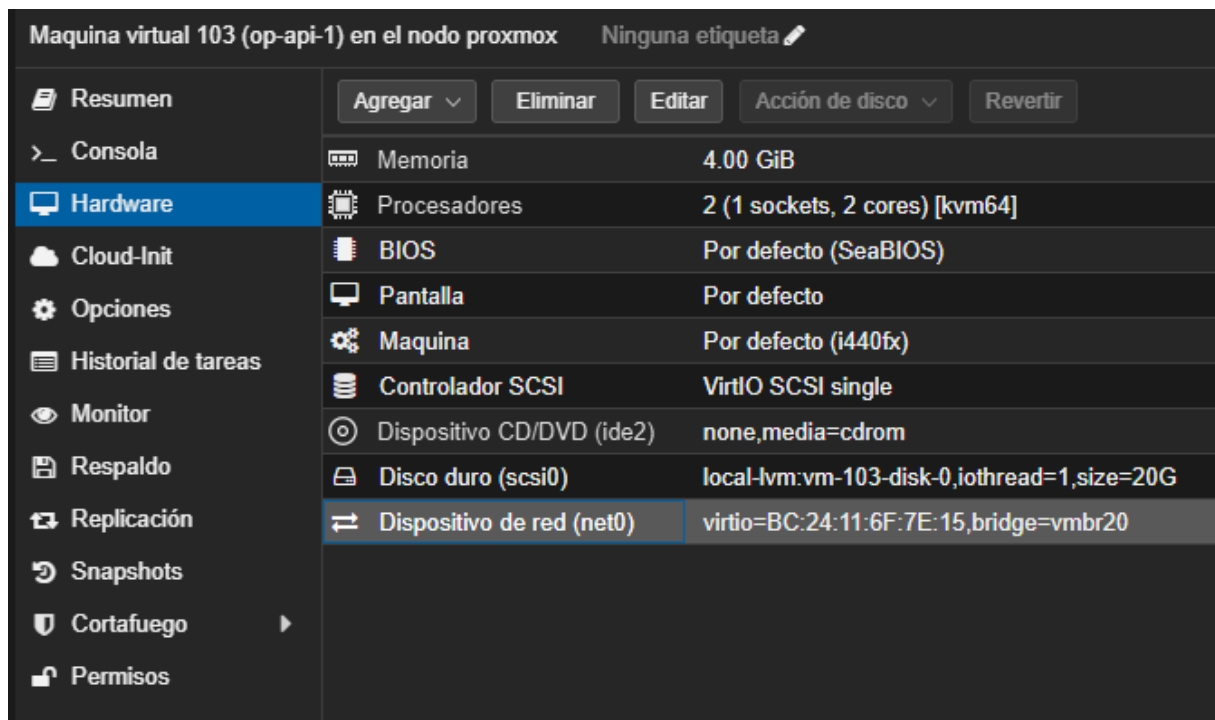
Su adaptador de red debe estar conectado al vmbr10:



- **103 (op-api-1):** Representa un nodo Worker del clúster K3s.

Servicios contenerizados alojados (gestionados por K3s en este nodo):

- API FastAPI de OpenPaDi: Backend de la aplicación, valida tokens JWT de Keycloak y se comunica con la base de datos y el almacenamiento de objetos.
- Conexión de Red de la VM: A vmbr20 (VLAN Aplicaciones). Para alta disponibilidad y escalabilidad de la API, se desplegarían múltiples réplicas del pod de la API distribuidas entre varios nodos worker.



- **104 (op-db-primary):** Actúa como el servidor de base de datos PostgreSQL primario.
 - Aloja las bases de datos opadi_db (para la aplicación) y keycloak_db (para Keycloak).
 - Conexión de Red de la VM: A vmbr30 (VLAN DB). Para alta disponibilidad, se implementaría replicación (streaming replication) a un servidor PostgreSQL secundario (representado conceptualmente por BD_Secondary en el diagrama de red y que podría ser otra VM como op-db-secondary en esta misma VLAN).

Maquina virtual 104 (op-db-primary) en el nodo proxmox

Ninguna etiqueta

Resumen

>_ Consola

Hardware

Cloud-Init

Opciones

Historial de tareas

Monitor

Respaldo

Replicación

Snapshots

Cortafuego

Permisos

Agregar

Eliminar

Editar

Acción de disco

Revertir

Memoria

Procesadores

BIOS

Pantalla

Maquina

Controlador SCSI

Dispositivo CD/DVD (ide2)

Disco duro (scsi0)

Dispositivo de red (net0)

4.00 GiB

2 (1 sockets, 2 cores) [kvm64]

Por defecto (SeaBIOS)

Por defecto

Por defecto (i440fx)

VirtIO SCSI single

none,media=cdrom

local-lvm:vm-104-disk-0,ioread=1,size=20G

virtio=BC:24:11:5D:5E:8F,bridge=vmbr30

- 105 (op-storage-1):** Simula un nodo del clúster de almacenamiento de objetos MinIO.
 - Utilizado para guardar documentos históricos digitalizados y otros archivos de la aplicación. El bucket principal es openpadi-documentos.
 - Conexión de Red de la VM: A vmbr40 (VLAN Almacenamiento). Para asegurar la durabilidad y disponibilidad de los datos, MinIO se configuraría en un modo distribuido (erasure coding) sobre múltiples nodos y discos en un entorno de producción (representados por Stor_node_1, 2, 3 en el diagrama).

Maquina virtual 105 (op-storage-1) en el nodo proxmox

Ninguna etiqueta

Resumen

Consola

Hardware

Cloud-Init

Opciones

Historial de tareas

Monitor

Respaldo

Replicación

Snapshots

Cortafuego

Agregar

Eliminar

Editar

Acción de disco

Revertir

Memoria

4.00 GiB

Procesadores

2 (1 sockets, 2 cores) [kvm64]

BIOS

Por defecto (SeaBIOS)

Pantalla

Por defecto

Maquina

Por defecto (i440fx)

Controlador SCSI

VirtIO SCSI single

Dispositivo CD/DVD (ide2)

none,media=cdrom

Disco duro (scsi0)

local-lvm:vm-105-disk-0,iothread=1,size=20G

Dispositivo de red (net0)

virtio=BC:24:11:E9:47:D6,bridge=vmbr40

- **106 (op-monitoring-1):** Servidor para la pila de monitorización (ej. Prometheus, Grafana, Loki). Conectada a vmbr50 (VLAN Monitorización). En producción, los componentes de monitorización también se configurarían para alta disponibilidad.

Maquina virtual 106 (op-monitoring-1) en el nodo proxmox

Ninguna etiqueta

Resumen

>_ Consola

Hardware

Cloud-Init

Opciones

Historial de tareas

Monitor

Respaldo

Replicación

Snapshots

Cortafuego

Permisos

Agregar

Eliminar

Editar

Acción de disco

Revertir

Memoria

4.00 GiB

Procesadores

2 (1 sockets, 2 cores) [kvm64]

BIOS

Por defecto (SeaBIOS)

Pantalla

Por defecto

Maquina

Por defecto (i440fx)

Controlador SCSI

VirtIO SCSI single

Dispositivo CD/DVD (ide2)

none,media=cdrom

Disco duro (scsi0)

local-lvm:vm-106-disk-0,iothread=1,size=20G

Dispositivo de red (net0)

virtio=BC:24:11:77:8D:AA,bridge=vmbr50

- **102 (op-client-1):** Estación de trabajo para administración y pruebas. Conectada a vmbr60 (VLAN Administración).

Maquina virtual 102 (op-client-1) en el nodo proxmox Ninguna etiqueta

Resumen
Consola
Hardware
Cloud-Init
Opciones
Historial de tareas
Monitor
Respaldo
Replicación
Snapshots
Cortafuego
Permisos

Agregar
Eliminar
Editar
Acción de disco
Revertir

Memoria	2.00 GiB
Procesadores	1 (1 sockets, 1 cores) [x86-64-v2-AES]
BIOS	Por defecto (SeaBIOS)
Pantalla	Por defecto
Maquina	Por defecto (i440fx)
Controlador SCSI	VirtIO SCSI single
Dispositivo CD/DVD (ide2)	none,media=cdrrom
Disco duro (scsi0)	local-lvm:vm-102-disk-0,iothread=1,size=10G
Dispositivo de red (net0)	virtio=BC:24:11:27:C4:91,bridge=vmbr60

Nos permitirá configurar la máquina 101 (*op-router-fw*) a través de su interfaz gráfica conectándonos con su IP en el navegador:

Maquina virtual 102 (op-client-1) en el nodo proxmox Ninguna etiqueta

Resumen
Consola
Hardware
Cloud-Init
Opciones
Historial de tareas
Monitor
Respaldo
Replicación
Snapshots
Cortafuego
Permisos

Iniciar
Apagar
Consola

Overview | Interfaces | OPNsense.localdomain — Mozilla Firefox

<https://10.0.60.1/ui/interfaces/overview>

root@OPNsense.localdomain

[APLICACIONES]
[DB]
[DMZ]
[LAN]
[MONITORIZACION]
[PRUEBAS]
[WAN]
Assignments
Overview
Settings
Neighbors
Virtual IPs
Wireless
Point-to-Point
Other Types
Diagnostics
Firewall
VPN
Services

Interfaces: Overview

Status	Interface	Device	Link Type	IPv4	IPv6	Gateway	Routes	Commands
✓	WAN (wan)	vtnet0	dhcp	192.168.1.137/24	2a0c:5a83:8406:b600:be24:11ff:fed5:dade/64 fe80::be24:11ff:fed5:dade/64	192.168.1.1 fe80::1	default 100.90.1.1	Expand
✓	DMZ (opt1)	vtnet1	static	10.0.10.1/24			10.0.10.0/24	Q
✓	APLICACIONES (opt2)	vtnet2	static	10.0.20.1/24			10.0.20.0/24	Q
✓	DB (opt3)	vtnet3	static	10.0.30.1/24			10.0.30.0/24	Q
✓	ALMACENAMIENTO (opt4)	vtnet4	static	10.0.40.1/24			10.0.40.0/24	Q
✓	MONITORIZACION (opt5)	vtnet5	static	10.0.50.1/24			10.0.50.0/24	Q
✓	LAN (lan)	vtnet6	static	10.0.60.1/24			10.0.60.0/24	Q

Showing 1 to 7 of 11 entries

- **107 (op-staging-1):** VM para un entorno de pruebas aislado. Conectada a vmbr70 (VLAN Pruebas).

60

Maquina virtual 107 (op-staging-1) en el nodo proxmox		Ninguna etiqueta 
Resumen	Agregar ▾ Eliminar Editar Acción de disco ▾ Revertir	
Consola	Memoria	4.00 GiB
Hardware	Procesadores	2 (1 sockets, 2 cores) [kvm64]
Cloud-Init	BIOS	Por defecto (SeaBIOS)
Opciones	Pantalla	Por defecto
Historial de tareas	Maquina	Por defecto (i440fx)
Monitor	Controlador SCSI	VirtIO SCSI single
Respaldo	Dispositivo CD/DVD (ide2)	none,media=cdrom
Replicación	Disco duro (scsi0)	local-lvm:vm-107-disk-0,ioread=1,size=20G
Snapshots	Dispositivo de red (net0)	virtio=BC:24:11:32:03:62,bridge=vmbr70
Cortafuego		
Permisos		

Funcionamiento General de la Red Segmentada:

La pieza central de la comunicación y seguridad de la red interna es la máquina virtual `op-router-fw`, que ejecuta el software de firewall OPNsense. Esta VM está conectada a todos los segmentos de red lógicos (VLANs) a través de los diferentes Linux Bridges (`vmbr10` a `vmbr70`).

Ninguna máquina virtual en una VLAN puede comunicarse directamente con una máquina virtual en otra VLAN; todo este tráfico inter-VLAN debe pasar obligatoriamente a través de OPNsense. Es en OPNsense donde se definen las reglas de firewall que permiten o deniegan explícitamente estas comunicaciones entre segmentos. Por ejemplo, la API en la VLAN de Aplicaciones podrá acceder a la base de datos en la VLAN DB solo si OPNsense tiene una regla que lo permita.

El tráfico dentro de una misma VLAN (por ejemplo, entre dos nodos *worker* de K3s en la VLAN de Aplicaciones) se conmuta localmente por el bridge correspondiente (`vmbr20` en este caso) sin pasar por OPNsense. El acceso desde Internet a los servicios de OpenPaDi también es gestionado por OPNsense, que recibe el tráfico en su interfaz WAN y lo reenvía (mediante NAT y reglas de firewall) al componente adecuado, típicamente el Ingress Controller en la DMZ.

Esta arquitectura garantiza un alto nivel de aislamiento, control y seguridad sobre los flujos de datos en toda la infraestructura.

4.3. Servicios de Red

A continuación se enumeran y describen los servicios de red utilizados para comprobar la Infraestructura de la PaaS de OpenPaDi.

Debido a las limitaciones técnicas a nivel de hardware, que hicieron extremadamente difíciles la reproducción del escenario completo con las máquinas y la red creada en Proxmox VE que se explicó en el apartado anterior, se creó un escenario en VirtualBox, donde se crearon las siguientes máquinas:



En este escenario se priorizó la correcta integración de los servicios de la infraestructura sin tener en cuenta la segmentación de red que será llevada a cabo en Proxmox VE, por lo que las máquinas fueron unidas en una misma red con el fin de facilitar la realización de las pruebas (a excepción de los servicios DNS, DHCP y de firewall, que, debido a su naturaleza, fue obligatorio realizar en el escenario de Proxmox). Las IPs de estas máquinas, aunque iguales en la parte de host, están dentro de la red 192.168.1.0/24, ya que la red 10.0.0.0/24 la coge automáticamente el adaptador de red `enp0s3` al estar en modo NAT.

A continuación, se detalla la función principal y la dirección IP asignada a cada una de estas máquinas virtuales dentro de este entorno de pruebas:

- **OP-API-1 (192.168.1.12):** Actúa como nodo worker del clúster K3s. Ejecuta el *backend* de la aplicación OpenPaDi (API FastAPI), que maneja la lógica de negocio y la interacción con los datos.

- **OP-storage-1 (192.168.1.14)**: Aloja el servidor de almacenamiento de objetos MinIO. Se encarga de guardar y servir los documentos históricos digitalizados y otros archivos de la aplicación.
- **OP-db-primary (192.168.1.13)**: Contiene el servidor de base de datos PostgreSQL. Almacena los datos estructurados de OpenPaDi y la configuración del servicio de autenticación Keycloak.
- **OP-web-1 (192.168.1.10)**: Funciona como el nodo master del clúster K3s. Orquesta los contenedores y aloja los servicios de autenticación (Keycloak), la interfaz de usuario (Frontend Svelte) y el gestor de tráfico entrante (Traefik).

Todas ellas tienen como SO Ubuntu Server 22.04. Para la comprobación de las pruebas se utilizó un cliente gráfico Debian 12.

4.3.1. Servicio de Base de Datos Relacional (PostgreSQL)



- **Motivo de su implantación:**

Este servicio es esencial para almacenar de forma persistente y estructurada los datos críticos tanto de la aplicación OpenPaDi (metadatos de documentos, transcripciones, información de usuarios) como del sistema de autenticación centralizada Keycloak (configuración de *realms*, clientes, usuarios, roles). La elección de un SGBD relacional garantiza la integridad, consistencia y recuperación de los datos.

- **Software empleado:**

- Sistema Gestor de Base de Datos (SGBD): PostgreSQL versión 16.
- Sistema Operativo de la Máquina Virtual: Ubuntu Server 22.04 LTS.

- **Archivos más relevantes:**

- <https://github.com/robleslf/OpenPaDi/blob/main/docs/infrastructure-configs/postgresql/postgresql.conf>

- https://github.com/robleslf/OpenPaDi/blob/main/docs/infrastructure-configs/postgresql/pg_hba.conf
- **Acceso a red y equipos:** se configuró PostgreSQL para escuchar peticiones en todas las interfaces de red disponibles. Esto se realizó modificando la directiva `listen_addresses = '*'` en el fichero `postgresql.conf`.

```
GNU nano 7.2 /etc/postgresql/16/main/postgresql.conf
# CONNECTIONS AND AUTHENTICATION
#-----
# - Connection Settings -

listen_addresses = '*'          # what IP address(es) to listen on;
                                # comma-separated list of addresses;
                                # defaults to 'localhost'; use '*' for >
                                # (change requires restart)
port = 5432                     # (change requires restart)
max_connections = 100           # (change requires restart)
```

2

Se establecieron, igualmente, reglas en el fichero `pg_hba.conf` para permitir conexiones exclusivamente desde las redes necesarias:

- Desde la red utilizada en este escenario para las máquinas virtuales (192.168.1.0/24) para el usuario `opadi_user` a la base de datos `opadi_db` (utilizada por la API de OpenPaDi).
- Desde la red interna del clúster K3s (10.42.0.0/16) para el usuario `keycloak_user` a la base de datos `keycloak_db` (utilizada por Keycloak).

#	TYPE	DATABASE	USER	ADDRESS	METHOD
host	opadi_db	opadi_user	192.168.1.0/24	md5	
host	keycloak_db	keycloak_user	10.42.0.0/16	scram-sha-256	
host	keycloak_db	keycloak_user	192.168.1.0/24	scram-sha-256	

3

² <https://github.com/robleslf/OpenPaDi/blob/main/docs/infrastructure-configs/postgresql/postgresql.conf>

³ https://github.com/robleslf/OpenPaDi/blob/main/docs/infrastructure-configs/postgresql/pg_hba.conf

- Se crearon usuarios y bases de datos específicas para cada servicio (*opadi_db* con *opadi_user* para OpenPaDi; *keycloak_db* con *keycloak_user* para Keycloak) con el fin de asegurar el aislamiento y una gestión de permisos modular.

List of databases									
Name	Owner	Encoding	Locale Provider	Collate	Ctype	ICU Locale	ICU Rules	Access privileges	
keycloak_db	keycloak_user	UTF8	libc	es_ES.UTF-8	es_ES.UTF-8			=Tc/keycloak_user	+
								keycloak_user=CTc/keycloak_user	
opadi_db	postgres	UTF8	libc	es_ES.UTF-8	es_ES.UTF-8			=Tc/postgres	+
								postgres=CTc/postgres	+
								opadi_user=CTc/postgres	
postgres	postgres	UTF8	libc	es_ES.UTF-8	es_ES.UTF-8			=c/postgres	+
template0	postgres	UTF8	libc	es_ES.UTF-8	es_ES.UTF-8			postgres=CTc/postgres	
								=c/postgres	+
template1	postgres	UTF8	libc	es_ES.UTF-8	es_ES.UTF-8			postgres=CTc/postgres	

```

administrator@op-db-primary:~$ sudo -u postgres psql
psql (16.8 (Ubuntu 16.8-0ubuntu0.24.04.1))
Type "help" for help.

postgres=# \l
postgres=# \du

                                List of roles
   Role name   |                               Attributes
-----+-----
keycloak_user |
opadi_user    |
postgres      | Superuser, Create role, Create DB, Replication, Bypass RLS

postgres=#

```

- *opadi_db* (Base de Datos) y *opadi_user* (Usuario): Almacena los datos específicos de la aplicación OpenPaDi, como información sobre documentos, transcripciones y usuarios de la plataforma. *opadi_user* es el usuario que usa la API para leer y escribir estos datos.
- *keycloak_db* (Base de Datos) y *keycloak_user* (Usuario): Garantiza la persistencia de Keycloak, al guardar toda la configuración y los datos de dicho sistema de autenticación.
- Integración con Clientes:

La API de OpenPaDi (FastAPI) se configura con las credenciales de *opadi_user* para acceder a la base de datos *opadi_db*.

```

app = FastAPI()

DB_HOST = "192.168.1.13"
DB_NAME = "opadi_db"
DB_USER = "opadi_user"
DB_PASS = "abc123.."

```

4

```

def get_db_connection():
    logger.info("LOGGING: Intentando conectar a la base de datos PostgreSQL...")
    conn = psycopg2.connect(host=DB_HOST, database=DB_NAME, user=DB_USER, password=DB_PASS)
    logger.info("LOGGING: Conexión a PostgreSQL exitosa!")
    return conn

@app.on_event("startup")

```

5

Keycloak se configura mediante variables de entorno en su despliegue en K3s para conectarse a la base de datos *keycloak_db* con el usuario *keycloak_user*, especificando el host (DB-Primary), nombre de la base de datos y credenciales.

⁴ <https://github.com/robleslf/OpenPaDi/blob/main/backend-api/main.py>

⁵ Ibid.

```
GNU nano 7.2
env:
- name: KEYCLOAK_ADMIN
  value: "keycloakadmin"
- name: KEYCLOAK_ADMIN_PASSWORD
  value: "TuClaveSuperSeguraParaKeycloakAdmin"
- name: KC_PROXY
  value: "edge"
- name: KC_DB
  value: "postgres"
- name: KC_DB_URL_HOST
  value: "192.168.1.13"
- name: KC_DB_URL_DATABASE
  value: "keycloak_db"
- name: KC_DB_URL_PORT
  value: "5432"
- name: KC_DB_USERNAME
  value: "keycloak_user"
- name: KC_DB_PASSWORD
  value: "abc123.."
- name: KC_DB_SCHEMA
  value: "public"
- name: KC_HOSTNAME_STRICT
  value: "false"
- name: KC_HOSTNAME_STRICT_BACKCHANNEL
  value: "false"
- name: KC_HTTP_ENABLED
  value: "true"
- name: KC_HTTP_PORT
  value: "8080"
- name: KC_HOSTNAME_URL
  value: "https://auth.openpadi.local"
- name: KC_HOSTNAME_ADMIN_URL
  value: "https://auth.openpadi.local"
ports:
- name: http
```

4.3.2. Servicio de Almacenamiento de Objetos (MinIO)



- **Motivo de su implantación:**

Este servicio es necesario para gestionar de forma eficiente el almacenamiento y acceso a ficheros de gran tamaño y datos no estructurados, como son los documentos históricos digitalizados (imágenes, PDFs) que se subirán a la plataforma

⁶ <https://github.com/robleslf/OpenPaDi/blob/main/kubernetes-manifests/keycloak-deployment.yaml>

OpenPaDi. Un sistema de almacenamiento de objetos compatible con la API S3 ofrece escalabilidad, durabilidad y una interfaz estándar para la gestión de estos archivos por parte de la aplicación.

- **Software empleado:**

- Servidor de Almacenamiento de Objetos: MinIO Server (RELEASE.2025-04-22T22-12-26Z).
- Despliegue: sobre máquina virtual Ubuntu Server 22.04 LTS.

- **Archivos más relevantes:**

- <https://github.com/robleslf/OpenPaDi/blob/main/docs/infrastructure-configs/minio/minio.conf>
- <https://github.com/robleslf/OpenPaDi/blob/main/docs/infrastructure-configs/minio/minio.service>

- **Instalación:**

MinIO Server se instaló como un binario ejecutable descargado directamente de su sitio oficial y ubicado en `/usr/local/bin/minio` en la VM OP-Storage-1.

```
administrator@op-storage-1:~$ ls /usr/local/bin/minio
/usr/local/bin/minio
administrator@op-storage-1:~$
```

- **Modo de Despliegue:**

En el entorno de pruebas actual, MinIO se ejecuta en modo *standalone* sobre un único directorio del sistema de ficheros de la VM.

Se destinó el directorio `/srv/minio/data` para el almacenamiento persistente de los objetos. Es importante que el usuario bajo el cual se ejecuta MinIO tenga permisos de lectura y escritura sobre este directorio.


```

administrador@op-storage-1:~$ ls /srv/minio/data -l
total 4
drwxrwxr-x 10 administrador administrador 4096 may 20 13:57 openpadi-documentos
administrador@op-storage-1:~$ ls /srv/minio/data -ld
drwxr-xr-x 4 administrador administrador 4096 may 18 11:27 /srv/minio/data
administrador@op-storage-1:~$

```

- **Gestión del Servicio con systemd:**

Para asegurar que MinIO se inicie automáticamente con el sistema y se gestione de forma centralizada, se creó un fichero de servicio systemd⁷.

El servicio se configuró para ejecutarse bajo el usuario *administrador* y se estableció el directorio de trabajo y las opciones de inicio.

```

feliperobleslopez@debian-base-gui: ~/O... x administrador@op-db-primary: ~ x administrador@op-web-1: ~ x administrador@op
administrador@op-storage-1:~$ systemctl status minio.service
• minio.service - MinIO Object Storage Server
   Loaded: loaded (/etc/systemd/system/minio.service; enabled; preset: enabled)
   Active: active (running) since Sat 2025-05-24 18:36:53 UTC; 2h 2min ago
     Docs: https://docs.min.io
   Main PID: 856 (minio)
    Tasks: 9 (limit: 4609)
   Memory: 342.6M (peak: 343.2M)
      CPU: 8.329s
   CGroup: /system.slice/minio.service
           └─856 /usr/local/bin/minio server --console-address :9001 /srv/minio/data

may 24 18:36:53 op-storage-1 systemd[1]: Started minio.service - MinIO Object Storage Server.
may 24 18:36:55 op-storage-1 minio[856]: MinIO Object Storage Server
may 24 18:36:55 op-storage-1 minio[856]: Copyright: 2015-2025 MinIO, Inc.
may 24 18:36:55 op-storage-1 minio[856]: License: GNU AGPLv3 - https://www.gnu.org/licenses/agpl-3.0.html
may 24 18:36:55 op-storage-1 minio[856]: Version: RELEASE.2025-04-22T22-12-26Z (go1.24.2 linux/amd64)
may 24 18:36:55 op-storage-1 minio[856]: API: http://192.168.1.100:9000 http://10.0.2.15:9000 http://192.168.1.14:9000 http://127.0.0.1:9000
may 24 18:36:55 op-storage-1 minio[856]: WebUI: http://192.168.1.100:9001 http://10.0.2.15:9001 http://192.168.1.14:9001 http://127.0.0.1:9001
may 24 18:36:55 op-storage-1 minio[856]: Docs: https://docs.min.io
administrador@op-storage-1:~$

```

- **Credenciales de Acceso:**

Las credenciales raíz para acceder al servidor MinIO (tanto a la API como a la consola web) se establecieron mediante variables de entorno, que se definieron en el archivo */etc/default/minio*⁸ de la siguiente manera.

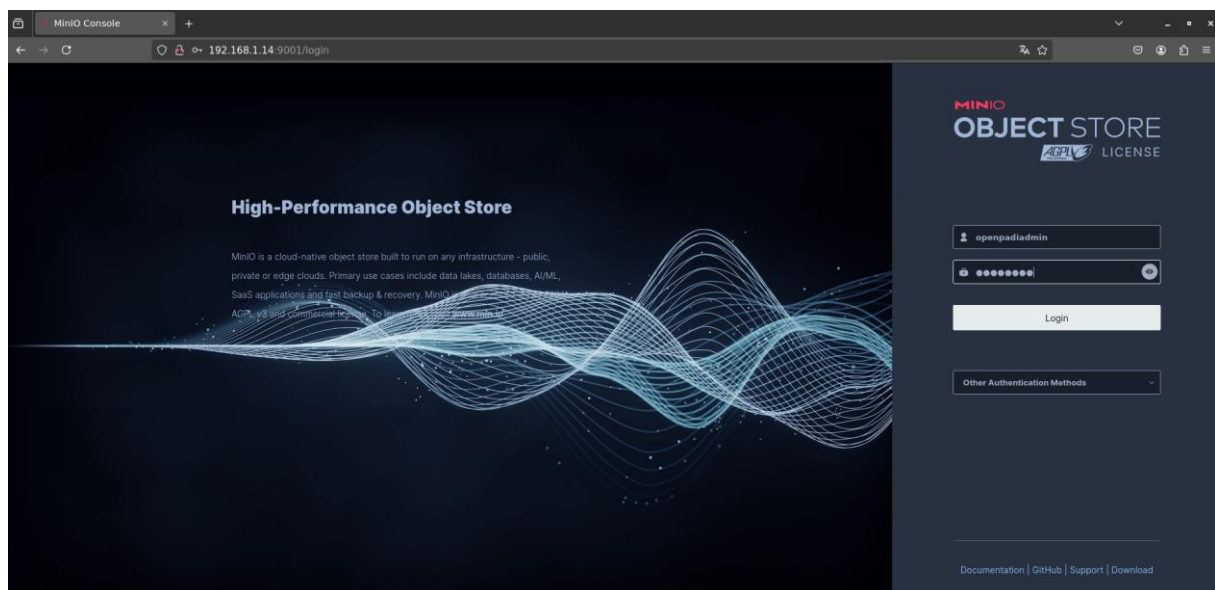
⁷ <https://github.com/robleslf/OpenPaDi/blob/main/docs/infrastructure-configs/minio/minio.service>

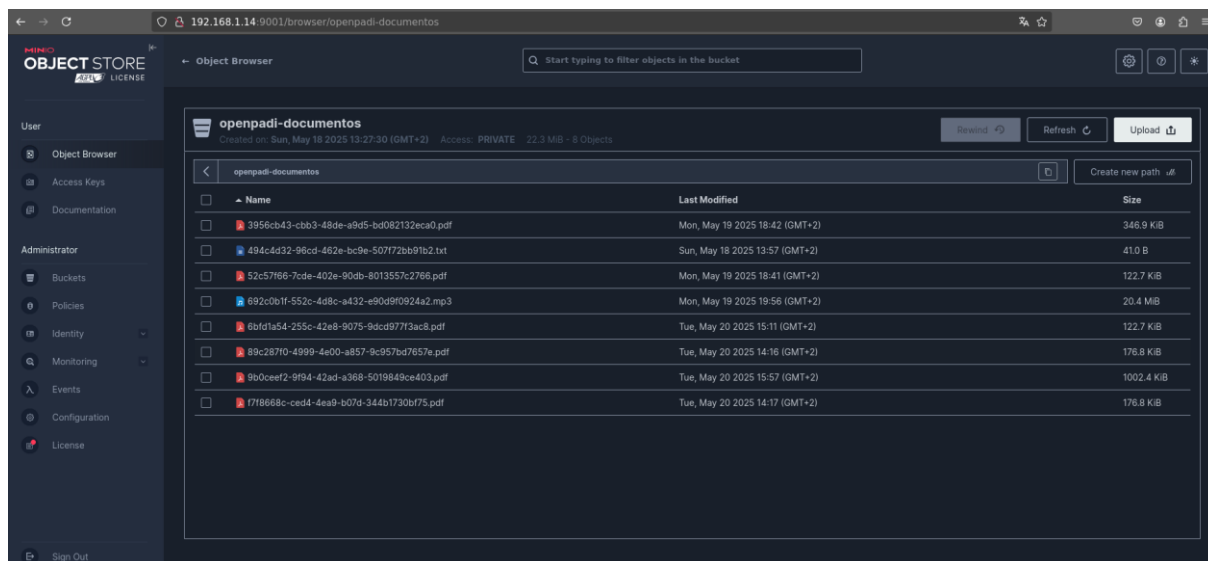
⁸ <https://github.com/robleslf/OpenPaDi/blob/main/docs/infrastructure-configs/minio/minio.conf>

```
administrator@op-storage-1:~$ cat /etc/default/minio
MINIO_ROOT_USER="openpadiadmin"
MINIO_ROOT_PASSWORD="abc123.."
MINIO_VOLUMES="/srv/minio/data"
MINIO_OPTS="--console-address :9001"
administrator@op-storage-1:~$
```

- **Puntos de Acceso (Endpoints) y Red:**

- API S3: MinIO expone su API en el puerto 9000 (HTTP) por defecto en la IP de la VM OP-Storage-1 (192.168.1.14:9000, o 10.0.40.10:9000 en el diseño de Proxmox).
- Consola Web: La interfaz gráfica de usuario para la gestión de MinIO se configuró para estar accesible en el puerto 9001 (HTTP) mediante la opción `--console-address ":9001"` de la captura anterior.





- **Buckets:**

Se creó manualmente un *bucket* principal llamado *openpadi-documentos*, como se puede ver en la captura anterior. Este *bucket* es el contenedor donde la aplicación OpenPaDi almacenará todos los ficheros de documentos digitalizados.

- **Integración con la API de OpenPaDi:**

La API FastAPI se configura con el *endpoint* del servidor MinIO, la clave de acceso y la clave secreta para poder interactuar con el *bucket* *openpadi-documentos* y realizar operaciones de subida, descarga y borrado de archivos.

```
DB_PASS = "abc123.."

MINIO_ENDPOINT = "192.168.1.14:9000"
MINIO_ACCESS_KEY = "openpadiadmin"
MINIO_SECRET_KEY = "abc123.."
MINIO_BUCKET_NAME = "openpadi-documentos"
MINIO_USE_SSL = False
```

9

La inicialización del cliente MinIO en la API se realiza como se muestra en el fichero *main.py*:

⁹ <https://github.com/robleslf/OpenPaDi/blob/main/backend-api/main.py>

```

try:
    minio_client = Minio(
        MINIO_ENDPOINT,
        access_key=MINIO_ACCESS_KEY,
        secret_key=MINIO_SECRET_KEY,
        secure=MINIO_USE_SSL
    )
except Exception as e:
    logger.critical(f"LOGGING: FATAL: Error al inicializar el cliente MinIO: {e}")
    minio_client = None

```

10

- **Planificación de Alta Disponibilidad y Resiliencia:**

Para un entorno de producción, MinIO se desplegaría en modo distribuido utilizando múltiples VMs y múltiples discos por VM. Esto proporcionaría redundancia y protección contra la pérdida de datos en caso de fallo de uno o varios discos o nodos.

Este diseño no se ha implementado en el entorno de pruebas actual por simplicidad y limitaciones de recursos.

4.3.3. Servicio de Autenticación y Autorización Centralizada (Keycloak)



- **Motivo de su implantación:**

Para gestionar de forma centralizada la identidad de los usuarios, controlar el acceso a las diferentes partes de la aplicación OpenPaDi (Frontend y API), y facilitar funcionalidades como el inicio de sesión único (SSO), se implementó Keycloak. Esto permite desacoplar la lógica de autenticación de las aplicaciones, mejorando la seguridad y la mantenibilidad, y ofreciendo una base para futuras integraciones o políticas de acceso más complejas.

- **Software empleado:**

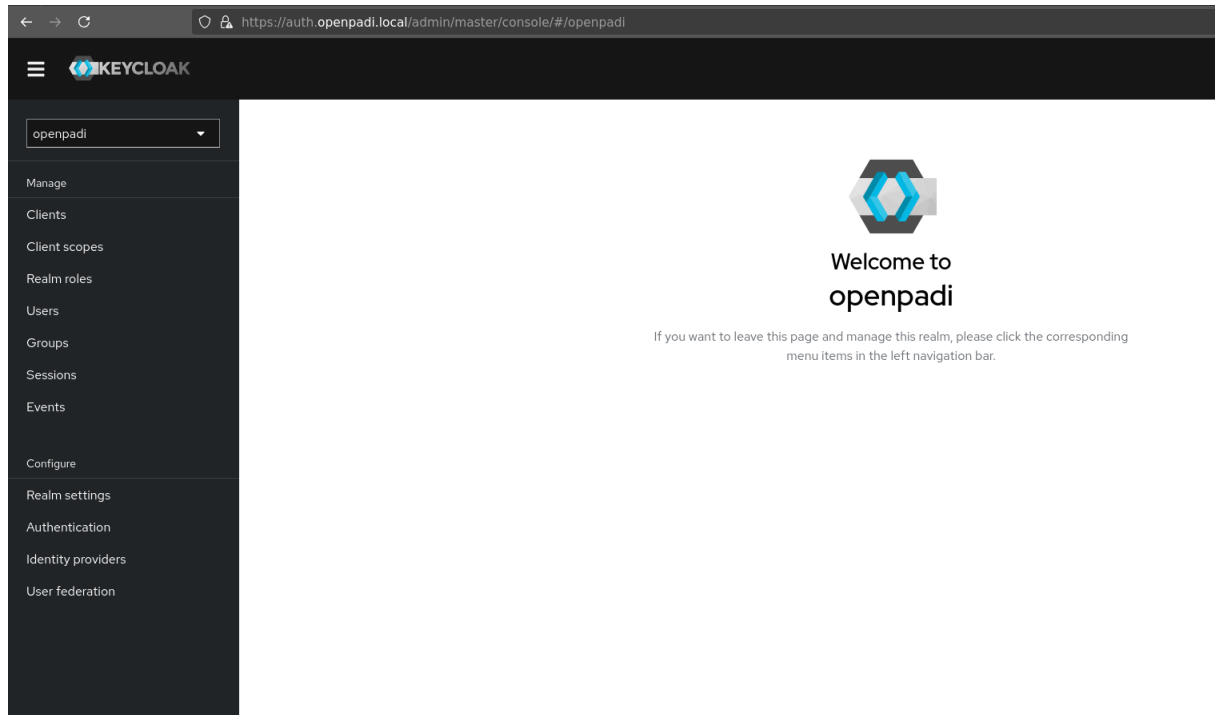
- Servidor de Identidad y Acceso: Keycloak versión 24.0.4.

¹⁰ [Ibid.](#)

- Despliegue: Ejecutándose como un *Pod* dentro del clúster K3s, en la máquina virtual OP-web-1 (nodo master de K3s).
 - Imagen Docker: *quay.io/keycloak/keycloak:24.0.4*.
 - Persistencia de Datos: Utiliza la base de datos *keycloak_db* en el servidor PostgreSQL externo (OP-db-primary), como se detalló en la sección 4.3.1.
- **Archivos más relevantes:**
 - <https://github.com/robleslf/OpenPaDi/blob/main/kubernetes-manifests/keycloak-deployment.yaml>
 - <https://github.com/robleslf/OpenPaDi/blob/main/kubernetes-manifests/keycloak-service.yaml>
 - https://github.com/robleslf/OpenPaDi/tree/main/frontend-svelte/keycloak_theme/openpadi-theme/login
- **Despliegue en K3s:**
 - Keycloak se despliega mediante un manifiesto de Kubernetes¹¹ que define un *Deployment* y un *Service*.
 - Se utilizan variables de entorno para la configuración inicial del administrador de Keycloak (*KEYCLOAK_ADMIN*, *KEYCLOAK_ADMIN_PASSWORD*) y la conexión a la base de datos PostgreSQL (ya mostradas anteriormente).
 - Se configura para ejecutarse detrás de un proxy inverso (*KC_PROXY="edge"*) y se definen las URLs públicas (*KC_HOSTNAME_URL*, *KC_HOSTNAME_ADMIN_URL*) para que coincidan con cómo se expone a través de Traefik.
- **Exposición del Servicio:**

¹¹ <https://github.com/robleslf/OpenPaDi/blob/main/kubernetes-manifests/keycloak-deployment.yaml>

- El servicio de Keycloak dentro de K3s (puerto 8080) se expone externamente a través del *Ingress Controller* Traefik en la URL <https://auth.openpadi.local>.



Servicio Keycloak en <https://auth.openpadi.local>

12

¹² Consola de administración de Keycloak, en la imagen aparece el Realm OpenPaDi, desde donde se administrarían los usuarios, roles, grupos...de OpenPaDi

Clients

Clients are applications and services that can request authentication of a user. [Learn more](#)

Clients list

Initial access token

Client registration

Q

Search for client

→

Create client

Import client

↺

Refresh

Client ID	Name	Type	Description
account	<code>\${client_account}</code>	OpenID Connect	–
account-console	<code>\${client_account-console}</code>	OpenID Connect	–
admin-cli	<code>\${client_admin-cli}</code>	OpenID Connect	–
broker	<code>\${client_broker}</code>	OpenID Connect	–
openpadi-api	OpenPaDi API Backend	OpenID Connect	Cliente para la API de OpenPaDi
openpadi-frontend	OpenPaDi Svelte Frontend	OpenID Connect	Cliente para la interfaz de usuario Svelte
realm-management	<code>\${client_realm-management}</code>	OpenID Connect	–
security-admin-console	<code>\${client_security-admin-console}</code>	OpenID Connect	–

Cientes del realm de OpenPaDi

De aquí nos interesan sobre todo los dos clientes que se crearon: *openpadi-api* y *openpadi-frontend*, que son los que definen cómo la interfaz de usuario y el servicio *backend*, respectivamente, interactúan con *Keycloak* para la autenticación de usuarios y la protección de los recursos de la aplicación. El cliente *openpadi-frontend* gestiona el proceso de inicio de sesión del usuario, mientras que *openpadi-api* se encarga de validar los tokens de acceso para autorizar las peticiones a los datos y funcionalidades de OpenPaDi.

Se les definieron las rutas correspondientes a cada uno para que puedan manejar el tráfico correctamente:

General settings

Client ID * ?

openpadi-api

Name ?

OpenPaDi API Backend

Description ?

Cliente para la API de OpenPaDi

Always display in UI ?



Off

Access settings

Root URL ?

https://openpadi.local/api

Home URL ?

https://openpadi.local/api

Valid redirect URIs ?

https://openpadi.local/api/*

[+ Add valid redirect URIs](#)

Rutas del cliente openpadi-api, encargado de validar los tokens de acceso para autorizar peticiones de datos

General settings

Client ID * ?

openpadi-frontend

Name ?

OpenPaDi Svelte Frontend

Description ?

Cliente para la interfaz de usuario Svelte

Always display in UI ?

☐ Off

Access settings

Root URL ?

https://openpadi.local

Home URL ?

https://openpadi.local

Valid redirect URIs ?

https://openpadi.local/*

[+ Add valid redirect URIs](#)

Valid post logout
redirect URIs ?

https://openpadi.local/*

[+ Add valid post logout redirect URIs](#)

Igualmente, al cliente *openpadi-api* se le creo un *Secret* para que pueda autenticar al propio servicio API ante *Keycloak*, necesario, por ejemplo, para la subida y bajada de documentos.

Al acceder al dominio de OpenPaDi aparecería una primera página que se encargaría de comprobar si el usuario está logueado mediante un token JWT; en caso de comprobar que no, le mostraría la opción de hacerlo:



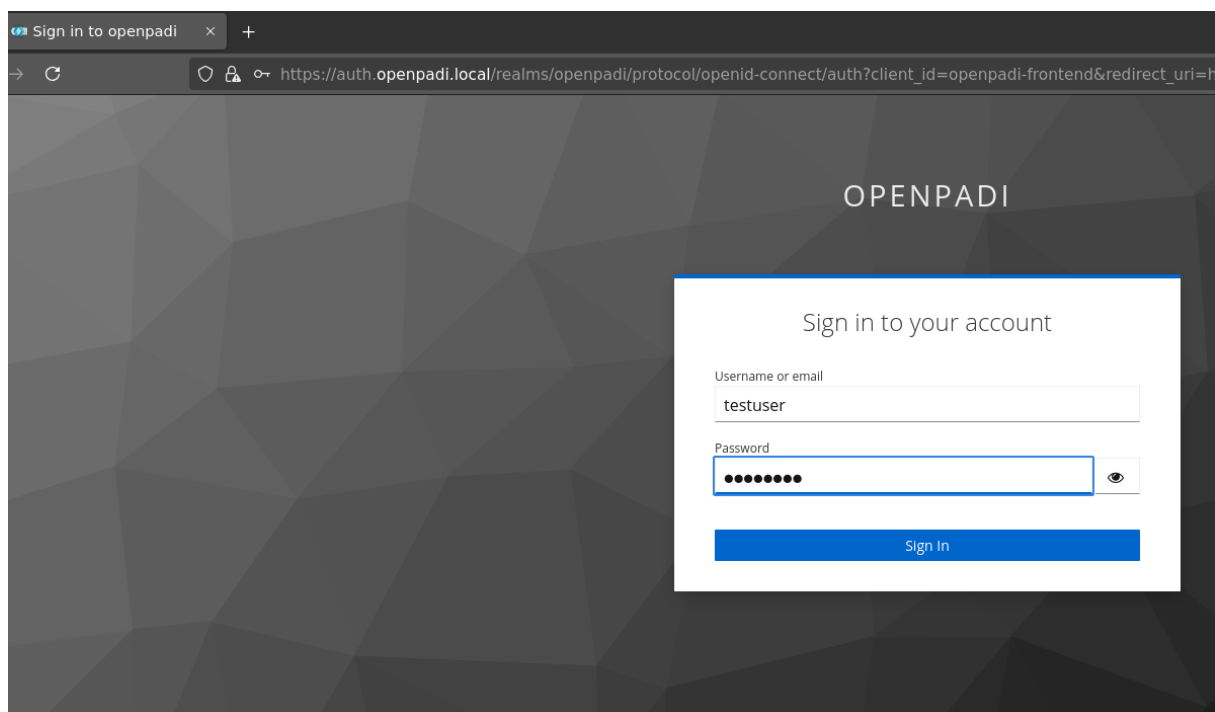
OpenPaDi - Repositorio de Transcripciones

Por favor, inicia sesión para continuar.

Iniciar Sesión con Keycloak

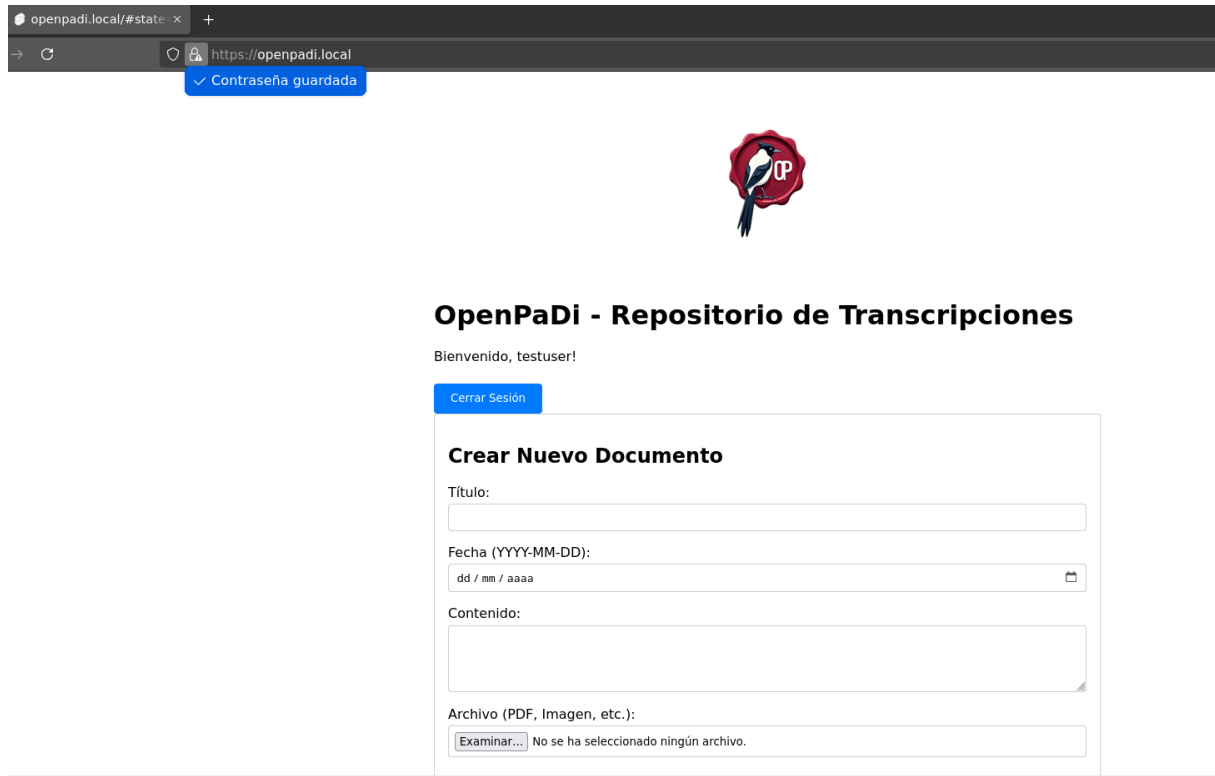
Página principal de OpenPaDi en <https://openpadi.local> sin estar autenticado

Esto le llevaría a la página de autenticación del servicio Keycloak:



Página redirigida desde <https://openpadi.local> para autenticarse mediante el servicio Keycloak

Al introducir uno de los usuarios creados en la consola de administración antes mencionada, sería de nuevo redirigido a OpenPaDi, esta vez con los permisos que se hayan fijado para este usuario en *Keycloak*:



Página principal en <https://openpadi.local> una vez autenticado

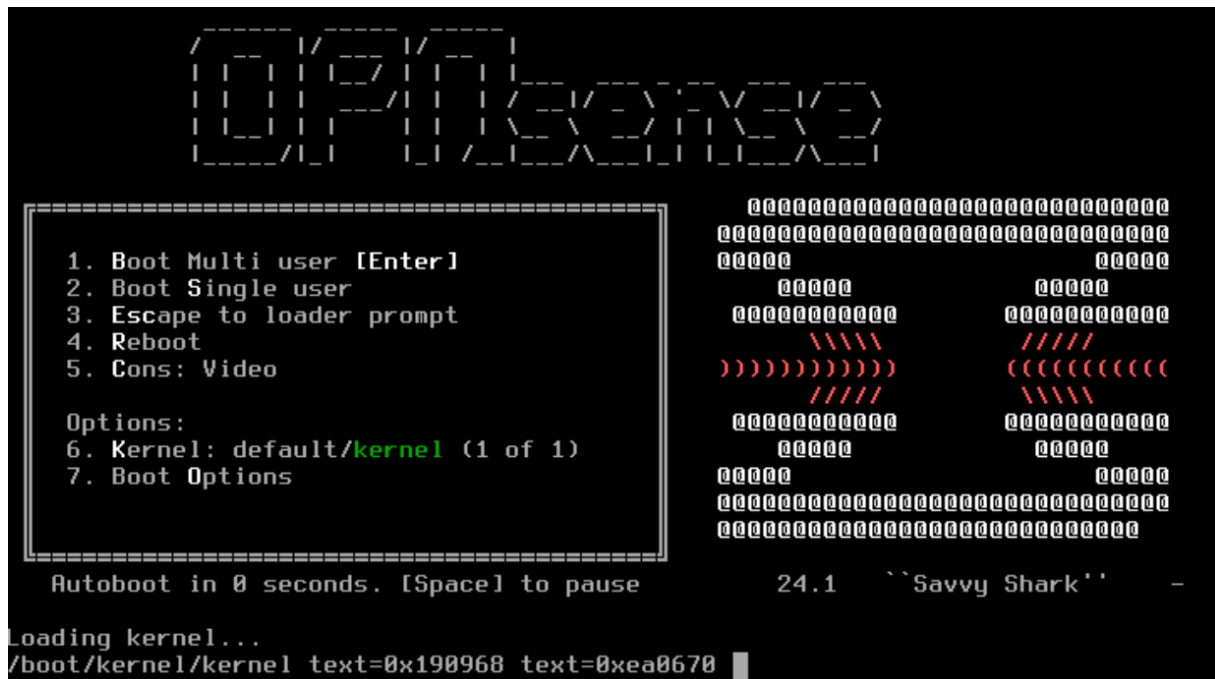
- *Traefik* se encarga de la terminación TLS para esta URL, utilizando certificados gestionados por Cert-Manager.

```
spec:
  ingressClassName: traefik
  tls:
    - hosts:
        - openpadi.local
        - auth.openpadi.local
      secretName: openpadi-tls
  rules:
    - host: openpadi.local
      http:
        paths:
```

13

¹³ <https://github.com/robleslf/OpenPaDi/blob/main/kubernetes-manifests/ingress.yaml>

4.3.4. Servicio Firewall/Router (OPNsense)



- **Motivo de su implantación:**

Este servicio es fundamental para establecer y mantener la seguridad perimetral de la infraestructura PaaS de OpenPaDi, así como para gestionar la segmentación de la red interna y controlar el flujo de tráfico entre las diferentes zonas funcionales (VLANs). Su implementación permite aislar los componentes críticos, aplicar políticas de seguridad modulares y proteger la plataforma contra amenazas externas e internas. Se eligió OPNsense, como solución de *firewall/router* por ser de código abierto y ofrecer un gran número de funcionalidades y personalización.

- **Software empleado:**

- ISO Firewall/Router: OPNsense 24.1¹⁴.

¹⁴ Se optó por esta imagen y no por la 25.1, más reciente, debido a que esta última dió errores importantes a la hora de trabajar en Proxmox, añadiendo más complicaciones a la virtualización anidada.

- Plataforma de Despliegue: Máquina Virtual dentro del entorno de virtualización Proxmox VE.



OPNsense-24.1-dvd-amd64.iso

- Interfaces de Red en la VM OPNsense:

```
*** OPNsense.localdomain: OPNsense 24.1 ***

ALMACENAMIENTO (vtnet4) -> v4: 10.0.40.1/24
APLICACIONES (vtnet2) -> v4: 10.0.20.1/24
DB (vtnet3)      -> v4: 10.0.30.1/24
DMZ (vtnet1)     -> v4: 10.0.10.1/24
LAN (vtnet6)     -> v4: 10.0.60.1/24
MONITORIZACION (vtnet5) -> v4: 10.0.50.1/24
PRUEBAS (vtnet7) -> v4: 10.0.70.1/24
WAN (vtnet0)     -> v4/DHCP4: 192.168.1.137/24
                  v6/DHCP6: 2a0c:5a83:8406:b600:be24:11ff:fed5:daae/64

HTTPS: SHA256 F1 72 77 9F E1 DE 47 D7 54 8A ED AC A4 64 6E 80
              C4 34 98 B5 D6 06 98 D8 87 80 AB 8C B2 7C A7 D7

FreeBSD/amd64 (OPNsense.localdomain) (ttyv0)
```

Configuración de VLANs en la MV OPNsense

⇌ Dispositivo de red (net0)	virtio=BC:24:11:D5:DA:AE,bridge=vmbr0
⇌ Dispositivo de red (net1)	virtio=BC:24:11:67:25:D8,bridge=vmbr10
⇌ Dispositivo de red (net2)	virtio=BC:24:11:E9:E3:01,bridge=vmbr20
⇌ Dispositivo de red (net3)	virtio=BC:24:11:8F:6A:2B,bridge=vmbr30
⇌ Dispositivo de red (net4)	virtio=BC:24:11:BF:44:9D,bridge=vmbr40
⇌ Dispositivo de red (net5)	virtio=BC:24:11:94:8E:FA,bridge=vmbr50
⇌ Dispositivo de red (net6)	virtio=BC:24:11:84:2D:F9,bridge=vmbr60
⇌ Dispositivo de red (net7)	virtio=BC:24:11:35:E9:CE,bridge=vmbr70

Hardware de red virtualizado en Proxmox para la MV OPNsense

Nombre ↑	Tipo	Activo	Inicio a...	Consci...	Puertos/Eslavos	Mod...	CIDR	Puerta de enlace	Comentario
enp0s3	Dispositivo de ...	Sí	No	No					
enp0s8	Dispositivo de ...	Sí	No	No					
vmbr0	Linux Bridge	Sí	Sí	No	enp0s3		192.168.1.138/24	192.168.1.1	
vmbr1	Linux Bridge	Sí	Sí	Sí	enp0s8				
vmbr10	Linux Bridge	Sí	Sí	No					VLAN DMZ 10.0.10.0/24
vmbr20	Linux Bridge	Sí	Sí	No					VLAN APLICACIONES 10.0.20.0/24
vmbr30	Linux Bridge	Sí	Sí	No					VLAN DB 10.0.30.0/24
vmbr40	Linux Bridge	Sí	Sí	No					VLAN ALMACENAMIENTO 10.0.40.0/24
vmbr50	Linux Bridge	Sí	Sí	No					VLAN MONITORIZACIÓN 10.0.50.0/24
vmbr60	Linux Bridge	Sí	Sí	No					VLAN ADMINISTRACIÓN 10.0.60.0/24
vmbr70	Linux Bridge	Sí	Sí	No					VLAN PRUEBAS 10.0.70.0/24

Hardware de red virtualizado en el nodo Proxmox para OpenPaDi; los bridges vmbr0 y vmbr1 son puertos de los adaptadores enp0s3 y enp0s8 de la MV de Proxmox en VirtualBox, que serían los adaptadores físicos del servidor en el que se alojase el hipervisor

- **WAN** (conectada a vmbr0 en Proxmox): Esta interfaz gestiona la conexión a Internet. En un entorno de producción, tendría una IP pública directamente o estaría detrás de un NAT del router del ISP.
- **DMZ** (conectada a vmbr10 en Proxmox): Sirve a la VLAN DMZ (10.0.10.0/24), donde residiría el Ingress Controller (Traefik).
- **APLICACIONES** (conectada a vmbr20 en Proxmox): Sirve a la VLAN de Aplicaciones (10.0.20.0/24), donde se ejecutan los nodos del clúster K3s (Frontend, Backend API, Keycloak).
- **DB** (conectada a vmbr30 en Proxmox): Sirve a la VLAN de Base de Datos (10.0.30.0/24) para PostgreSQL.
- **ALMACENAMIENTO** (conectada a vmbr40 en Proxmox): Sirve a la **VLAN** de Almacenamiento (10.0.40.0/24) para MinIO.
- **MONITORIZACION** (conectada a vmbr50 en Proxmox): Sirve a la VLAN de Monitorización (10.0.50.0/24). En este proyecto no tiene ninguna función ya que la implementación de los servicios de monitorización no se llevará a cabo, pero se ha optado por su creación igualmente para simular la misma distribución que habría en el entorno real de producción.
- **LAN/ADMINISTRACION** (conectada a vmbr60 en Proxmox): Sirve a la VLAN de Administración (10.0.60.0/24). En esta simulación ha servido para poder administrar, desde interfaz gráfica, las VLANs y reglas de firewall desde la interfaz web de OPNsense.
- **PRUEBAS** (conectada a vmbr70 en Proxmox): Sirve a la VLAN de Pruebas (10.0.70.0/24).

- **Enrutamiento Inter-VLAN:** Todo el tráfico que necesite pasar de una VLAN a otra debe ser enrutado a través de la VM OPNsense. OPNsense examinará este tráfico y aplicará las reglas de firewall correspondientes.
- **Políticas de Firewall:**

Estas son las reglas de firewall que se aplicaron en las distintas interfaces para la correcta comunicación entre ellas y el exterior. Hay que tener en cuenta que *OPNsense*, por defecto, restringe todas las comunicaciones y que se trata de un firewall *stateful*, por lo que todas las reglas aplicadas son *IN* sin necesidad de crear sus correspondientes reglas espejo.

○ INTERFAZ DMZ

☐	Protocol	Source	Port	Destination	Port	Gateway	Schedule	Description	
Automatically generated rules									
☐	IPv4 ICMP	DMZ net	*	LAN address	*	*	*		
☐	IPv4 TCP	*	*	10.0.20.0/24	80 (HTTP)	*	*	DMZ a APLICACIONES: Permitir HTTP	
☐	IPv4 TCP	*	*	10.0.20.0/24	8000	*	*	DMZ a APLICACIONES: Permitir Traefik al Backend API	
☐	IPv4 TCP	*	*	10.0.20.0/24	8080	*	*	DMZ a APLICACIONES: Permitir Traefik a Keycloak	
pass		block		reject				log	in
pass (disabled)		block (disabled)		reject (disabled)				log (disabled)	out
Active/Inactive Schedule (click to view/edit)									
Alias (click to view/edit)									
DMZ rules are evaluated on a first-match basis by default (i.e. the action of the first rule to match a packet will be executed). This means that if you use block rules, you will have to pay attention to the rule order. Everything that is not explicitly passed is blocked by default.									

Nº	Acción	Dirección	Protocolo	Origen	Puerto Origen	Destino	Puerto Destino	Fin
1	PASS	IN	IPV4 TCP	DMZ	*	APLICACIONES	80 (HTTP)	Permitir en DMZ el acceso HTTP en APLICACIONES
2	PASS	IN	IPV4 TCP	DMZ	*	APLICACIONES	8000	Permitir en DMZ el acceso al Backend API
3	PASS	IN	IPV4	DMZ	*	APLICACIONES	8080	Permitir en DMZ el acceso a Keycloak en APLICACIONES

○ INTERFAZ APLICACIONES

Firewall: Rules: APLICACIONES Select category Inspect

The changes have been applied successfully.

	Protocol	Source	Port	Destination	Port	Gateway	Schedule	Description
Automatically generated rules 18								
☐	IPv4 TCP	10.0.20.0/24	*	10.0.30.0/24	5432	*	*	APLICACIONES a DB: Permitir acceso a PostgreSQL
☐	IPv4 TCP	10.0.20.0/24	*	10.0.40.0/24	9000	*	*	APLICACIONES a ALMACENAMIENTO: Permitir acceso MinIO API
☐	IPv4 *	10.0.20.0/24	*	*	*	WAN_DHCP	*	APLICACIONES a Internet: Permitir trafico de salida general

☐ pass ☒ block ☒ reject ☒ log ☒ in ☒ first match
☐ pass (disabled) ☒ block (disabled) ☒ reject (disabled) ☒ log (disabled) ☒ out ☒ last match

Active/Inactive Schedule (click to view/edit)

[Alias \(click to view/edit\)](#)

Nº	Acción	Dirección	Protocolo	Origen	Puerto Origen	Destino	Puerto Destino	Gateway	Fin
1	PASS	IN	IPV4 TCP	APLICACIONES	*	DB	5432 (POSTGRES QL)	*	Permitir acceso a PostgreSQL desde APLICACIONES
2	PASS	IN	IPV4 TCP	APLICACIONES	*	ALMACENAMIENTO	9000 (MINIO)	*	Permitir acceso a la API de MinIO desde APLICACIONES
3	PASS	IN	IPV4 TCP	APLICACIONES	*	ANY	ANY	WAN	Permitir salida general a Internet desde APLICACIONES

○ INTERFAZ DB

Firewall: Rules: DB

Select category

Inspect

The changes have been applied successfully.

☐	Protocol	Source	Port	Destination	Port	Gateway	Schedule	Description	☐	☐	☐	☐
Automatically generated rules										18		
☐	IPv4 TCP	DB net	*	*	80 (HTTP)	WAN_DHCP	*	DB a Internet: Actualizaciones HTTP	☐	☐	☐	☐
☐	IPv4 TCP	DB net	*	*	443 (HTTPS)	WAN_DHCP	*	DB a Internet: Actualizaciones HTTPS	☐	☐	☐	☐
☐	pass	☒	block	☒	reject	☒	log	☒	in	☒	first match	
☐	pass (disabled)	☒	block (disabled)	☒	reject (disabled)	☒	log (disabled)	☒	out	☒	last match	
Active/Inactive Schedule (click to view/edit)												
Alias (click to view/edit)												
DB rules are evaluated on a first-match basis by default (i.e. the action of the first rule to match a packet will be executed). This means that if you use block rules, you will have to pay attention to the rule order. Everything that is not explicitly passed is blocked by default.												

Nº	Acción	Dirección	Protocolo	Origen	Puerto Origen	Destino	Puerto Destino	Gateway	Fin
1	PASS	IN	IPV4 TCP	DB	*	ANY	80 (HTTP)	WAN	Permitir a la DB descargar actualizaciones por HTTP
2	PASS	IN	IPV4 TCP	DB	*	ANY	443 (HTTPS)	WAN	Permitir a la DB descargar actualizaciones por HTTPS

○ INTERFAZ ALMACENAMIENTO

Firewall: Rules: ALMACENAMIENTO

Select category

Inspect

The changes have been applied successfully.

☐	Protocol	Source	Port	Destination	Port	Gateway	Schedule	Description	☐	☐	☐	☐
Automatically generated rules										18		
☐	IPv4 TCP	ALMACENAMIENTO net	*	*	80 (HTTP)	WAN_DHCP	*	ALMACENAMIENTO a Internet: Permitir actualizaciones HTTP	☐	☐	☐	☐
☐	IPv4 TCP	ALMACENAMIENTO net	*	*	443 (HTTPS)	WAN_DHCP	*	ALMACENAMIENTO a Internet: Permitir actualizaciones HTTPS	☐	☐	☐	☐
☐	pass	☒	block	☒	reject	☒	log	☒	in	☒	first match	
☐	pass (disabled)	☒	block (disabled)	☒	reject (disabled)	☒	log (disabled)	☒	out	☒	last match	
Active/Inactive Schedule (click to view/edit)												
Alias (click to view/edit)												
ALMACENAMIENTO rules are evaluated on a first-match basis by default (i.e. the action of the first rule to match a packet will be executed). This means that if you use block rules, you will have to pay attention to the rule order. Everything that is not explicitly passed is blocked by default.												

Nº	Acción	Dirección	Protocolo	Origen	Puerto o Origen	Destino	Puerto Destino	Gateway	Fin
1	PASS	IN	IPV4 TCP	ALMACENAMIENTO	*	ANY	80 (HTTP)	WAN	Permitir a la interfaz ALMACENAMIENTO descargar actualizaciones por HTTP
2	PASS	IN	IPV4 TCP	ALMACENAMIENTO	*	ANY	443 (HTTPS)	WAN	Permitir a la interfaz ALMACENAMIENTO descargar actualizaciones por HTTPS

○ INTERFAZ MONITORIZACIÓN

☐	Protocol	Source	Port	Destination	Port	Gateway	Schedule	Description					
☐	Automatically generated rules												
☐	IPv4 TCP	MONITORIZACION net	*	*	80 (HTTP)	WAN_DHCP	*	MONITORIZACION a Internet: Permitir actualizaciones HTTP					
☐	IPv4 TCP	MONITORIZACION net	*	*	443 (HTTPS)	WAN_DHCP	*	MONITORIZACION a Internet: Permitir actualizaciones HTTPS					
☐	IPv4 TCP	MONITORIZACION net	*	APLICACIONES net	*	*	*	MONITORIZACIÓN a APLICACIONES: Permitir monitorización					
☐	IPv4 TCP	MONITORIZACION net	*	DB net	*	*	*	MONITORIZACIÓN a DB: Permitir monitorización					
☐	IPv4 TCP	MONITORIZACION net	*	ALMACENAMIENTO net	*	*	*	MONITORIZACIÓN a ALMACENAMIENTO: Permitir monitorización					

© 2014-2024 Pihon BV

Nº	Acción	Dirección	Protocolo	Origen	Puerto o Origen	Destino	Puerto Destino	Gateway	Fin
1	PASS	IN	IPV4 TCP	MONITORIZACIÓN	*	ANY	80 (HTTP)	WAN	Permitir a la interfaz MONITORIZACIÓN descargar actualizaciones por HTTP

2	PASS	IN	IPV4 TCP	MONITORIZAC IÓN	*	ANY	443 (HTTP S)	WAN	Permitir a la interfaz MONITORIZACI ÓN descargar actualizaciones por HTTPS
3	PASS	IN	IPV4 TCP	MONITORIZAC IÓN	*	APLICACIONES	*	*	Permitir recoger métricas de APLICACIONES
4	PASS	IN	IPV4 TCP	MONITORIZAC IÓN	*	DB	*	*	Permitir recoger métricas de DB
5	PASS	IN	IPV4	MONITORIZAC IÓN	*	ALMACENAMIE NTO	*	*	Permitir recoger métricas de ALMACENAMIE NTO

Se están aplicando reglas poco restrictivas, dejando cualquier puerto para las VLANs de APLICACIONES, ALMACENAMIENTO y DB; estos pueden restringirse a puertos más específicos, como los 6433, 9100, 8080 que usa K3s, los de PostgreSQL, MinIO...

Igualmente, se está permitiendo solo métricas de estas VLANs, pero podrían hacerse reglas para monitorizar otras.

○ INTERFAZ ADMINISTRACIÓN

☐	Protocol	Source	Port	Destination	Port	Gateway	Schedule	Description	
Automatically generated rules									
☐	IPv4 *	LAN net	*	*	*	*	*	Default allow LAN to any rule	
☐	IPv6 *	LAN net	*	*	*	*	*	Default allow LAN IPv6 to any rule	
☐	IPv4 TCP	LAN net	*	LAN address	443 (HTTPS)	*	*	ADMINISTRACIÓN a OPNSENSE: Permitir acceso GUI	
☐	IPv4 *	LAN net	*	*	*	WAN_DHCP	*	ADMINISTRACIÓN a OPNSENSE: Permitir acceso GUI	
☐	IPv4 TCP	LAN net	*	APLICACIONES net	22 (SSH)	*	*	ADMINISTRACION a APLICACIONES: Permitir SSH	
☐	IPv4 TCP	LAN net	*	DB net	22 (SSH)	*	*	ADMINISTRACION a DB: Permitir SSH	
☐	IPv4 TCP	LAN net	*	ALMACENAMIENTO net	22 (SSH)	*	*	ADMINISTRACION a ALMACENAMIENTO: Permitir SSH	
☐	pass	☒ block	☒ reject	☒ log	☒ in	☒ first match			
☐	pass (disabled)	☒ block (disabled)	☒ reject (disabled)	☒ log (disabled)	☒ out	☒ last match			

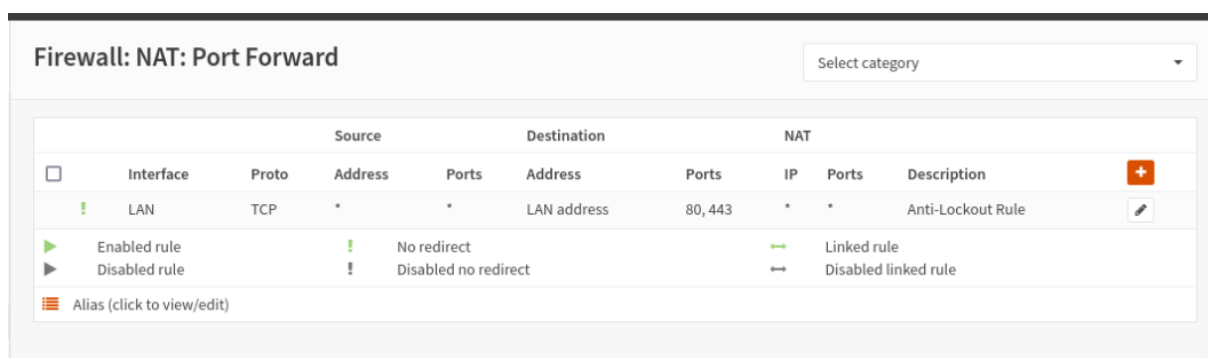
Nº	Acción	Dirección	Protocolo	Origen	Puerto o Origen	Destino	Puerto Destino	Gateway	Fin
1	PASS	IN	IPV4 TCP	LAN	*	LAN	443 (HTTPS)	*	Permitir acceso a la interfaz web de OPNsense
2	PASS	IN	IPV4 TCP	LAN	*	ANY	443 (HTTPS)	WAN	Permitir la salida a Internet
3	PASS	IN	IPV4 TCP	LAN	*	APLICACIONES	*	*	Permitir SSH a APLICACIONES
4	PASS	IN	IPV4 TCP	LAN	*	DB	*	*	Permitir SSH a DB
5	PASS	IN	IPV4	LAN	*	ALMACENAMIENTO	*	*	Permitir SSH a ALMACENAMIENTO
6	PASS	IN	IPV4	LAN	*	DMZ	*	*	Permitir SSH a DMZ

○ INTERFAZ PRUEBAS

Para esta interfaz no se definieron reglas en esta simulación. Para cualquier prueba necesaria se podrían ajustar las reglas específicas adecuadas para estas.

○ NAT

Además de las reglas anteriores, es imprescindible tener activado el *Port Forward* de NAT para que las reglas que permiten la salida a Internet funcionen correctamente:



Asimismo, en la salida NAT es recomendable seleccionar la siguiente opción:

Firewall: NAT: Outbound

Mode	
☒ Automatic outbound NAT rule generation (no manual rules can be used)	☐ Hybrid outbound NAT rule generation (automatically generated rules are applied after manual rules)
☐ Manual outbound NAT rule generation (no automatic rules are being generated)	☐ Disable outbound NAT rule generation (outbound NAT is disabled)

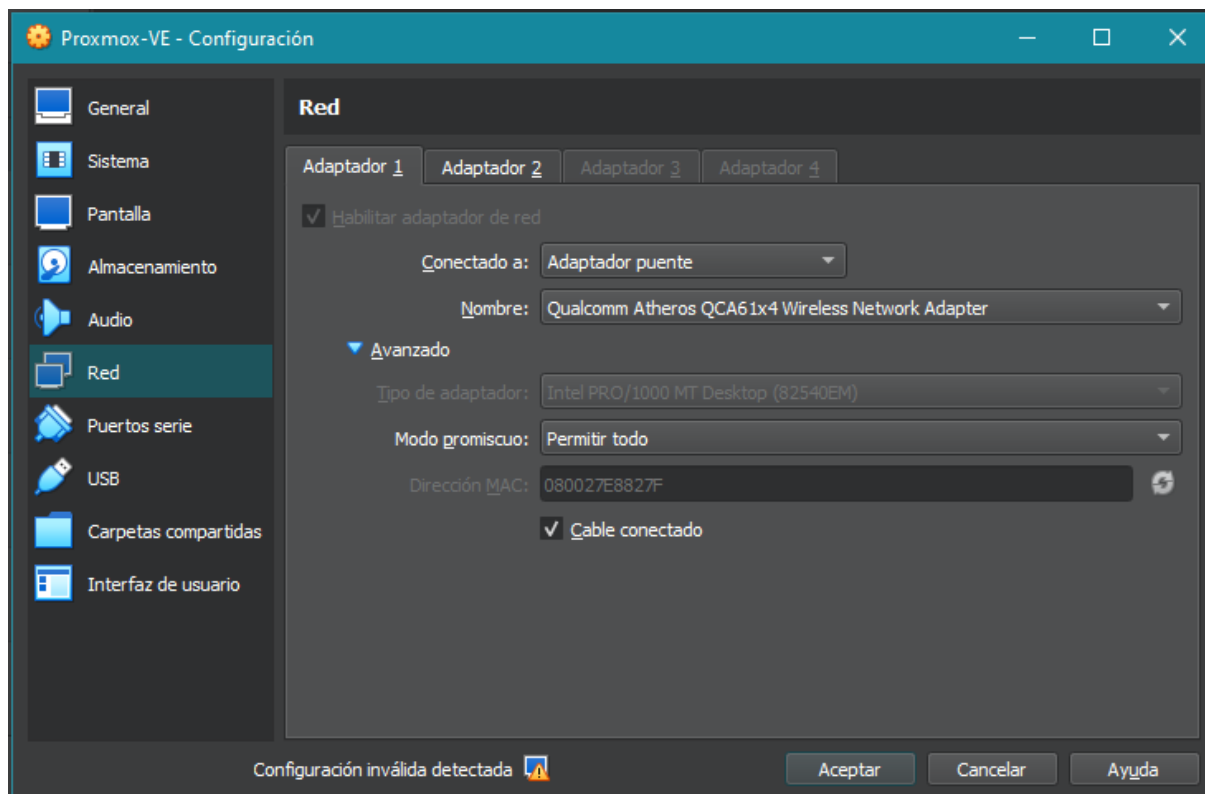
Aunque otras opciones permitirían añadir reglas más personalizadas. Para este escenario nos vale con el modo automático.

Igualmente, la interfaz WAN debe tener desmarcada la opción *Block private networks*:

Generic configuration

 Block private networks	☐
 Block bogon networks	☒

Como estamos utilizando una máquina virtual de Proxmox VE en VirtualBox, es también recomendable tener activado el modo promiscuo en el adaptador puente:



- **Integración con Otros Servicios:**

- Actúa como el *gateway* por defecto para todas las VMs en las diferentes VLANs.
- Controla el acceso al *Ingress Controller* (Traefik), que reside en la VLAN DMZ, reenviando el tráfico web externo a este.
- Media en la comunicación entre la VLAN de Aplicaciones (donde están K3s, API, Keycloak) y las VLANs de servicios de backend (DB, Almacenamiento).

- **Planificación de Alta Disponibilidad y Resiliencia:**

- Clúster HA de OPNsense: En un entorno de producción, se desplegarían dos VMs OPNsense en un clúster de alta disponibilidad utilizando la redundancia para la sincronización de estados y configuraciones. Cada VM OPNsense estaría conectada a una interfaz WAN distinta (idealmente de ISPs diferentes) y a todos los bridges de VLAN internas. (Como se ilustra en el diagrama de red con *Router_A* y *Router_B*).

4.3.5. Servicio DHCP (Gestionado por OPNsense)

A screenshot of a terminal window titled 'Máquina virtual 101 (op-router-fw) en el nodo proxmox'. The terminal shows the command 'ps aux | grep -i dhcp' being executed. The output lists three processes: '_dhcp' (PID 41165, PPID 0.0, USER 0.1, COMMAND 'dhcpd -t -f -u root -K 0:00:07'), 'dhcpd' (PID 58117, PPID 0.0, USER 0.4, COMMAND 'dhcpd -t -f -u root -K 0:00:10'), and 'root' (PID 45599, PPID 0.0, USER 0.1, COMMAND 'grep -i dhcp'). The terminal prompt is 'root@OPNsense:~ #'.

```
root@OPNsense:~ # ps aux | grep -i dhcp
_dhcp  41165  0.0  0.1 13844 2644 -  SCs  11:54   0:00.07 dhcpd: vtinet
dhcpd  58117  0.0  0.4 25664 8964 -  Is   11:54   0:00.10 /usr/local/sbin
root   45599  0.0  0.1 12720 2264 v0  S+   15:03   0:00.00 grep -i dhcp
root@OPNsense:~ #
```

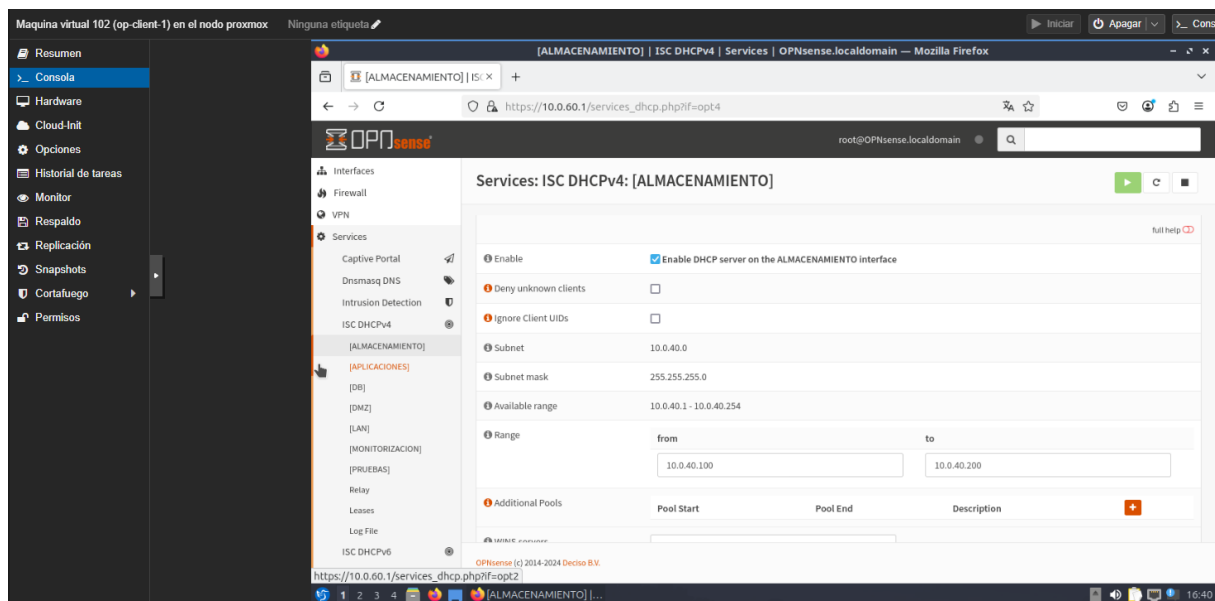
- **Motivo de su implantación:**

El servicio DHCP se implanta para automatizar y centralizar la asignación de configuraciones de red IP a las máquinas virtuales dentro de cada segmento de red lógico (VLAN) de la infraestructura. Esto simplifica enormemente la administración de la red, reduce la posibilidad de errores manuales en la configuración de IPs, y asegura que cada VM reciba automáticamente la información necesaria para operar en su red, como su dirección IP, máscara de subred, dirección del gateway y servidores DNS.

- **Software empleado:**

- Servidor DHCP: Se decidió utilizar el módulo DHCPD integrado en OPNsense (basado en ISC DHCPD o un demonio similar) debido a su estrecha integración con la plataforma firewall/router central, lo que simplifica la gestión y configuración al centralizar los servicios de red esenciales en un mismo nodo.
- Plataforma de Gestión: OPNsense, ejecutándose en la máquina virtual *op-router-fw* (ID 101) dentro de Proxmox VE.

La configuración del servicio DHCP se realiza directamente desde la interfaz web de OPNsense, habilitándolo por interfaz (VLAN). Para cada VLAN donde se requiera asignación dinámica de IPs, se configura un ámbito DHCP específico.



A modo de ejemplo podemos ver la configuración en la interfaz web de OPNsense que correspondería a la VLAN DB (10.0.30.0/24 en el diseño simulado en Proxmox):

Services: ISC DHCPv4: [DB]

Enable	☒ Enable DHCP server on the DB interface
Deny unknown clients	☐
Ignore Client UIDs	☐
Subnet	10.0.30.0
Subnet mask	255.255.255.0
Available range	10.0.30.1 - 10.0.30.254

Range

from	to
10.0.30.100	10.0.30.200

DNS servers	10.0.60.1
Gateway	10.0.30.1

Domain name	openpadi.local
Domain search list	openpadi.local

Se configuraron las opciones básicas para el correcto funcionamiento del servicio DHCP:

- Servicio habilitado.
- Subnet.
- Máscara de subnet.
- Rango de IPs.
- Servidor DNS.
- Gateway.
- Dominio.

Podemos comprobar que, efectivamente, las MVs conectadas a la VLAN 30 DB están recibiendo concesiones. La MV *op-db-primary* está conectada en su adaptador de red al vmbr30 del nodo Proxmox:

104 (op-db-primary)

Dispositivo de red (net0) virtio=BC:24:11:5D:5E:8F,bridge=vmbr30

Recordemos que en la MV de OPNsense esta VLAN corresponde a su adaptador vtnet3:

DB (vtnet3) -> v4: 10.0.30.1/24

Y este a su vez está conectado al vmbr30:

```
⇒ Dispositivo de red (net3)      virtio=BC:24:11:8F:6A:2B,bridge=vmbr30
```

(Es importante fijarse en la MAC aquí para asegurar que los adaptadores de red están conectados donde queremos).

La MV *op-db-primary* está configurada para recibir concesiones DHCP en su adaptador de red:

```
GNU nano 7.2 /etc/netplan/01-netcfg.yaml
network:
  version: 2
  ethernets:
    ens18:
      dhcp4: true
```

Y, por tanto, al aplicar la configuración de red, se le concede la configurada en la interfaz gráfica de OPNsense:

```
sadministrator@op-web-1openpadilocal:~$ sudo netplan apply
** (generate:1144): WARNING **: 14:57:53.979: Permissions for /etc/netplan/01-ne
.
** (process:1142): WARNING **: 14:57:54.543: Permissions for /etc/netplan/01-net
** (process:1142): WARNING **: 14:57:54.689: Permissions for /etc/netplan/01-net
administrator@op-web-1openpadilocal:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gro
    link/ether bc:24:11:5d:5e:8f brd ff:ff:ff:ff:ff:ff
    altname enp0s18
    inet 10.0.30.100/24 metric 100 brd 10.0.30.255 scope global dynamic ens18
        valid_lft 7198sec preferred_lft 7198sec
    inet6 fe80::be24:11ff:fe5d:5e8f/64 scope link
        valid_lft forever preferred_lft forever
administrator@op-web-1openpadilocal:~$ ip r
default via 10.0.30.1 dev ens18 proto dhcp src 10.0.30.100 metric 100
8.8.8.8 via 10.0.30.1 dev ens18 proto dhcp src 10.0.30.100 metric 100
10.0.30.0/24 dev ens18 proto kernel scope link src 10.0.30.100 metric 100
10.0.30.1 dev ens18 proto dhcp scope link src 10.0.30.100 metric 100
administrator@op-web-1openpadilocal:~$ _
```

Estas concesiones están almacenadas en el archivo `/var/dhcpd/var/db/dhcpd.leases` de la MV OPNsense:

```
root@OPNsense:~ # sudo cat /var/dhcpd/var/db/dhcpd.leases | less
```

```
lease 10.0.30.100 {
  starts 0 2025/05/25 14:57:59;
  ends 0 2025/05/25 16:57:59;
  cltt 0 2025/05/25 14:57:59;
  binding state active;
  next binding state free;
  rewind binding state free;
  hardware ethernet bc:24:11:5d:5e:8f;
  uid "\377\312S\011Z\000\002\000\000\253\021I\245\377\022\034\324\177:";
  client-hostname "op-web-1openpadilocal";
}
```

- **Habilitación del Servidor DHCP por Interfaz:**

Como se mencionó anteriormente, se ha habilitado el servidor DHCP específicamente para cada interfaz marcando la casilla correspondiente en cada una de las interfaces.



- **Centralización de la configuración de red**

Este servicio no solo nos permite realizar concesiones dinámicas a los nuevos equipos que se conecten a nuestra red en Proxmox, sino que podemos reservar direcciones estáticas para aquellos nodos que necesiten una IP conocida, como son los servidores que representan nuestras MVs; en el ejemplo de concesión anterior, se le asignó una IP más o menos aleatoria a nuestra MV *op-db-primary*; sin embargo, tendría más sentido reservarle una IP fija a este equipo, ya que los diferentes servicios de OpenPaDi van a necesitar saber llegar a esta máquina cuando necesiten

interactuar con la Base de Datos de PostgreSQL que tiene configurada. En este caso, se debe configurar del siguiente modo:

- 1) Miraremos la MAC del adaptador de red de la máquina a la que le vamos a asignar la IP fija:

Maquina virtual 104 (op-db-primary) en el nodo proxmox

Ninguna etiqueta

Resumen

Consola

Hardware

Cloud-Init

Opciones

Historial de tareas

Monitor

Respaldo

Replicación

Agregar

Eliminar

Editar

Acción de disco

Revertir

Memoria

Procesadores

BIOS

Pantalla

Maquina

Controlador SCSI

Dispositivo CD/DVD (ide2)

Disco duro (scsi0)

Dispositivo de red (net0)

4.00 GiB

2 (1 sockets, 2 cores) [kvm64]

Por defecto (SeaBIOS)

Por defecto

Por defecto (i440fx)

VirtIO SCSI single

none,media=cdrom

local-lvm:vm-104-disk-0,ioread=1,size=20G

virtio=BC:24:11:5D:5E:8F,bridge=vmbr30

- 2) En la interfaz de OPNsense, iremos al servicio DHCP4 y asignaremos la IP fija a esta MAC:

Static DHCP Mapping

MAC address
Copy my MAC address

Client identifier

IP address

Hostname

Description

ARP Table Static Entry ☐

3) Renovamos la concesión DHCP en la máquina *op-db-primary*:

Maquina virtual 104 (op-db-primary) en el nodo proxmox Ninguna etiqueta

Resumen
Consola
Hardware
Cloud-Init
Opciones
Historial de tareas
Monitor
Respaldo
Replicación
Snapshots
Cortafuego
Permisos

```

administrator@op-web-1openpadilocal:~$ sudo systemctl restart systemd-networkd
[sudo] password for administrator:
administrator@op-web-1openpadilocal:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether bc:24:11:5d:5e:8f brd ff:ff:ff:ff:ff:ff
    altname enp0s18
    inet 10.0.30.13/24 metric 100 brd 10.0.30.255 scope global dynamic ens18
        valid_lft 7197sec preferred_lft 7197sec
    inet6 fe80::be24:11ff:fe5d:5e8f/64 scope link
        valid_lft forever preferred_lft forever
administrator@op-web-1openpadilocal:~$

```

```

2: ens18: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500
    link/ether bc:24:11:5d:5e:8f brd ff:ff:ff:ff:ff:ff
    altname enp0s18
    inet 10.0.30.13/24 metric 100 brd 10.0.30.255 scope global dynamic ens18
        valid_lft 7197sec preferred_lft 7197sec
    inet6 fe80::be24:11ff:fe5d:5e8f/64 scope link
        valid_lft forever preferred_lft forever

```

- **Aislamiento entre VLANs:**

El servicio DHCP opera dentro de los límites de cada VLAN definida. Una solicitud DHCP de una VM en la VLAN de Aplicaciones solo será atendida por el ámbito DHCP configurado para la interfaz de OPNsense conectada a la VLAN de Aplicaciones, asegurando el aislamiento y la correcta configuración de red por segmento.

4.3.6. Servicio DNS (Gestionado por OPNsense y K3s CoreDNS)



- **Motivo de su implantación:**

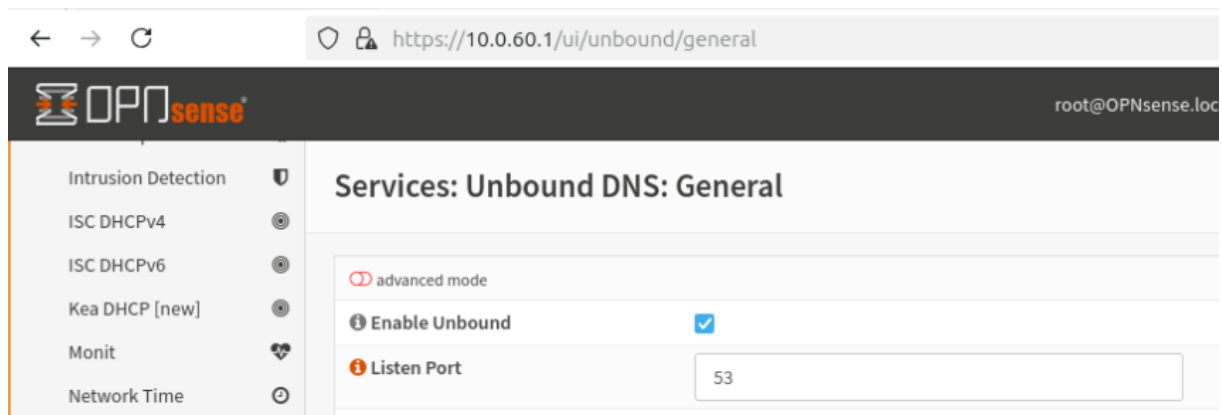
El servicio DNS es esencial para:

- Resolución de Nombres Internos: Permitir que las máquinas virtuales (VMs) y los servicios dentro de la infraestructura PaaS se encuentren y comuniquen entre sí utilizando nombres en lugar de direcciones IP, lo que simplifica la configuración y el mantenimiento. Por ejemplo, que la API pueda encontrar el servidor de base de datos por su nombre.
- Resolución de Nombres de Servicios en Kubernetes: Dentro del clúster K3s, permitir que los pods y servicios se descubran y comuniquen utilizando los nombres de servicio definidos en Kubernetes (por ejemplo, *keycloak-service*, *opadi-frontend-service*).
- Resolución de Nombres Externos (Internet): Permitir que todas las VMs y servicios accedan a recursos en Internet resolviendo nombres de dominio públicos.
- Facilitar el Acceso a la Aplicación: Permitir que los usuarios finales accedan a la aplicación OpenPaDi y a Keycloak utilizando nombres de dominio como *openpadi.local* y *auth.openpadi.local*.

- **Software empleado:**

- Para la Red General y VMs:

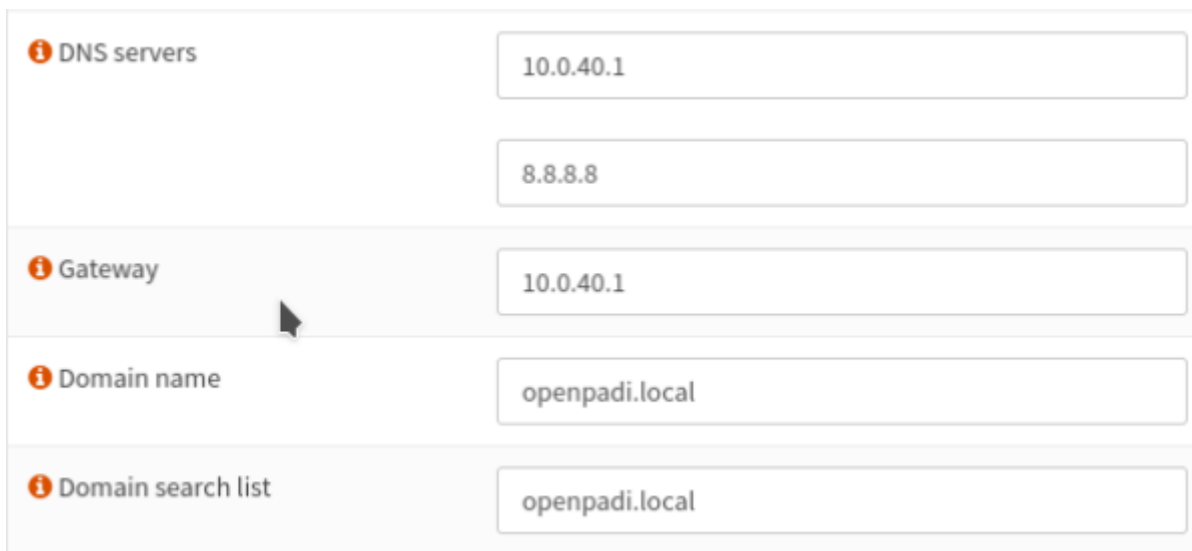
- El módulo **Unbound DNS** de resolución y reenvío DNS integrado en **OPNsense**. Se escogió por los mismos motivos que se escogió utilizar el módulo de DHCP que ofrece OPNsense: simplicidad, centralización y fácil integración con la plataforma de firewall/enrutamiento. Se prefirió utilizar el módulo Unbound DNS al módulo Dnsmasq DNS debido a que el primero ofrece más opciones, lo que lo haría más adecuado a un entorno real de producción.



- Plataforma de Gestión: **OPNsense**, ejecutándose en la máquina virtual *op-router-fw* (ID 101) y a través del cliente gráfico *op-client-1* (ID 102) de la VLAN de Administración.
- Para Servicios dentro del Clúster K3s:
 - CoreDNS, que es el servicio DNS de clúster por defecto instalado y gestionado por K3s.
 - Plataforma de Gestión: K3s, ejecutándose en los nodos *op-web-1* (master) y *op-api-1* (worker).
- **Configuración DNS en OPNsense (para la red de VMs y resolución externa):**

Cuando se configura el servicio DHCP en OPNsense para una VLAN (como se vio en el apartado 4.3.5), la dirección IP de la interfaz de OPNsense en esa VLAN (ej. 10.0.40.1 para la VLAN de ALMACENAMIENTO) se entrega típicamente como el servidor DNS a los clientes DHCP. No obstante, se ha configurado en el DHCP para

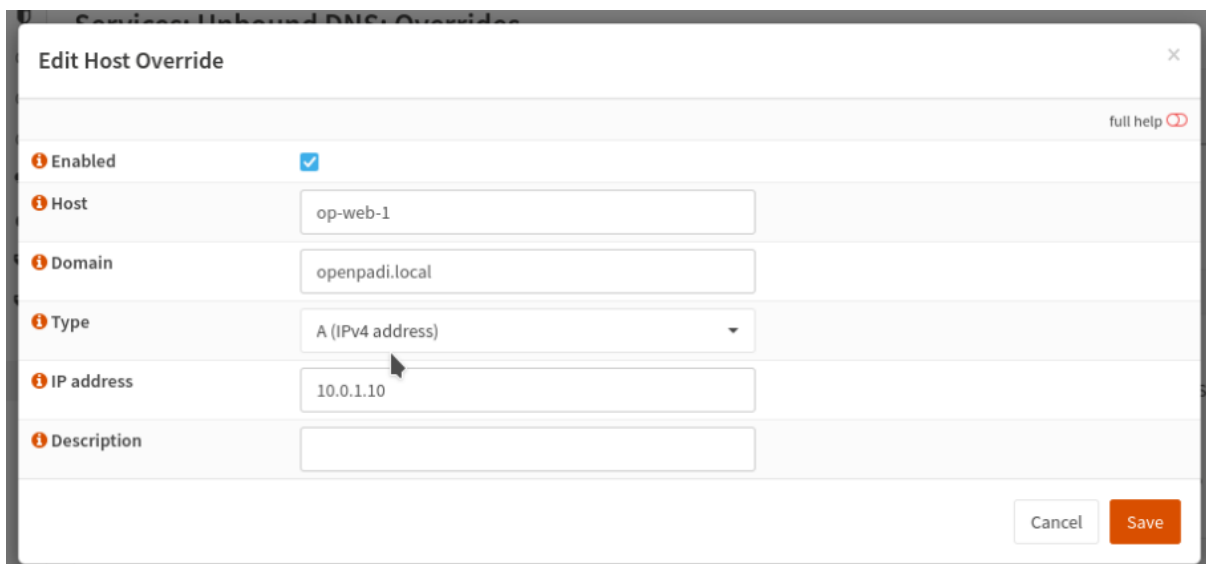
cada VLAN explícitamente dicha interfaz correspondiente. Por ejemplo, en la configuración DHCP de la VLAN ALMACENAMIENTO (10.0.40.0/24) se ha indicado explícitamente que su servidor DNS es la IP correspondiente a la interfaz de OPNsense para esta VLAN (10.0.40.1):



The screenshot shows the DHCP configuration interface for a specific VLAN. It contains four sections, each with an information icon (i) and a label:

- DNS servers:** Two input fields. The first contains "10.0.40.1" and the second contains "8.8.8.8".
- Gateway:** One input field containing "10.0.40.1".
- Domain name:** One input field containing "openpadi.local".
- Domain search list:** One input field containing "openpadi.local".

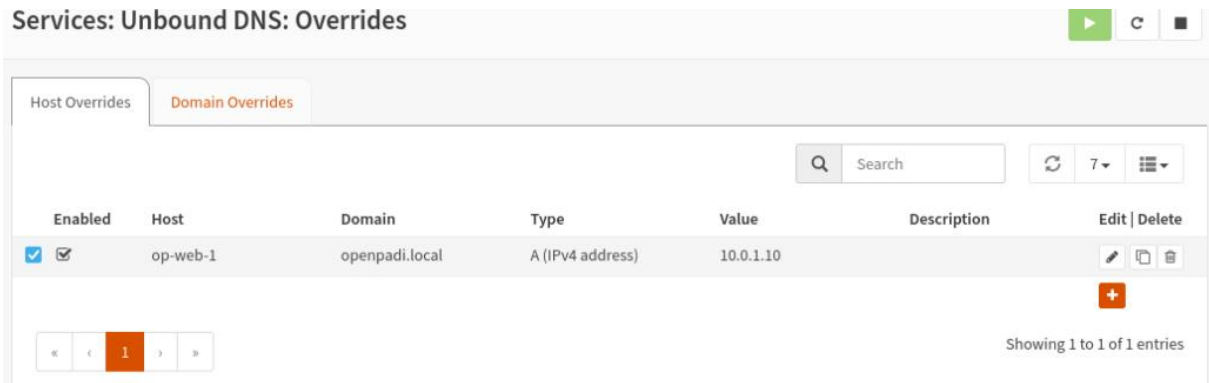
- **Registro de Nombres Locales:**



The screenshot shows the "Edit Host Override" dialog box. It has a title bar with a close button (X) and a "full help" link. The dialog contains several fields:

- Enabled:** A checkbox that is checked.
- Host:** An input field containing "op-web-1".
- Domain:** An input field containing "openpadi.local".
- Type:** A dropdown menu showing "A (IPv4 address)".
- IP address:** An input field containing "10.0.1.10".
- Description:** An empty input field.

At the bottom right, there are two buttons: "Cancel" and "Save".



```

administrator@op-client-1: ~
# operation for /etc/resolv.conf.
nameserver 10.0.60.1
options edns0 trust-ad
search openpadi.local
administrator@op-client-1:~$ nslookup op-web-1
Server:      10.0.60.1
Address:     10.0.60.1#53

Name:   op-web-1.openpadi.local
Address: 10.0.1.10
administrator@op-client-1:~$

```

Comprobación de que el DNS está funcionando mediante nslookup

- **Integración con DHCP:** Las concesiones DHCP pueden, opcionalmente, registrar dinámicamente los nombres de host de los clientes en el DNS de OPNsense, permitiendo la resolución de nombres para clientes con IPs dinámicas. En este escenario no se ha utilizado esta funcionalidad, pero es algo a tener en cuenta para un entorno de producción.
- **Configuración DNS en K3s (CoreDNS para servicios del clúster):**
 - **Descubrimiento de Servicios Interno:** K3s instala y configura automáticamente CoreDNS. Este servicio es

responsable de la resolución de nombres para los servicios y pods dentro del clúster K3s.

```
felipero... x adminis... x adminis... x adminis... x adminis... x
administrator@op-web-1:~$ sudo kubectl get pods -n kube-system -l k8s-app=kube-d
ns
NAME                                READY   STATUS    RESTARTS   AGE
coredns-697968c856-plvst           1/1     Running   11 (25m ago)  9d
administrator@op-web-1:~$
```

- **Nombres de Servicio:** Cuando se crea un Service en Kubernetes (por ejemplo, keycloak-service definido en kubernetes-manifests/keycloak-service.yaml), CoreDNS crea automáticamente un registro DNS para él.

```
administrator@op-web-1:~$ sudo kubectl get service keycloak-service -n default -o wide
NAME                TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE   SELECTOR
keycloak-service    ClusterIP   10.43.55.229 <none>        8080/TCP    7d    app=keycloak
administrator@op-web-1:~$
```

Por ejemplo, otros pods dentro del clúster pueden acceder al servicio Keycloak usando el nombre keycloak-service.default.svc.cluster.local o simplemente keycloak-service (si están en el mismo namespace).

- **Resolución Externa desde K3s:** Los pods en K3s necesitan resolver nombres externos. CoreDNS en K3s está configurado por defecto para reenviar las consultas de nombres no locales (que no son servicios del clúster) a los servidores DNS configurados en el archivo /etc/resolv.conf del nodo K3s donde se ejecuta el pod de CoreDNS. Estos nodos K3s (*op-web-1*, *op-api-1*) a su vez obtienen su configuración DNS de OPNsense (si están configurados como clientes DHCP de OPNsense).
- **No requiere configuración manual extensa:** La configuración básica de CoreDNS es manejada por K3s. Las personalizaciones avanzadas son posibles, pero

generalmente no son necesarias para funcionalidades estándar y en este proyecto no se han utilizado.

- **Flujo General de Consultas:**

- Una VM como *op-db-primary* consultaría primero a OPNsense (su DNS configurado). Si es un nombre local conocido por OPNsense, este responde. Si no, OPNsense reenvía a un DNS externo.
- Un pod dentro de K3s (por ejemplo, el backend API) al intentar conectar con *keycloak-service* consultaría a CoreDNS, que resolvería internamente. Si el pod API necesita acceder a *google.com*, CoreDNS reenviaría la consulta (a través de la cadena de DNS del nodo K3s) a OPNsense, y este a Internet.

4.3.7. Servicio de Orquestación de Contenedores (K3s)



- **Motivo de su implantación:**

La implantación de un servicio de orquestación de contenedores es fundamental para gestionar el ciclo de vida de las aplicaciones modernas de manera eficiente, escalable y resiliente. Para OpenPaDi, que se compone de múltiples microservicios (*frontend*, *backend* API, servicio de autenticación), un orquestador permite:

- **Despliegue Automatizado:** Simplifica y automatiza el proceso de despliegue de las diferentes partes de la aplicación OpenPaDi (*Frontend*, *Backend* API, Keycloak) como contenedores Docker.
- **Escalabilidad:** Facilita el escalado horizontal de los servicios (aumenta o disminuye el número de réplicas de un contenedor) según la demanda, optimizando el uso de recursos.

- Alta Disponibilidad y Auto-reparación: Asegura que las aplicaciones se mantengan en funcionamiento incluso si alguna instancia falla, reiniciando automáticamente los contenedores caídos.
- Descubrimiento de Servicios y Balanceo de Carga Interno: Provee mecanismos para que los diferentes servicios dentro del clúster puedan encontrarse y comunicarse entre sí de forma fiable (a través de los *Services* de Kubernetes y CoreDNS), y distribuye la carga entre las réplicas de un servicio.
- Gestión Centralizada: Ofrecer una plataforma unificada para gestionar todos los componentes de la aplicación.

Se eligió K3s, una distribución ligera y certificada de Kubernetes, debido a su mayor ligereza, facilidad de instalación y gestión, sin sacrificar las funcionalidades esenciales de orquestación, en comparación con un clúster Kubernetes completo, que debido a las limitaciones de hardware para este entorno de pruebas fue más difícil de implementar. Para un entorno de producción, se podría valorar esta otra opción.

- **Software empleado:**
 - Orquestador de Contenedores: K3s v1.32.4+k3s1.
 - Motor de Contenedores Subyacente: Containerd (integrado por defecto en K3s).
- **Plataforma de Despliegue:** Máquinas Virtuales (VMs) ejecutándose sobre Proxmox VE (en el diseño ideal pensado para producción) o VirtualBox (en entornos de pruebas en esta simulación), con Ubuntu Server 22.04.
 - Nodo Master: *op-web-1*.
 - Nodo(s) Worker: *op-api-1*.
 - Estos nodos, en producción, se replicarían para mayor eficiencia en rendimiento y alta disponibilidad.

```

administrator@op-web-1:~$ sudo kubectl get nodes
[sudo] password for administrator:
NAME           STATUS    ROLES          AGE   VERSION
op-api-1       Ready     <none>         9d    v1.32.4+k3s1
op-web-1       Ready     control-plane, 9d    v1.32.4+k3s1
administrator@op-web-1:~$

```

- **Servicios del clúster K3S:**

```

administrator@op-web-1:~$ sudo kubectl get service -n default -o wide
[sudo] password for administrator:
NAME           TYPE        CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE   SELECTOR
keycloak-service ClusterIP    10.43.55.229   <none>        8080/TCP   8d    app=keycloak
kubernetes      ClusterIP    10.43.0.1     <none>        443/TCP   10d   <none>
op-api          ClusterIP    10.43.29.150  <none>        8000/TCP   10d   app=opadi-api
opadi-frontend-service ClusterIP    10.43.221.243 <none>        80/TCP    10d   app=opadi-frontend
administrator@op-web-1:~$

```

15

El clúster K3S es el encargado de los siguientes servicios:

- keycloak-service:
- kubernetes:
- op-api
- opadi-frontend-service

Además, se encarga del servicio de Traefik, que viene integrado con el propio Kubernetes/K3S y del que se hablará más adelante

- **Archivos de configuración más relevantes:**

- <https://github.com/robleslf/OpenPaDi/blob/main/kubernetes-manifests/certificate.yaml> - Especifica la solicitud de un certificado TLS (para openpadi.local y auth.openpadi.local) a Cert-Manager, indicando qué Issuer debe usar y dónde guardar el Secret con el certificado generado.
- <https://github.com/robleslf/OpenPaDi/blob/main/kubernetes-manifests/selfsigned-issuer.yaml> - Define un emisor de certificados

¹⁵ La red 10.43.0.0/16 es la red que se definió por defecto al instalar KS3 para gestionar exclusivamente los servicios.

autofirmados para Cert-Manager, utilizado en el entorno de desarrollo/pruebas para generar los certificados TLS necesarios para el Ingress.

- <https://github.com/robleslf/OpenPaDi/blob/main/kubernetes-manifests/cluster-issuer.yaml> - Define un emisor de certificados a nivel de clúster para Cert-Manager, configurado para obtener certificados válidos de una Autoridad de Certificación como Let's Encrypt. En el entorno de pruebas no fue utilizado y se utilizó el anterior.
- <https://github.com/robleslf/OpenPaDi/blob/main/kubernetes-manifests/ingress.yaml> - Configura el Ingress Controller (Traefik) para enrutar el tráfico externo HTTP/HTTPS hacia los servicios correspondientes (Frontend, Backend API, Keycloak) basándose en el host y la ruta, y define la terminación TLS.
- <https://github.com/robleslf/OpenPaDi/blob/main/kubernetes-manifests/keycloak-deployment.yaml> - Describe el despliegue del servicio de autenticación Keycloak en K3s, especificando su imagen Docker, variables de entorno para su configuración (conexión a BBDD, admin, etc.) y volúmenes para temas.
- <https://github.com/robleslf/OpenPaDi/blob/main/kubernetes-manifests/keycloak-service.yaml> - Crea un servicio interno en K3s para Keycloak, permitiendo que otros componentes del clúster (como Traefik Ingress) lo descubran y se comuniquen con él por un nombre DNS interno.
- <https://github.com/robleslf/OpenPaDi/blob/main/kubernetes-manifests/deployment-api.yaml> - Define cómo se debe desplegar y gestionar la aplicación Backend API (FastAPI) en K3s, incluyendo la imagen Docker a usar, el número de réplicas y los puertos del contenedor.
- <https://github.com/robleslf/OpenPaDi/blob/main/kubernetes-manifests/service-api.yaml> - Expone el Backend API como un servicio interno dentro del clúster K3s, asignándole un nombre DNS y una IP

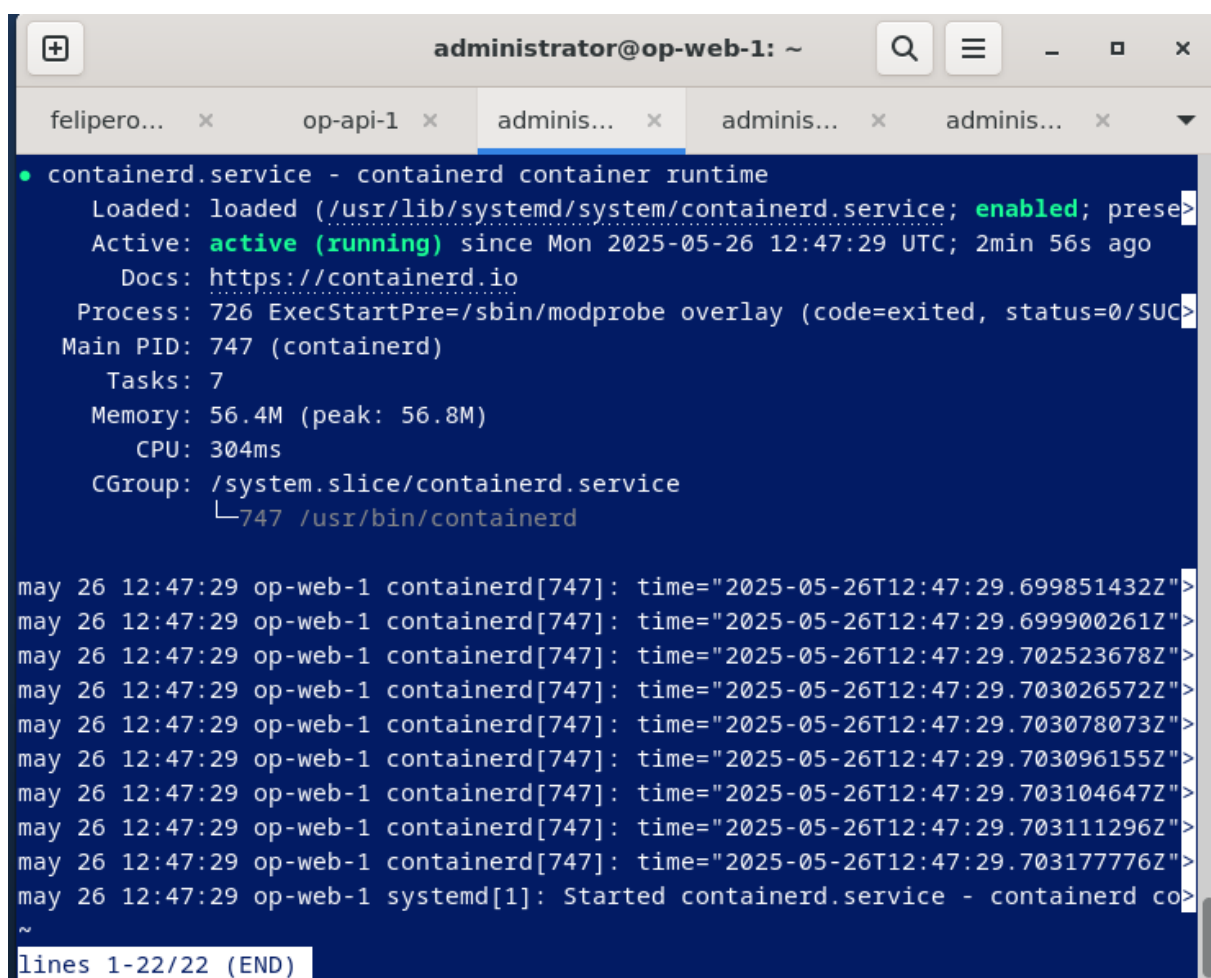
interna (ClusterIP) para que otros servicios (como el Frontend) puedan acceder a él.

- **Instalación y Configuración de Red de Nodos:**

- K3s se instaló en las VMs designadas: OP-Web-1 como nodo master, y OP-API-1 como nodo *worker*.
- La comunicación entre nodos y la red de pods utiliza la red interna, que en el diseño de Proxmox correspondería a la VLAN de Aplicaciones (10.0.20.0/24).

- **Componentes Integrados por K3s:** se simplifica la puesta en marcha de un clúster Kubernetes al incluir componentes esenciales preconfigurados:

- Containerd: Como runtime de contenedores.



```
administrator@op-web-1: ~
felipero... x op-api-1 x adminis... x adminis... x adminis... x
• containerd.service - containerd container runtime
  Loaded: loaded (/usr/lib/systemd/system/containerd.service; enabled; prese>
  Active: active (running) since Mon 2025-05-26 12:47:29 UTC; 2min 56s ago
  Docs: https://containerd.io
  Process: 726 ExecStartPre=/sbin/modprobe overlay (code=exited, status=0/SUC>
  Main PID: 747 (containerd)
  Tasks: 7
  Memory: 56.4M (peak: 56.8M)
  CPU: 304ms
  CGroup: /system.slice/containerd.service
          └─747 /usr/bin/containerd

may 26 12:47:29 op-web-1 containerd[747]: time="2025-05-26T12:47:29.699851432Z">
may 26 12:47:29 op-web-1 containerd[747]: time="2025-05-26T12:47:29.699900261Z">
may 26 12:47:29 op-web-1 containerd[747]: time="2025-05-26T12:47:29.702523678Z">
may 26 12:47:29 op-web-1 containerd[747]: time="2025-05-26T12:47:29.703026572Z">
may 26 12:47:29 op-web-1 containerd[747]: time="2025-05-26T12:47:29.703078073Z">
may 26 12:47:29 op-web-1 containerd[747]: time="2025-05-26T12:47:29.703096155Z">
may 26 12:47:29 op-web-1 containerd[747]: time="2025-05-26T12:47:29.703104647Z">
may 26 12:47:29 op-web-1 containerd[747]: time="2025-05-26T12:47:29.703111296Z">
may 26 12:47:29 op-web-1 containerd[747]: time="2025-05-26T12:47:29.703177762Z">
may 26 12:47:29 op-web-1 systemd[1]: Started containerd.service - containerd co>
~
lines 1-22/22 (END)
```

- Flannel: Como CNI (Container Network Interface) por defecto, gestionando la red superpuesta para los pods.
- CoreDNS: Para el descubrimiento de servicios DNS dentro del clúster (detallado en el apartado 4.3.6).

```

administrator@op-web-1:~$ sudo kubectl get pods -n kube-system -l k8s-app=kube-dns -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE     NOMINATED NODE   READINESS GATES
coredns-697968c856-plvst            1/1     Running   12 (4m3s ago)  9d    10.42.0.94    op-web-1   <none>           <none>
administrator@op-web-1:~$

```

- Traefik Proxy Ingress Controller: Preinstalado para gestionar el tráfico entrante HTTP/HTTPS (detallado en el apartado 4.3.8).

```

administrator@op-web-1:~$ sudo kubectl get pods -n kube-system -l app.kubernetes.io/name=traefik -o wide
NAME                                READY   STATUS    RESTARTS   AGE   IP            NODE     NOMINATED NODE   READINESS GATES
traefik-c98fd6fb-7jffn              1/1     Running   12 (4m50s ago)  9d    10.42.0.92    op-web-1   <none>           <none>
administrator@op-web-1:~$ sudo kubectl get svc traefik -n kube-system -o wide
NAME      TYPE      CLUSTER-IP   EXTERNAL-IP   PORT(S)
traefik   LoadBalancer  10.43.234.59  192.168.1.10,192.168.1.12  80:30957/TCP,443:30971/TCP
administrator@op-web-1:~$

```

En esta simulación del proyecto, se ajustó la configuración del servicio de Traefik para usar la IP 192.168.1.10 (OP-Web-1) para la exposición externa, asegurando que Traefik escuche en la IP de la red interna correcta.

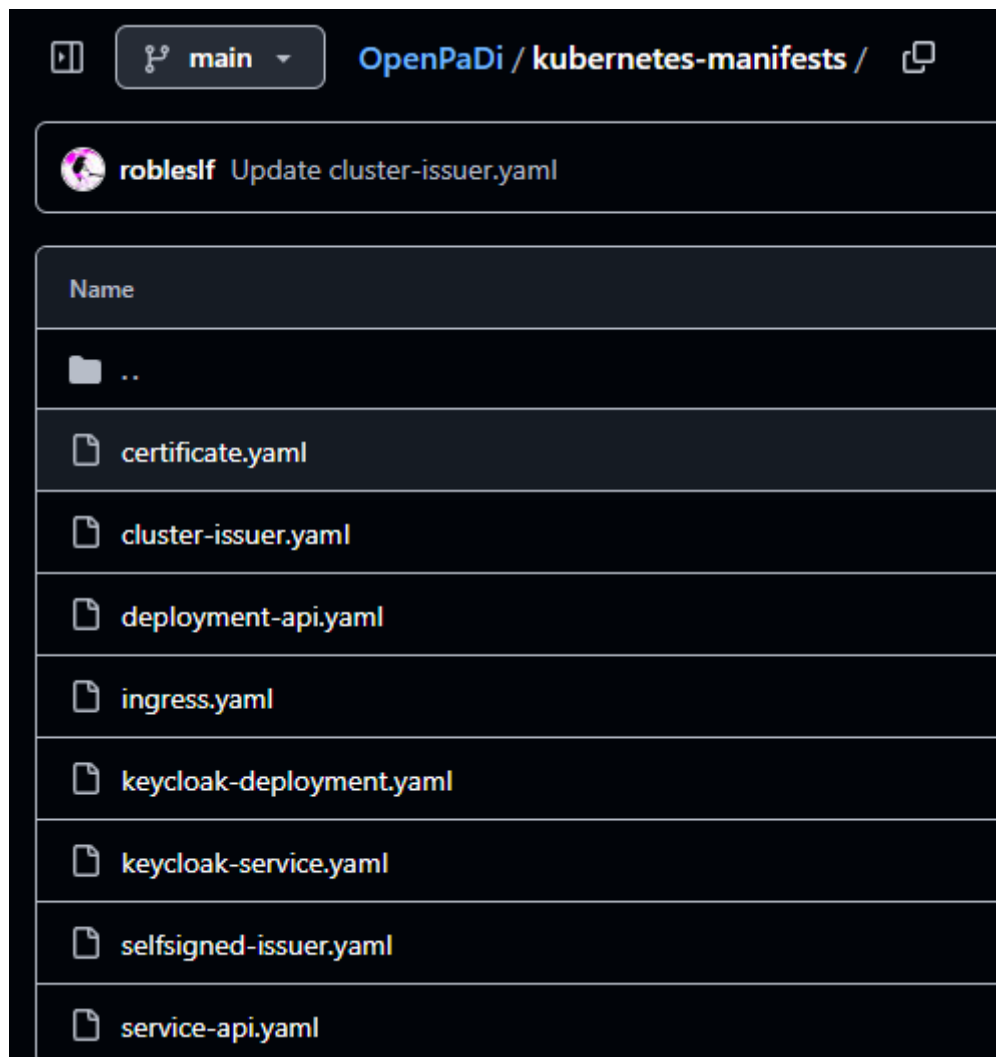
NAME	TYPE	CLUSTER-IP
traefik	LoadBalancer	10.43.234.59

Para esto hay que tener en cuenta que no existe un archivo de configuración como tal para indicar la IP expuesta, ya que no creamos ningún archivo de manifiesto kubernetes para Traefik, se debe usar el comando `sudo kubectl edit svc traefik -n kube-system`, para modificar el manifiesto del servicio existente, editando la siguiente línea:

```

status:
  loadBalancer:
    ingress:
      - ip: 192.168.1.10
        ipMode: VIP

```

La razón de por qué no se decidió crear un archivo de configuración para el manifiesto de Traefik a diferencia de lo que se hizo con otros servicios relacionados con el clúster Kubernetes, es debido a el despliegue base de Traefik es gestionado automáticamente por K3s, y su comportamiento se define mediante los manifiestos *Ingress* que sí se han creado para las aplicaciones.

- **Gestión de Aplicaciones mediante Manifiestos YAML:**
 - Todas las aplicaciones (*Frontend*, *Backend* API, Keycloak) y sus configuraciones asociadas (*Services*, *Ingresses*, *Certificates*) se definen mediante archivos de manifiesto YAML estándar de Kubernetes, correspondientes a la imagen del punto anterior.

- Estos manifiestos se aplican al clúster K3s desde el nodo master (OP-Web-1) utilizando el comando `sudo kubectl apply -f <archivo.yaml>`. K3s se encarga de crear o actualizar los recursos correspondientes (*Deployments*, *Pods*, *Services*, etc.) para alcanzar el estado deseado.

- **Persistencia de Datos para Servicios en K3s:**

K3s gestiona los contenedores que, por naturaleza, son efímeros. Para los servicios desplegados en K3s que requieren persistencia de datos, se han adoptado las siguientes estrategias:

- Para Keycloak: La persistencia de datos críticos de Keycloak (configuración de realms, usuarios, clientes, etc.) se ha desacoplado completamente del clúster K3s. Keycloak está configurado para utilizar la VM PostgreSQL externa *op-db-primary* (192.168.1.13 en esta simulación) como su base de datos. Esto se define mediante variables de entorno en el manifiesto *keycloak-deployment.yaml* que apuntan a la instancia de PostgreSQL.

```
- name: KC_DB
  value: "postgres"
- name: KC_DB_URL_HOST
  value: "192.168.1.13"
- name: KC_DB_URL_DATABASE
  value: "keycloak_db"
- name: KC_DB_URL_PORT
  value: "5432"
- name: KC_DB_USERNAME
  value: "keycloak_user"
- name: KC_DB_PASSWORD
  value: "abc123.."
```

16

- Para la Aplicación OpenPaDi (*Backend API*): De manera similar al servicio de Keycloak, el Backend API (*opadi-api*) también utiliza la VM

¹⁶ <https://github.com/robleslf/OpenPaDi/blob/main/kubernetes-manifests/keycloak-deployment.yaml>

PostgreSQL externa *op-db-primary* para almacenar los metadatos de los documentos y las transcripciones.

```
app = FastAPI()  
  
DB_HOST = "192.168.1.13"  
DB_NAME = "opadi_db"  
DB_USER = "opadi_user"  
DB_PASS = "abc123.."
```

17

Además, los archivos físicos de los documentos (imágenes, PDFs) son gestionados por la VM MinIO externa *op-storage-1*. La API FastAPI dentro de K3s actúa como intermediaria, pero no almacena datos persistentes directamente en volúmenes de K3s.

```
MINIO_ENDPOINT = "192.168.1.14:9000"  
MINIO_ACCESS_KEY = "openpadiadmin"  
MINIO_SECRET_KEY = "abc123.."   
MINIO_BUCKET_NAME = "openpadi-documentos"  
MINIO_USE_SSL = False
```

18

- Para la Aplicación OpenPaDi: El *Frontend* Svelte consiste en archivos estáticos (HTML, CSS, JavaScript) generados durante el proceso de *build* (a partir del comando *npm run build*). Estos archivos se empaquetan directamente dentro de la imagen Docker del *frontend* y son servidos por Nginx (que también corre en el mismo contenedor).

¹⁷ <https://github.com/robleslf/OpenPaDi/blob/main/backend-api/main.py>

¹⁸ [Ibid.](#)

```
FROM node:20 AS builder

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

RUN npm run build

FROM nginx:stable-alpine

COPY --from=builder /app/build /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

19

Por lo tanto, el *frontend* en sí mismo no requiere un volumen persistente en K3s para su funcionamiento, ya que sus datos son los propios archivos de la aplicación servidos estáticamente.

- Se escogió esta opción de desacoplar la persistencia de datos para los servicios *stateful* (Keycloak y la persistencia de la aplicación OpenPaDi) hacia VMs externas dedicadas (PostgreSQL y MinIO) con el fin de simplificar la gestión dentro de K3s para este proyecto. Si el proyecto escalase y se necesitasen arquitecturas más complejas o con otros requisitos, se podrían utilizar soluciones de almacenamiento nativas de Kubernetes con proveedores de almacenamiento compatibles.

¹⁹ <https://github.com/robleslf/OpenPaDi/blob/main/frontend-svelte/Dockerfile>

- La herramienta de línea de comandos *kubectl* es la interfaz principal para interactuar con el API server de K3s y administrar el clúster (desplegar aplicaciones, verificar estado, ver logs, etc.).

4.3.8. Servicio de Ingress y Gestión de Certificados (Traefik Proxy y Cert-Manager)



- **Motivo de su implantación:**

Para exponer de forma segura y controlada los servicios de la aplicación OpenPaDi (Frontend, API) y el servicio de autenticación (Keycloak) a los usuarios finales a través de Internet (o la red local), se requiere un mecanismo de entrada de tráfico (*Ingress*) y una gestión de los certificados TLS/SSL.

- **Traefik Proxy (Ingress Controller):** Se utiliza para actuar como un proxy inverso. Recibe todas las peticiones HTTP/HTTPS entrantes y las enruta al servicio interno correcto en K3s basándose en reglas (como el nombre de host y la ruta de la URL). También se encarga de la terminación TLS, es decir, gestiona las conexiones HTTPS cifradas.
- **Cert-Manager:** Se implanta para automatizar la gestión de los certificados TLS. Es capaz de obtener certificados de diversas fuentes (como Let's Encrypt para producción o emisores autofirmados, que son los que se han utilizado para estas pruebas) y renovarlos automáticamente antes de que expiren. Esto asegura que las conexiones a OpenPaDi sean siempre seguras (HTTPS) sin intervención manual constante.

La combinación de Traefik y Cert-Manager proporciona un punto de entrada robusto, seguro y automatizado para la plataforma.

- **Software empleado:**

- *Ingress Controller*: Traefik Proxy (la versión integrada y gestionada por K3s).
 - Gestor de Certificados: Cert-Manager (instalado en el clúster K3s mediante sus manifiestos oficiales).
 - Emisor de Certificados (para este entorno de pruebas): Un emisor autofirmado configurado a través de Cert-Manager. Lo ideal para un entorno de producción sería *Let's Encrypt*.
- **Plataforma de Despliegue:** Ambos se ejecutan como servicios dentro del clúster K3s.
- **Archivos de configuración más relevantes:**
 - <https://github.com/robleslf/OpenPaDi/blob/main/kubernetes-manifests/ingress.yaml>: Archivo principal que define las reglas de enrutamiento para Traefik. Especifica los hosts (*openpadi.local*, *auth.openpadi.local*), sus rutas (*/*, */api*) y los servicios de *backend* a los que se dirige el tráfico, así como la configuración TLS (qué *Secret* contiene el certificado).
 - <https://github.com/robleslf/OpenPaDi/blob/main/kubernetes-manifests/certificate.yaml>: Define la solicitud del certificado TLS a Cert-Manager. Especifica los nombres de dominio que debe cubrir el certificado, el *issuerRef* (el emisor que lo generará, en este caso autofirmado), y el *secretName* donde Cert-Manager almacenará el certificado y la clave privada una vez emitidos. Traefik usará este *Secret*.
 - <https://github.com/robleslf/OpenPaDi/blob/main/kubernetes-manifests/selfsigned-issuer.yaml>: Define el emisor autofirmado que Cert-Manager utiliza en este entorno.
 - <https://github.com/robleslf/OpenPaDi/blob/main/kubernetes-manifests/cluster-issuer.yaml>: en este caso no se ha utilizado, pero en un entorno de producción se sustituiría el fichero anterior por este para utilizar Let's Encrypt.

- **Verificación del Secret TLS:**

```
administrator@op-web-1:~$ sudo kubectl get secret openpadi-tls -n default -o yaml
apiVersion: v1
data:
  ca.crt: LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tCK1JSUN5VENDQWJHZ0F3SUJBZ01SQ5Ubjgv
3FHU01iMwpEUUVCCQVFVQUE0SUJEd0F3Z2dFS0FvSUJBURteC9UcW02QmZCMWFrZEFCamtoTzVtYlZyeEx
K1ROUjFmTmZnNWl5WVBWbzZqODE5SDhQRnpXNTlJOHJ0RHQ4c3lyK3lzZApTa2E1VjZTOU50bkRHWS91bD
uWE76NmxwNno1Y2E5eHEKRBHJ2a0E4N2svTEpTOUJlBampwZ1byOUhDNiNEMHbQUiRTeijlNjVGV085O3cwe

metadata:
  annotations:
    cert-manager.io/alt-names: openpadi.local
    cert-manager.io/certificate-name: openpadi-tls
    cert-manager.io/common-name: ""
    cert-manager.io/ip-sans: ""
    cert-manager.io/issuer-group: ""
    cert-manager.io/issuer-kind: Issuer
    cert-manager.io/issuer-name: selfsigned-issuer
    cert-manager.io/uri-sans: ""
  creationTimestamp: "2025-05-16T15:06:58Z"
  labels:
    controller.cert-manager.io/fao: "true"
  name: openpadi-tls
  namespace: default
  resourceVersion: "1534"
  uid: cbc39e92-da9f-45a0-abb8-39f8546ae2e4
type: kubernetes.io/tls
```

Esta salida confirma que el *Secret* llamado *openpadi-tls* existe y es de tipo *kubernetes.io/tls*. Esto confirma que *Cert-Manager* ha creado el secreto que *Traefik* utilizará.

- **Instalación de Cert-Manager:**

Su instalación en K3s implicó dos pasos: primero, se definieron los nuevos tipos de recursos que *Cert-Manager* utiliza (como *Certificate* para solicitar un certificado, e *Issuer* para indicar quién lo emite), aplicando directamente el manifiesto de CRDs desde la URL oficial de *Cert-Manager*²⁰. Luego, se desplegó el software principal de

²⁰ `sudo kubectl apply -f https://github.com/cert-manager/cert-manager/releases/latest/download/cert-manager.crds.yaml`

Cert-Manager, aplicando también su manifiesto de instalación desde la fuente oficial²¹, el cual actúa sobre estos recursos para obtener y administrar los certificados. Una vez instalado Cert-Manager, se utilizaron archivos como *certificate.yaml* y *selfsigned-issuer.yaml*, mencionados anteriormente (ver subapartado *Archivos de configuración más relevantes* de este servicio), para definir las solicitudes de certificados y los emisores específicos para OpenPaDi.

```
administrator@op-web-1:~$ sudo kubectl -n cert-manager get pods
[sudo] password for administrator:
NAME                                READY   STATUS    RESTARTS   AGE
cert-manager-6468fc8f56-xkl49       1/1     Running   14 (113m ago)  11d
cert-manager-cainjector-7fd85dcc7-5f8gw 1/1     Running   19 (113m ago)  11d
cert-manager-webhook-57df45f686-8ftrj 1/1     Running   13 (113m ago)  11d
administrator@op-web-1:~$
```

Pods de Cert-Manager en ejecución

- **cert-manager**: el controlador principal, responsable de gestionar el ciclo de vida de los certificados.
- **cert-manager-cainjector**: inyector de CA.
- **cert-manager-webhook**: valida las configuraciones de Cert-Manager.

- **Creación del Emisor de Certificados (Issuer):**

Para el entorno de pruebas, se creó un *Issuer* de tipo *selfSigned* llamado *selfsigned-issuer*.

```
apiVersion: cert-manager.io/v1
kind: Issuer
metadata:
  name: selfsigned-issuer
spec:
  selfSigned: {}
```

²¹ `sudo kubectl apply -f https://github.com/cert-manager/cert-manager/releases/latest/download/cert-manager.yaml`

²² `https://github.com/robleslf/OpenPaDi/blob/main/kubernetes-manifests/selfsigned-issuer.yaml`

Este *Issuer* le indica a Cert-Manager que genere certificados autofirmados.

Se creó también, como ya se ha mencionado, un *ClusterIssuer* llamado *letsencrypt-prod* configurado para usar el servidor de Let's Encrypt destinado a obtener certificados públicamente válidos en un entorno de producción²³.

- **Solicitud de Certificado:**

Se creó un recurso *Certificate* llamado *openpadi-tls* que solicita a Cert-Manager la emisión de un certificado para los dominios *openpadi.local* y *auth.openpadi.local* (utilizado por el servicio de Keycloak), utilizando el mencionado *selfsigned-issuer*.

```
apiVersion: cert-manager.io/v1
kind: Certificate
metadata:
  name: openpadi-tls
spec:
  secretName: openpadi-tls
  dnsNames:
    - openpadi.local
  issuerRef:
    name: selfsigned-issuer
    kind: Issuer
```

24

Cert-Manager automáticamente procesa esta solicitud y almacena el certificado y la clave privada resultantes en un *Secret* de Kubernetes también llamado *openpadi-tls*.

```
administrator@op-web-1:~$ sudo kubectl get secret -n default
NAME                TYPE                DATA  AGE
openpadi-tls        kubernetes.io/tls   3      11d
administrator@op-web-1:~$ S
```

²³ <https://github.com/robleslf/OpenPaDi/blob/main/kubernetes-manifests/cluster-issuer.yaml>

²⁴ <https://github.com/robleslf/OpenPaDi/blob/main/kubernetes-manifests/certificate.yaml>

```

administrator@op-web-1:~$ sudo kubectl describe secret openpadi-tls -n default
Name:         openpadi-tls
Namespace:    default
Labels:       controller.cert-manager.io/fao=true
Annotations:  cert-manager.io/alt-names: openpadi.local
              cert-manager.io/certificate-name: openpadi-tls
              cert-manager.io/common-name:
              cert-manager.io/ip-sans:
              cert-manager.io/issuer-group:
              cert-manager.io/issuer-kind: Issuer
              cert-manager.io/issuer-name: selfsigned-issuer
              cert-manager.io/uri-sans:

Type:  kubernetes.io/tls

Data
====
ca.crt:  1025 bytes
tls.crt: 1025 bytes
tls.key: 1675 bytes
administrator@op-web-1:~$

```

- **tls.crt**: Contiene el certificado público codificado en base64.
- **tls.key**: Contiene la clave privada codificada en base64.
- **ca.crt**: Contiene el certificado de la CA que firmó *tls.crt* (en este caso, la CA del *selfsigned-issuer*).

- **Configuración del Ingress (ingress.yaml):**

El recurso *Ingress* se configuró con *ingressClassName*: Traefik para asegurar que sea gestionado por el *Traefik Ingress Controller* integrado en K3s.

```

spec:
  ingressClassName: traefik
  tls:
  - hosts:
    - openpadi.local
    - auth.openpadi.local
    secretName: openpadi-tls

```

25

²⁵ <https://github.com/robleslf/OpenPaDi/blob/main/kubernetes-manifests/ingress.yaml>

En la sección *tls*, se especifica que para los hosts *openpadi.local* y *auth.openpadi.local*, se debe usar el certificado almacenado en el *Secret* mencionado anteriormente.

Las *rules* definen el enrutamiento:

```
rules:
- host: openpadi.local
  http:
    paths:
    - path: /api
      pathType: Prefix
      backend:
        service:
          name: op-api
          port:
            number: 8000
    - path: /
      pathType: Prefix
      backend:
        service:
          name: opadi-frontend-service
          port:
            number: 80
- host: auth.openpadi.local
  http:
    paths:
    - path: /
      pathType: Prefix
      backend:
        service:
          name: keycloak-service
          port:
            number: 8080
```

- Peticiones a *openpadi.local/api/...* se dirigen al servicio *op-api* en el puerto 8000.

```

administrator@op-web-1:~$ sudo kubectl get service op-api
NAME         TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
op-api       ClusterIP     10.43.29.150   <none>         8000/TCP    11d
administrator@op-web-1:~$

```

- Peticiones a *openpadi.local/...* (cualquier otra cosa) se dirigen al servicio *opadi-frontend-service* en el puerto 80.

```

administrator@op-web-1:~$ sudo kubectl get service opadi-frontend-service
NAME                     TYPE          CLUSTER-IP    EXTERNAL-IP    PORT(S)    AGE
opadi-frontend-service   ClusterIP     10.43.221.243 <none>         80/TCP     11d
administrator@op-web-1:~$

```

- Peticiones a *auth.openpadi.local/...* se dirigen al servicio *keycloak-service* en el puerto 8080.

- **Exposición y Prueba de Acceso TLS:**

Traefik, al estar expuesto en la IP 192.168.1.10 (mediante la configuración de *externalIPs* en su Service), escucha en los puertos 80 y 443.

```

administrator@op-web-1:~$ sudo kubectl get ingress opadi-frontend-ingress -o wide
[sudo] password for administrator:
NAME                     CLASS    HOSTS                                     ADDRESS                                     PORTS    AGE
opadi-frontend-ingress   traefik  openpadi.local,auth.openpadi.local       192.168.1.10,192.168.1.12               80, 443  11d
administrator@op-web-1:~$

```

Al acceder a <https://openpadi.local> desde un navegador, *Traefik* recibe la petición, utiliza el certificado autofirmado de *openpadi-tls* y enruta la petición al servicio del *frontend*²⁶. Peticiones a <https://openpadi.local/api/documentos> son enrutadas al *backend* API.

4.3.9. Servidor Web Frontend (Nginx + Svelte en K3s)



²⁶ Conviene recordar que la aplicación *frontend* se encarga de detectar si el usuario está logueado o no al acceder a <https://openpadi.local>, y decide reenviar o no la petición a <https://auth.openpadi.local> en función de esto.

- **Motivo de su implantación:**

El Servidor Web es el componente responsable de entregar la interfaz de usuario de OpenPaDi a los navegadores de los usuarios finales. Es la cara visible de la plataforma, permitiendo a los investigadores, estudiantes y demás interesados interactuar con los documentos, transcripciones y funcionalidades de colaboración. Su implantación es esencial para:

- Proveer una experiencia de usuario eficaz e intuitiva.
- Gestionar la interacción del usuario, incluyendo la visualización de datos, la entrada de formularios y la navegación.
- Comunicarse con el *Backend API* (*op-api*) para obtener y enviar datos (listar documentos, subir nuevos documentos, etc.).
- Integrarse con el servicio de autenticación (*Keycloak*) para gestionar el inicio/cierre de sesión y el acceso a funcionalidades protegidas.

Se eligió *Svelte* como *framework* JavaScript para construir la interfaz de usuario debido a su comodidad y sencillez para desplegar aplicaciones rápidas. No obstante, el apartado de desarrollo de la aplicación es algo que queda fuera de este proyecto y esta solución podría ser diferente en el entorno de producción final.

Como servidor web se eligió Nginx, ya que es eficiente y robusto para servir los archivos estáticos generados por la compilación de Svelte.

Ambos componentes se ejecutan dentro de Docker gestionado por K3s.

- **Software empleado:**

- **Framework Frontend:** Svelte 5.0.0.
- **Herramientas de Desarrollo/Construcción Frontend:** Vite (usado por SvelteKit), Node.js y npm (para la gestión de dependencias y scripts de construcción).
- **Servidor Web para archivos estáticos:** Nginx (versión *stable-alpine* a partir de la imagen Docker oficial).
- **Contenerización:** Docker.
- **Orquestación:** K3s.

- **Máquina donde se desarrolla y construye la imagen Docker:** *op-web-1*

- **Archivos de configuración más relevantes:**

- <https://github.com/robleslf/OpenPaDi/blob/main/frontend-svelte/Dockerfile>: Define el proceso de construcción de la imagen Docker. Primero usa una imagen de Node.js para instalar dependencias y construir la aplicación Svelte (*npm run build*), y luego copia los artefactos estáticos generados a una imagen de Nginx que los servirá.
- <https://github.com/robleslf/OpenPaDi/blob/main/frontend-svelte/deployment.yaml>: Manifiesto de Kubernetes que describe cómo K3s debe desplegar el contenedor del *frontend*, especificando la imagen Docker a usar (*robleslf/opadi-frontend:latest*²⁷), el número de réplicas (que en el entorno de producción deberían aumentarse) y el puerto del contenedor (80).
- <https://github.com/robleslf/OpenPaDi/blob/main/frontend-svelte/service.yaml>: Manifiesto de Kubernetes que crea un servicio interno (*opadi-frontend-service*). Este servicio expone los *Pods* del frontend en el puerto 80 dentro del clúster K3s, permitiendo que el Ingress Controller de Traefik les envíe tráfico.
- <https://github.com/robleslf/OpenPaDi/blob/main/frontend-svelte/svelte.config.js>: de aquí lo que más nos interesa es *el fallback*: *'index.html'*, que es el que se encarga de que, sin importar la dirección específica que el usuario escriba en el navegador para OpenPaDi, Nginx siempre envíe la página principal (*index.html*).
- <https://github.com/robleslf/OpenPaDi/blob/main/frontend-svelte/src/routes/%2Bpage.svelte>: es el componente principal de la interfaz de usuario, donde se implementa la lógica para mostrar documentos, interactuar con formularios, realizar peticiones CRUD a la API y gestionar la autenticación con Keycloak.
- <https://github.com/robleslf/OpenPaDi/blob/main/frontend-svelte/src/lib/keycloak.js>: contiene la lógica para inicializar y gestionar la

²⁷ <https://hub.docker.com/repository/docker/robleslf/opadi-frontend/tags/latest/sha256-a15f983cc57eb4eeb085643b698918f83719c037256781c736ff1bae82e00ce1>

instancia de Keycloak en el *frontend*, incluyendo el inicio de sesión, cierre de sesión y actualización de tokens.

- <https://github.com/robleslf/OpenPaDi/blob/main/kubernetes-manifests/ingress.yaml>: de este archivo ya se habló con anterioridad, pero aquí nos interesa la regla para *host: openpadi.local* y *path: /*, que define cómo Traefik enruta el tráfico externo a este servicio *frontend*.

- **Construcción de la Aplicación Svelte:**

La aplicación Svelte se desarrolla en la VM *op-web-1*.

Se utilizan los comandos *npm install* para descargar las dependencias (definidas en *package.json*²⁸) y *npm run build* para compilar la aplicación Svelte y generar los archivos estáticos (HTML, CSS, JavaScript) en la carpeta *build*. Estos comandos están definidos en el *Dockerfile*²⁹.

- **Dockerización con Nginx:**

El mencionado *Dockerfile* utiliza un enfoque de construcción de múltiples etapas. La primera etapa es la explicada en el párrafo anterior: con una imagen de Node.js se encarga de construir la aplicación Svelte.

```
FROM node:20 AS builder

WORKDIR /app

COPY package*.json ./

RUN npm install

COPY . .

RUN npm run build
```

²⁸ <https://github.com/robleslf/OpenPaDi/blob/main/frontend-svelte/package.json>

²⁹ <https://github.com/robleslf/OpenPaDi/blob/main/frontend-svelte/Dockerfile>

Una segunda etapa utiliza la imagen ligera *nginx:stable-alpine* y copia los archivos estáticos construidos en la primera etapa al directorio raíz de Nginx (*/usr/share/nginx/html*).

```
FROM nginx:stable-alpine

COPY --from=builder /app/build /usr/share/nginx/html

EXPOSE 80

CMD ["nginx", "-g", "daemon off;"]
```

Nginx se configura para servir estos archivos estáticos en el puerto 80 del contenedor.

- **Despliegue en K3s:**

La imagen Docker creada se etiquetó y se subió a Docker Hub para poder desplegar el *frontend* de OpenPadi en otra máquina con conexión a Internet de forma rápida (*robleslf/opadi-frontend:latest*³⁰).

El *deployment.yaml*³¹ le indica a K3s que descargue esta imagen y ejecute un número especificado de réplicas (*Pods*), que en la fase de producción debería aumentarse consecuentemente.

```
spec:
  containers:
  - name: opadi-frontend
    image: robleslf/opadi-frontend:latest
```

El *service.yaml*³² crea un punto de acceso interno (*opadi-frontend-service*) para estos *Pods*.

³⁰ <https://hub.docker.com/repository/docker/robleslf/opadi-frontend/tags/latest/sha256-a15f983cc57eb4eeb085643b698918f83719c037256781c736ff1bae82e00ce1>

³¹ <https://github.com/robleslf/OpenPaDi/blob/main/frontend-svelte/deployment.yaml>

³² <https://github.com/robleslf/OpenPaDi/blob/main/frontend-svelte/service.yaml>


```
apiVersion: v1
kind: Service
metadata:
  name: opadi-frontend-service
```

- **Exposición Externa a través de Traefik:**

Cuando un usuario visita <https://openpadi.local>, el *Ingress Controller* (Traefik), dirige esta petición al servicio interno *opadi-frontend-service*, que es el encargado de mostrar la página web. La propia aplicación *frontend* se encarga de comprobar si el usuario está logueado o no en el fichero *keycloak.js*³³.

```
export async function initKeycloak() {
  const keycloak = getKeycloak();
  try {
    const authenticated = await keycloak.init({
      onLoad: 'check-sso',
      silentCheckSsoRedirectUri: window.location.origin + '/silent-check-sso.html'
    });
    if (authenticated) {
      setInterval(() => {
        keycloak.updateToken(30).then(refreshed => {
          if (refreshed) {
            // Token refrescado
          } else {
            // Token no refrescado
          }
        }).catch(() => {
          keycloak.logout();
        });
      }, 60000);
    }
    return authenticated;
  } catch (error) {
    console.error("Failed to initialize Keycloak adapter:", error);
    return false;
  }
}
```

En caso de que el usuario no lo esté, le dará la opción de hacerlo:

³³ <https://github.com/robleslf/OpenPaDi/blob/main/frontend-svelte/src/lib/keycloak.js>

https://openpadi.local



OpenPaDi - Repositorio de Transcripciones

Por favor, inicia sesión para continuar.

Iniciar Sesión con Keycloak

Y será llevado a la interfaz de *login* de *Keycloak* en el subdominio <https://auth.openpadi.local>:

https://auth.openpadi.local/realms/openpadi/protocol/openid-connect/auth?client_id=openpadi-frontend&redirect_uri=https%3A%2F%2Fopenpadi.local%2F&state

OPENPADI

Sign in to your account

Username or email
testuser

Password
●●●●●●

Sign In

Una vez comprobado que el usuario existe, mostrará la interfaz de usuario:



OpenPaDi - Repositorio de Transcripciones

Bienvenido, testuser!

Cerrar Sesión

Crear Nuevo Documento

Título:

Fecha (YYYY-MM-DD):

Contenido:

Archivo (PDF, Imagen, etc.):

- **Integración con el Backend API:**

La comunicación entre la interfaz de usuario (+*page.svelte*³⁴) y el servidor que maneja los datos (*Backend API*) se realiza mediante peticiones HTTP. Para facilitar estas peticiones, como por ejemplo solicitar la lista de documentos desde */api/documentos*, utiliza la librería *axios*.

```
import axios from 'axios';
```

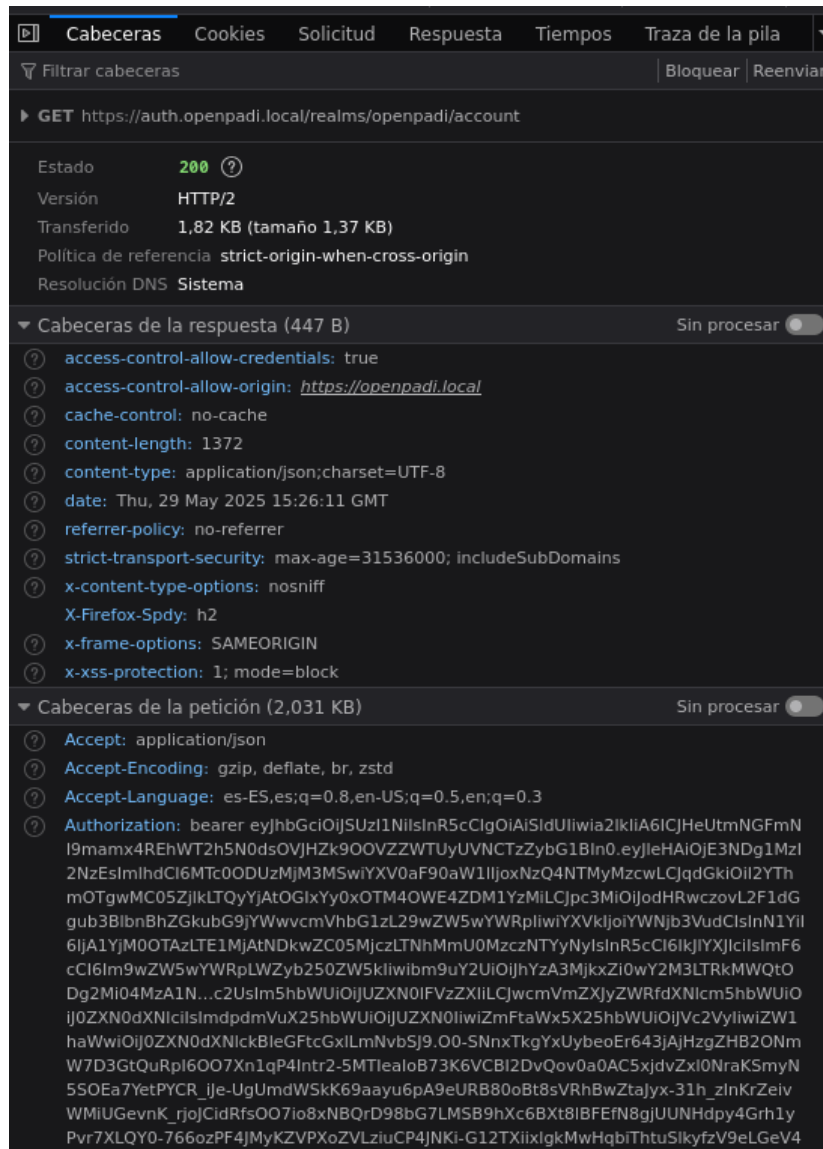
La URL relativa */api/documentos* funciona porque Traefik está configurado para enrutar las peticiones a <https://openpadi.local/api/>... al servicio del *backend*, mientras que las peticiones a <https://openpadi.local/>... (sin */api*) van al *frontend*.

- **Integración con Keycloak para Autenticación:**

³⁴ <https://github.com/robleslf/OpenPaDi/blob/main/frontend-svelte/src/routes/%2Bpage.svelte>

El archivo *keycloak.js* inicializa el cliente JavaScript de Keycloak, como se ha explicado antes.

Una vez autenticado, el *frontend* puede acceder al token JWT de Keycloak y enviarlo en las cabeceras de las peticiones a los *endpoints* protegidos de la API.



Al solicitar información de la cuenta del usuario al endpoint */realms/openpadi/account* de Keycloak, el *frontend* incluye el token JWT en la cabecera *Authorization* de la petición. Un mecanismo similar se emplea para las peticiones a los *endpoints* protegido

4.3.10. Servidor API Backend (FastAPI en K3s)

- **Motivo de su implantación:**

El Servidor API *Backend* es el que se encarga de toda la lógica de la aplicación y de la gestión de los datos. Actúa como el intermediario principal entre la interfaz de usuario y los sistemas de almacenamiento (PostgreSQL y MinIO), además de verificar la identidad de los usuarios a través de Keycloak. Su función es crucial para:

- Procesar solicitudes del Frontend: Recibe peticiones para acciones como listar, crear, actualizar o eliminar documentos y archivos.
- Interactuar con la Base de Datos: Guarda y recupera información sobre los documentos (títulos, fechas, transcripciones) en *PostgreSQL*.
- Gestionar Archivos Grandes: Sube los documentos digitalizados (PDFs, imágenes) a MinIO y proporciona enlaces seguros para su descarga.
- Aplicar Seguridad: Asegura que solo usuarios autenticados (con un token de Keycloak como se explicó en el apartado anterior) puedan realizar operaciones determinadas.

Se eligió FastAPI (un *framework* de *Python*) por su rapidez, facilidad para crear APIs y su capacidad para validar datos automáticamente. Todo esto se ejecuta en contenedores *Docker* dentro del clúster *K3s*.

- **Software empleado:**

- Framework: FastAPI (*Python* 3.11).
- Servidor: Uvicorn, necesario para ejecutar aplicaciones *Python* asíncronas como FastAPI.
- Librerías Esenciales:
 - *psycopg2-binary*: Para hablar con la base de datos PostgreSQL.
 - *minio*: Para interactuar con el almacenamiento de objetos MinIO.
 - *python-jose*: Para verificar los tokens de seguridad de Keycloak.
 - *python-multipart*: Para manejar la subida de archivos desde el *frontend*.

- **Entorno de Ejecución:**

Contenedor Docker orquestado por K3s, desarrollado en la VM OP-API-1.

- **Archivos más relevantes:**

- <https://github.com/robleslf/OpenPaDi/blob/main/backend-api/main.py>: Contiene todo el código Python: define las rutas (*endpoints* como */api/documentos*), la lógica para conectarse a *PostgreSQL* y *MinIO*, y cómo se validan los tokens de *Keycloak*.
- <https://github.com/robleslf/OpenPaDi/blob/main/backend-api/Dockerfile>: Detalla cómo empaquetar el código *Python* y sus dependencias para que *K3s* pueda ejecutarlo.
- <https://github.com/robleslf/OpenPaDi/blob/main/backend-api/requirements.txt>: Enumera todas las librerías que la API necesita para funcionar.
- <https://github.com/robleslf/OpenPaDi/blob/main/kubernetes-manifests/deployment-api.yaml>: Le dice a *K3s* cómo ejecutar la API: qué imagen Docker usar (*robleslf/opadi-api:latest*³⁵), cuántas copias tener, y qué puerto usa internamente (8000).
- <https://github.com/robleslf/OpenPaDi/blob/main/kubernetes-manifests/service-api.yaml>: Crea un nombre DNS interno (*op-api*) en el clúster para que otros servicios, como *Traefik*, puedan encontrar y comunicarse con la API.
- <https://github.com/robleslf/OpenPaDi/blob/main/kubernetes-manifests/ingress.yaml>: La regla para */api* en este archivo le dice a *Traefik* que dirija las peticiones externas a este servicio API.

- **Endpoints y Lógica de Datos:**

³⁵ <https://hub.docker.com/repository/docker/robleslf/opadi-api/tags/latest/sha256-4ccaaf13319f03daed1c992bd7e4c52f287f8b80babdbd89d6c8507cd72e1d21>

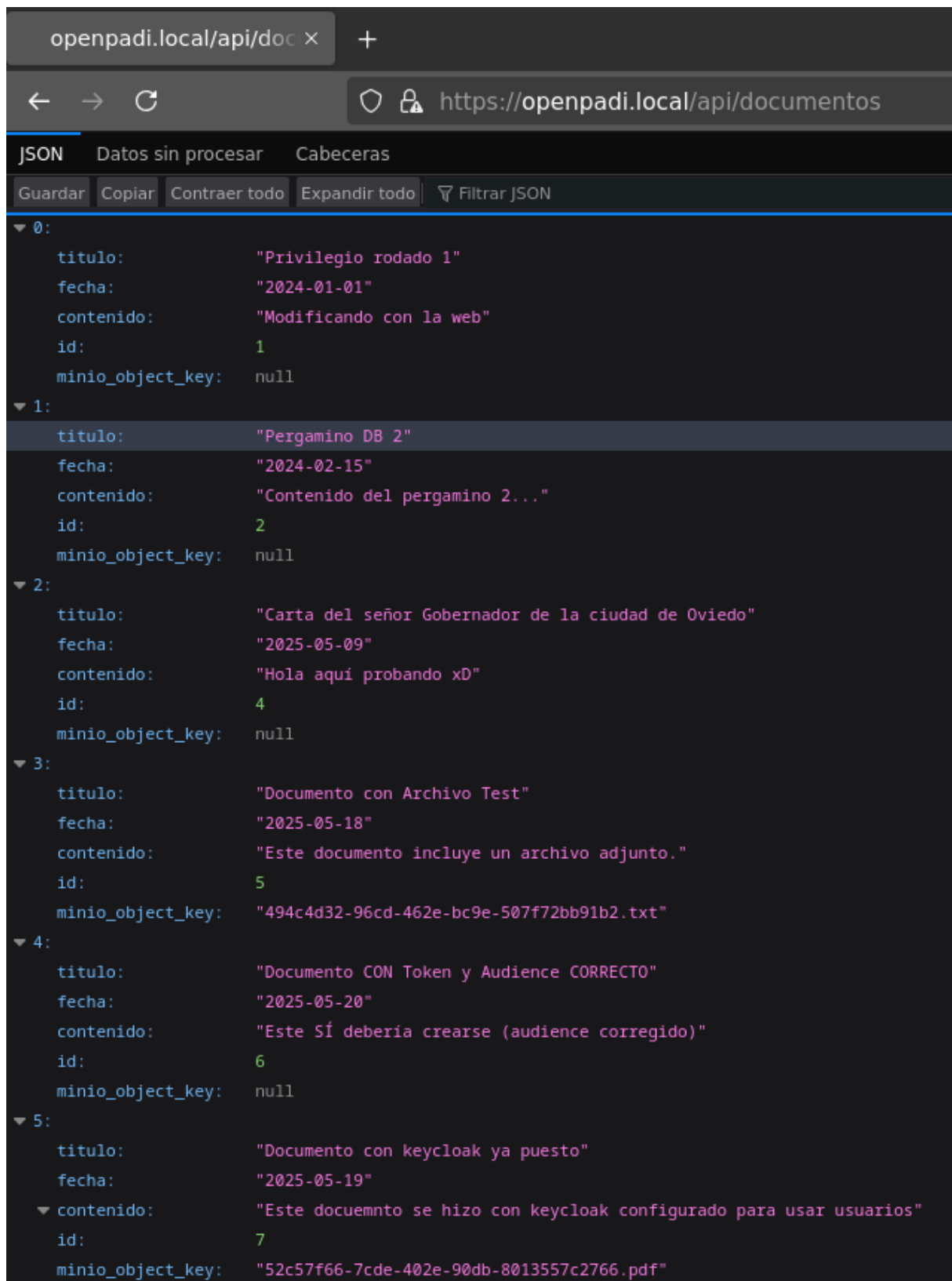
La API expone diferentes rutas (*endpoints*) como `/api/documentos` (para listar y crear) y `/api/documentos/{id}` (para leer, actualizar, eliminar uno específico), y `/api/documentos/{id}/archivo` (para descargar archivos).

```
@app.get("/api/documentos", response_model=List[Documento])
async def get_documentos_db():
    conn = None
    try:
        conn = get_db_connection()
        cur = conn.cursor(cursor_factory=psycopg2.extras.RealDictCursor)
        cur.execute("SELECT id, titulo, fecha, contenido, minio_object_key FROM documentos ORDER BY id;")
        documentos_data = cur.fetchall()
        cur.close()
        return documentos_data
    except psycopg2.OperationalError as db_conn_err:
        raise HTTPException(status_code=503, detail=f"Servicio de base de datos no disponible: {db_conn_err}")
    except psycopg2.Error as db_query_err:
        raise HTTPException(status_code=500, detail=f"Error de base de datos: {db_query_err}")
    except Exception as e:
        raise HTTPException(status_code=500, detail=f"Error interno inesperado del servidor: {e}")
    finally:
        if conn:
            conn.close()
```

36

Definición del endpoint GET `/api/documentos` para listar documentos

³⁶ <https://github.com/robleslf/OpenPaDi/blob/main/backend-api/main.py>



Respuesta JSON del endpoint GET /api/documentos mostrando la lista de documentos

- **Conexión Segura a Bases de Datos y Almacenamiento:**

Se conecta a la base de datos *PostgreSQL* (*opadi_db* en OP-db-primary) y al almacenamiento MinIO (*openpadi-documentos* en OP-storage-1) usando credenciales definidas en *main.py*.

```
DB_HOST = "192.168.1.13"
DB_NAME = "opadi_db"
DB_USER = "opadi_user"
DB_PASS = "abc123.."

MINIO_ENDPOINT = "192.168.1.14:9000"
MINIO_ACCESS_KEY = "openpadiadmin"
MINIO_SECRET_KEY = "abc123.."
MINIO_BUCKET_NAME = "openpadi-documentos"
MINIO_USE_SSL = False
```

37

Idealmente, en producción, serían gestionadas como secretos de Kubernetes.

Al iniciar, la API verifica que la tabla documentos y el *bucket* en MinIO existan, creándolos si es necesario.

³⁷ [Ibid.](#)

```

@app.on_event("startup")
async def startup_event():
    logger.warning("LOGGING: --- INICIO DEL EVENTO STARTUP FastAPI ---")

    try:
        logger.warning("LOGGING: Intentando llamar a get_keycloak_public_keys desde startup...")
        await get_keycloak_public_keys()
        logger.warning("LOGGING: Llamada a get_keycloak_public_keys desde startup completada (sin error explicito).")
        logger.warning("LOGGING: Configuración OpenID y JWKS URI verificados/precargados en startup (si la llamada anterior tuvo éxito).")
    except HTTPException as http_exc:
        logger.error(f"LOGGING: HTTPException durante precarga de Keycloak en startup: {http_exc.status_code} - {http_exc.detail}")
    except Exception as e:
        logger.error(f"LOGGING: EXCEPCIÓN GENERAL durante precarga de Keycloak en startup: TIPO={type(e)}, MSG={e}")

    logger.info("LOGGING: Continuando con la inicialización de MinIO...")
    if minio_client:
        try:
            logger.info(f"LOGGING: Verificando/creando bucket MinIO: {MINIO_BUCKET_NAME}")
            found = minio_client.bucket_exists(MINIO_BUCKET_NAME)
            if not found:
                minio_client.make_bucket(MINIO_BUCKET_NAME)
                logger.info(f"LOGGING: Bucket MinIO '{MINIO_BUCKET_NAME}' creado.")
            else:
                logger.info(f"LOGGING: Bucket MinIO '{MINIO_BUCKET_NAME}' ya existe.")
        except S3Error as s3_err:
            logger.error(f"LOGGING: Error S3 durante la verificación/creación del bucket MinIO: {s3_err}")
        except Exception as e:

```

38

Lógica de inicialización en `main.py` (`startup_event`) para verificar/crear el bucket en MinIO y la tabla 'documentos' en PostgreSQL

Name	Objects	Size	Access
 openpadi-documentos	8	22.3 MiB	R/W

Bucket openpadi-documentos en la interfaz web de Minio

- **Gestión de Archivos:**

Permite la subida de archivos, que se guardan en MinIO con un nombre único (UUID). La referencia a este archivo se guarda en PostgreSQL.

38 [Ibid.](#)

Crear Nuevo Documento

Título:

Inter

Fecha (YYYY-MM-DD):

03 / 05 / 2025



Contenido:

Probando la conexión entre MinIO y PostgreSQL

Archivo (PDF, Imagen, etc.):

Examinar... inter.txt

Crear Documento

Formulario del frontend OpenPaDi para la creación de un nuevo documento, incluyendo la selección de un archivo de prueba (inter.txt) que será almacenado en MinIO

https://openpadi.local

Documento Creado con Token FRESCO (ID: 10)
Fecha: 2025-05-20
Contenido: Prueba con token fresco y audiencia 'account' validado

hola (ID: 11)
Fecha: 2025-05-01
Contenido: sdfg
[Ver/Descargar Archivo](#)

Probando lo de tokes (ID: 12)
Fecha: 2025-05-08
Contenido: dsfg
[Ver/Descargar Archivo](#)

Probando permisos (ID: 13)
Fecha: 2025-05-06
Contenido: MAcedonia ARY
[Ver/Descargar Archivo](#)

probado_despues_de_reinicios (ID: 14)
Fecha: 2025-05-22
Contenido: Yugoslavia
[Ver/Descargar Archivo](#)

Inter (ID: 15)
Fecha: 2025-05-03
Contenido: Probando la conexión entre MinIO y PostgreSQL
[Ver/Descargar Archivo](#)

Lista de documentos en el frontend después de la creación. Se observa el nuevo documento "Inter (ID: 15)" con la opción para acceder a su archivo adjunto.

```
13:
  titulo:      "Inter"
  fecha:       "2025-05-03"
  contenido:   "Probando la conexión entre MinIO y PostgreSQL"
  id:          15
  minio_object_key: "b9d64fd4-e9cb-4fd5-9df1-18bbade87b7b.txt"
```

Respuesta JSON de la API para el documento "Inter (ID: 15)", mostrando el campo `minio_object_key`, que referencia el archivo almacenado en MinIO.

```
administrator@op-db-primary:~$ sudo -u postgres psql
[sudo] password for administrator:
psql (16.9 (Ubuntu 16.9-0ubuntu0.24.04.1))
Type "help" for help.

postgres=# \c opadi_db
You are now connected to database "opadi_db" as user "postgres".
opadi_db=# SELECT id, titulo, fecha, contenido, minio_object_key FROM documentos WHERE id = 15;
 id | titulo | fecha | contenido | minio_object_key
-----+-----+-----+-----+-----
 15 | Inter | 2025-05-03 | Probando la conexión entre MinIO y PostgreSQL | b9d64fd4-e9cb-4fd5-9df1-18bbade87b7b.txt
(1 row)

opadi_db=#
```

Consulta a la tabla 'documentos' en PostgreSQL para el ID 15, mostrando la columna minio_object_key con el identificador del archivo almacenado en MinIO, vinculando así el registro de la base de datos con el objeto en MinIO

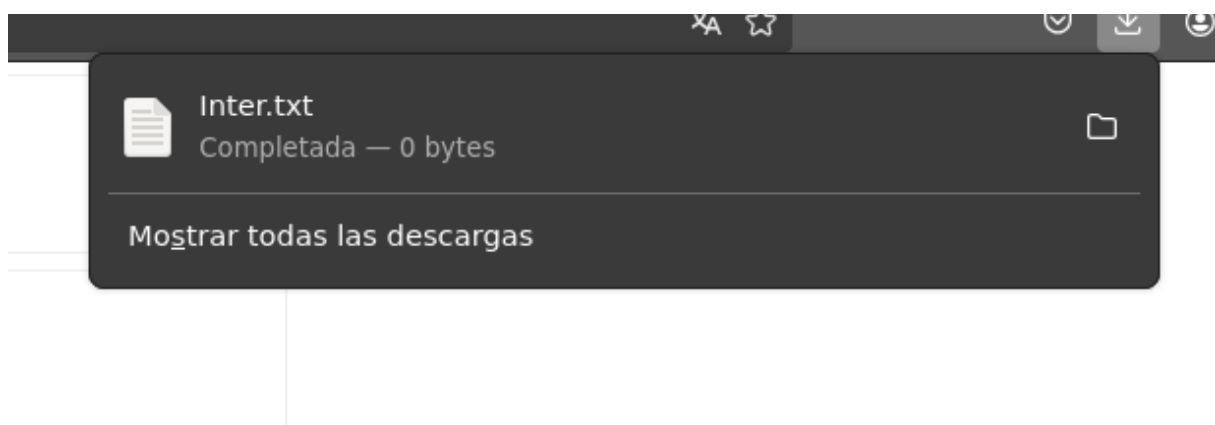
El flujo completo de las imágenes anteriores se puede resumir así: el *frontend* envía peticiones a la API; esta gestiona los metadatos con PostgreSQL y los archivos con MinIO, utilizando una clave en PostgreSQL para enlazar cada documento con su archivo correspondiente en MinIO, y luego devuelve la información o una URL de descarga al *frontend*.

Inter (ID: 15)

Fecha: 2025-05-03

Contenido: Probando la conexión e

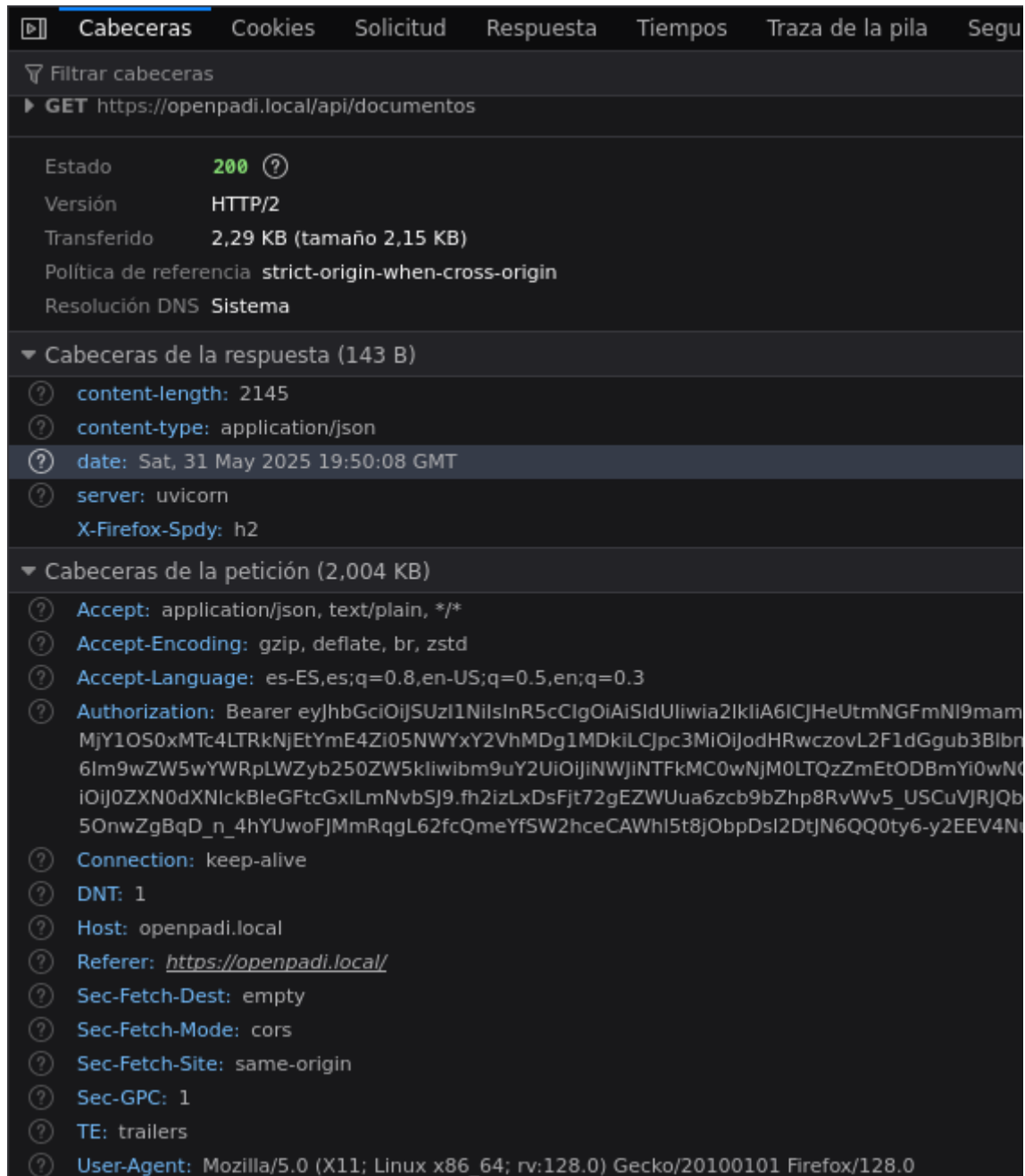
Ver/Descargar Archivo



Para la descarga directa de archivos desde MinIO, la API Genera URLs seguras y temporales forzando la descarga en el navegador.

- **Seguridad con Keycloak:**

Los *endpoints* que modifican datos están protegidos. La API valida los tokens JWT (firmados por Keycloak) enviados por el *frontend* en la cabecera *Authorization* para asegurar que el usuario tiene permiso:



Cabeceras de una petición GET autenticada a /api/documentos. Se observa la cabecera 'Authorization' con el token JWT Bearer enviado por el frontend, y la respuesta exitosa (200 OK) servida por Uvicorn (Backend API).

En la imagen anterior podemos observar:

- La petición se dirige a <https://openpadi.local/api/documentos>, uno de los *endpoints* definidos anteriormente.
- La petición fue exitosa, como indica el Estado: 200.
- La cabecera *server*: uvicorn en las confirma que la respuesta proviene del Backend API FastAPI (que corre sobre Uvicorn).
- En la Cabecera *Authorization* se demuestra que el *frontend* está enviando el token de acceso JWT obtenido de Keycloak al *Backend* API.

- **Despliegue Contenerizado:**

La API se empaqueta como una imagen Docker y se despliega en K3s, lo que facilita su gestión, escalado y actualización. Traefik se encarga de dirigir el tráfico externo a la ruta /api hacia los *pods* de esta API (ver apartado 4.3.7. Servicio de Orquestación de Contenedores (K3s)).

4.4. Base de Datos



La infraestructura PaaS desarrollada para OpenPaDi incluye un servicio de base de datos relacional (PostgreSQL) que es fundamental para la persistencia de datos tanto de la propia aplicación OpenPaDi como del sistema de autenticación Keycloak. A continuación, se detalla el modelo de datos principal utilizado por la aplicación OpenPaDi.

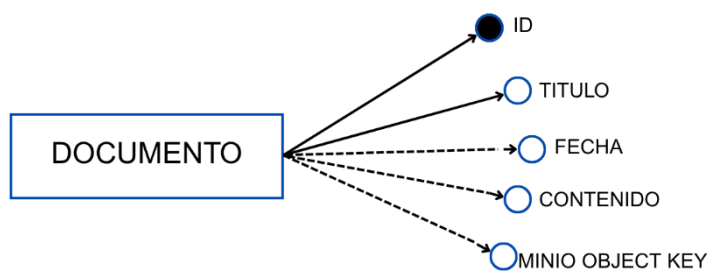
4.4.1. Modelo Entidad/Relación de la Aplicación OpenPaDi

La aplicación OpenPaDi, en su estado actual de desarrollo dentro de este proyecto, gestiona únicamente una entidad central: Documento. Esta entidad almacena los metadatos de los documentos creados en la plataforma y la referencia a sus archivos digitalizados.

- Entidades Principales:
 - Documento:
 - Descripción: Representa un documento histórico o una transcripción gestionada por la plataforma. Contiene la información descriptiva del documento y, opcionalmente, una referencia al archivo digitalizado almacenado en el sistema de objetos (MinIO).
 - Atributos:
 - id (Clave Primaria): Identificador único autoincremental para cada documento.
 - titulo: Título o nombre descriptivo del documento.
 - fecha: Fecha asociada al documento. Puede ser nula.
 - contenido: Transcripción textual del documento o una descripción más extensa. Puede ser nulo.
 - minio_object_key: Clave única (nombre del objeto) que identifica el archivo digitalizado asociado a este documento en el bucket de MinIO. Es nulo si el documento no tiene un archivo adjunto.

- **Diagrama Entidad/Relación:**

Para este proyecto, al estar este enfocado en la infraestructura de la PaaS, la Base de Datos utilizada ha sido extremadamente simple para comprobar la correcta integración del servicio PostgreSQL en *op-db-primary* con el resto de componentes. Por tanto, el diagrama E-R consta es muy simple y consta solo de una relación:



En PostgreSQL se ha implementado de la siguiente manera:

```

administrator@op-db-primary:~$ sudo -u postgres psql
psql (16.9 (Ubuntu 16.9-0ubuntu0.24.04.1))
Type "help" for help.

postgres=# \c opadi_db
You are now connected to database "opadi_db" as user "postgres".
opadi_db=# \dt
          List of relations
 Schema |   Name   | Type  | Owner
-----+-----+-----+-----
 public | documentos | table | opadi_user
(1 row)

opadi_db=#
  
```

La relación *documentos* representa a la entidad *Documento* descrita antes.

Table "public.documentos"				
Column	Type	Collation	Nullable	Default
id	integer		not null	nextval('documentos_id_seq'::regclass)
titulo	character varying(255)		not null	
fecha	date			
contenido	text			
minio_object_key	character varying(255)			

Indexes:

"documentos_pkey" PRIMARY KEY, btree (id)

El valor `nextval('documentos_id_seq'::regclass)` en el campo *Default* de *id* indica que es un valor autoincremental y actúa como clave primaria

```

opadi_db=# SELECT * FROM documentos;
id |          titulo          | fecha |          contenido          |          minio_object_key
-----+-----+-----+-----+-----
 2 | Pergamino DB 2           | 2024-02-15 | Contenido del pergamino 2... |
 4 | Carta del señor Gobernador de la ciudad de Oviedo | 2025-05-09 | Hola aquí probando xD      |
 1 | Privilegio rodado 1       | 2024-01-01 | Modificando con la web     |
 5 | Documento con Archivo Test | 2025-05-18 | Este documento incluye un archivo adjunto. | 494c4d32-96cd-462e-bc9e-507f72bb91b2.txt
 6 | Documento CON Token y Audience CORRECTO | 2025-05-20 | Este SÍ debería crearse (audience corregido) |
 7 | Documento con keycloak ya puesto | 2025-05-19 | Este documento se hizo con keycloak configurado para usar usuarios | 52c57f66-7cde-402e-90db-8013557c2766.pdf
 8 | asd                      | 2025-05-21 | asdfasd                    | 3956cb43-cbb3-48de-a9d5-bd082132eca0.pdf
 9 | Hola                     | 2025-05-29 | nomas                      | 692c0b1f-552c-4d8c-a432-e90d9f0924a2.mp3
10 | Documento Creado con Token FRESCO | 2025-05-20 | Prueba con token fresco y audience 'account' validado |
11 | hola                     | 2025-05-01 | sdfg                      | 89c287f0-4999-4e00-a857-9c957bd7657e.pdf
12 | Probando lo de tokes     | 2025-05-08 | dsfg                     | f7f8668c-ced4-4ea9-b07d-344b1730bf75.pdf
13 | Probando permisos        | 2025-05-06 | Macedonia ARY            | 6bfda54-255c-42e8-9075-9dc0977f3ac8.pdf
14 | probado_despues_de_reinicios | 2025-05-22 | Yugoslavia                | 9b0ceef2-9f94-42ad-a368-5019849ce403.pdf
15 | Inter                    | 2025-05-03 | Probando la conexión entre MinIO y PostgreSQL | b9d64fd4-e9cb-4fd5-9df1-18bbade87b7b.txt
(14 rows)

```

4.4.2. Uso de la Base de Datos por Keycloak

Además de la anterior Base de Datos, el mismo servidor PostgreSQL (*op-db-primary*) aloja la base de datos utilizada por Keycloak (llamada *keycloak_db*). *Keycloak* utiliza esta base de datos para persistir toda su configuración, incluyendo:

- *Realms* (en este caso solo tenemos el *realm openpadi*).
- Clientes (*openpadi-frontend* y *openpadi-api*).
- Usuarios y sus credenciales.
- Roles y permisos.
- Sesiones de usuario.
- Etc.

El esquema de la base de datos de Keycloak es complejo y está gestionado internamente por Keycloak. Para este proyecto, solo es relevante entender que Keycloak depende de esta instancia de PostgreSQL para su funcionamiento persistente, tal como se configuró en su *Deployment* de *Kubernetes*³⁹. No obstante, para cualquier cuestión relacionada con esta BD que pueda surgir, puede consultarse su diagrama E/R en el siguiente enlace: https://htmlpreview.github.io/?https://gist.githubusercontent.com/thomasdarimont/b1c19da5e8df747b8596e6ddcda7e36f/raw/29309467f4ea07519cf614fd74943272e7d939f4/keycloak_db_overview_4.0.0.CR1-SNAPSHOT.svg

³⁹ <https://github.com/robleslf/OpenPaDi/blob/main/kubernetes-manifests/keycloak-deployment.yaml>

4.5. Software y paquetes de Software

Este apartado detalla la configuración de software esencial para la estación de trabajo utilizada en el diseño, implementación, gestión y pruebas de la infraestructura PaaS para OpenPaDi. Esta configuración refleja las herramientas necesarias por el administrador de sistemas encargado de este proyecto, así como los sistemas operativos de las máquinas virtuales que componen la PaaS.

- **Sistema Operativo de las Máquinas Virtuales del Servidor (PaaS):**
 - Distribución: Ubuntu Server 22.04 LTS.
 - Justificación: es una elección robusta y ampliamente utilizada para entornos de servidor debido a su estabilidad, amplio soporte de la comunidad, ciclos de soporte a largo plazo (LTS), y excelente compatibilidad con Docker, K3s, PostgreSQL, MinIO y otras tecnologías clave de este proyecto.
- **Sistema Operativo Principal de la Estación de Trabajo Cliente (Administración/Desarrollo):**
 - Distribución Linux: en el proyecto actual y para gestionar las diferentes máquinas del escenario que se ha simulado, se ha utilizado Debian 12, que sería igualmente útil para la gestión y administración en un entorno de producción.
 - Justificación: Proporciona un entorno de línea de comandos nativo (para SSH, *kubectl*, *curl*, etc.) y compatibilidad con las herramientas de virtualización y navegadores web necesarios.
- **Software de Virtualización Local (Entorno de Pruebas en la Estación Cliente):**

Oracle VM VirtualBox: Empleado en la estación de trabajo cliente para crear y gestionar las máquinas virtuales Ubuntu Server que simulan los servidores del entorno PaaS. También se utiliza para alojar la instancia de Proxmox VE virtualizada y la VM del firewall OPNsense.

- **Herramientas de Conexión y Gestión Remota (en la Estación Cliente):**

- Cliente SSH: Imprescindible para el acceso por línea de comandos a todas las máquinas virtuales Ubuntu Server para su gestión y administración.
- Navegador Web Mozilla Firefox (se eligió por su eficiencia, rapidez y ser de código abierto, lo que encaja con la filosofía del proyecto): Utilizado para acceder a las interfaces de gestión web de:
 - OPNsense.
 - Consola web de MinIO.
 - Consola de administración de Keycloak.
 - Interfaz de la aplicación OpenPaDi (<https://openpadi.local>).
- **Herramientas de Gestión de Kubernetes (en la Estación Cliente vía SSH al Master K3s):**
 - kubectl: Para interactuar con el clúster K3s.
- **Herramientas de Desarrollo y Edición de Texto (en la Estación Cliente o en las VMs de desarrollo):**
 - Editor de Código/Texto Avanzado para crear y editar manifiestos YAML, *Dockerfiles*, y *scripts*: se utilizó Visual Studio Code para edición más compleja y Nano para edición puntual y sencilla.
 - Node.js y npm: Instalados en *OP-Web-1* para el desarrollo/construcción del frontend Svelte.
 - Python y pip (con python3-venv): Instalados en *OP-API-1* para el desarrollo/construcción del *backend* FastAPI.
- **Herramientas de Contenerización (en las VMs donde se construyen imágenes):**
 - Docker Engine/CLI (docker.io): Instalado en *OP-Web-1* (para el *frontend*) y *OP-API-1* (para el *backend*) para construir las imágenes Docker de las aplicaciones.
- **Software de Control de Versiones (en la Estación Cliente y/o VMs de desarrollo):**

- Git: Para la gestión del código fuente y la sincronización con GitHub.

4.6. Infraestructura Hardware

Este apartado detalla la infraestructura hardware necesaria para el despliegue y eficiente de la Plataforma como Servicio (PaaS). Se describe tanto la configuración de los servidores como las consideraciones para las estaciones de trabajo.

4.6.1. Servidores e distribución de servicios

La infraestructura de servidor para OpenPaDi se diseñará como una solución *on-premises*, utilizando servidores físicos sobre los cuales se implementará la plataforma de virtualización para alojar todos los componentes de la PaaS.

- **Configuración General de Servidores Físicos (Hosts Proxmox VE):**

Se utilizará un clúster de tres servidores físicos idénticos para asegurar la alta disponibilidad y la capacidad de migración en vivo de las máquinas virtuales. Esta configuración permite la tolerancia a fallos a nivel de host: si un servidor físico falla, las máquinas virtuales que aloja pueden reiniciarse o migrarse automáticamente a los otros nodos del clúster.

- **Modelos de máquinas empleadas:**

Se emplearán servidores de tipo rack de un fabricante reconocido (para asegurar una mayor fiabilidad y soporte técnico) optimizados para virtualización. La elección específica el modelo se basará en la mejor relación rendimiento/costo/soporte disponible en el momento de la adquisición.

- **Configuración hardware (por cada uno de los 3 servidores):**

- CPU:

- 2 procesadores multi-núcleo que ofrezcan un total de 32 a 64 núcleos físicos por servidor, con soporte para virtualización por hardware.
- Justificación: Múltiples núcleos y soporte de virtualización son esenciales para ejecutar eficientemente un gran

número de máquinas virtuales concurrentes que alojarán los diversos servicios de la PaaS.

- RAM:
 - Mínimo 256 GB de RAM ECC DDR4 (o superior) por servidor, idealmente 512 GB.
 - Justificación: Esta cantidad permite alojar las VMs de OpenPaDi y otros posibles servicios PaaS sin cuellos de botella, asegurando un buen rendimiento y espacio para crecimiento. ECC (*Error-Correcting Code*) es fundamental para la integridad de los datos en un servidor.
- Almacenamiento Local (para el Hipervisor y VMs):
 - 2 x Discos SSD NVMe de 1TB (o superior) en configuración RAID 1 (espejo) para el sistema operativo del hipervisor Proxmox VE y, opcionalmente, para VMs que requieran latencia de disco extremadamente baja o para almacenamiento temporal.
 - Justificación: RAID 1 proporciona redundancia para el SO del hipervisor. Los SSDs NVMe ofrecen el máximo rendimiento de I/O para el arranque del sistema y las operaciones críticas.
- Conectividad de Red:
 - Mínimo 4 x Interfaces de red Ethernet de 10 Gbps (SFP+ o RJ45 según el switch) por servidor. Dos se dedicarán al tráfico de datos de las VMs y la gestión del clúster Proxmox (agregadas mediante *LACP/bonding* para redundancia y mayor ancho de banda), y las otras dos se utilizarán para la conexión al almacenamiento centralizado (SAN/NAS) si se usa iSCSI o NFS sobre Ethernet.
 - 1 Interfaz de red dedicada de 1 Gbps para gestión remota.
 - Justificación: Múltiples interfaces de red de alta velocidad son necesarias para segmentar el tráfico (gestión, almacenamiento, VMs), proporcionar redundancia y

asegurar un ancho de banda suficiente para todas las operaciones.

- Fuentes de Alimentación:
 - Redundantes (2).
 - Justificación: Crítico para la alta disponibilidad, permitiendo que el servidor continúe funcionando si una fuente de alimentación falla.
- Interfaz de Gestión Remota:
 - Integrada, para gestión.
 - Justificación: Permite la administración del servidor (encendido, apagado, monitorización de hardware) incluso si el sistema operativo principal no está disponible.
- Almacenamiento Centralizado (SAN/NAS):
 - Se utilizará un sistema de almacenamiento de red dedicado (SAN o NAS de alto rendimiento) con discos SSD o una combinación híbrida SSD/HDD, controladoras redundantes y capacidad escalable.
 - Justificación: El almacenamiento compartido es esencial para las funcionalidades de clúster de Proxmox VE como la migración en vivo de VMs y la alta disponibilidad. Permite que cualquier host del clúster acceda a los discos virtuales de las VMs. Una solución dedicada ofrece mejor rendimiento, fiabilidad y gestionabilidad que depender únicamente de almacenamiento local distribuido para todas las VMs en un clúster de solo tres nodos. (Alternativamente, con más nodos o para ciertos escenarios, se podría considerar Ceph integrado en Proxmox usando discos locales de los hosts).
- Equipamiento de Red Físico:
- Switches de Núcleo/Agregación:
 - Mínimo dos switches gestionables L2/L3 de 10 Gbps (con suficientes puertos) configurados en *stack* o con protocolos de

redundancia. Todos los servidores físicos y el almacenamiento centralizado se conectarán de forma redundante a ambos switches.

- Justificación: La redundancia de switches elimina un punto único de fallo en la red. Soportarán VLANs (802.1Q), LACP, y tendrán capacidad de conmutación suficiente para el tráfico agregado.
- Router/Firewall Perimetral Físico:
 - Aunque el diseño principal virtualiza el firewall con OPNsense, en un entorno de producción existiría un par de dispositivos firewall físicos dedicados en HA en el borde de la red para una primera capa de seguridad y la gestión de las conexiones WAN. Para este proyecto, la solución primaria es la VM OPNsense, que se conectará a Internet a través de interfaces de red físicas del host Proxmox dedicadas a la WAN.
- Sistema de Alimentación Ininterrumpida (SAI/UPS):
 - Unidades SAI de capacidad suficiente para alimentar todos los servidores, el almacenamiento y los switches, permitiendo un apagado ordenado en caso de corte eléctrico prolongado o la continuidad con un generador.
 - Justificación: Protege contra la pérdida de datos y daños en el hardware debidos a fallos de alimentación.
- **Distribución de Servicios (sobre la plataforma de virtualización Proxmox VE):**

Para asegurar la alta disponibilidad y un rendimiento óptimo, las máquinas virtuales que componen la PaaS OpenPaDi se distribuirán estratégicamente entre los tres servidores físicos descritos anteriormente, que conformarán el clúster Proxmox VE. Proxmox VE gestionará la migración o reinicio de VMs en caso de fallo de un host. A continuación, se presenta una distribución conceptual:

- **VM Firewall/Router (OPNsense) - op-router-fw:**
 - Distribución:

- op-router-fw-A en Host 1.
- op-router-fw-B en Host 2.

La redundancia del firewall es crítica para la conectividad y seguridad perimetral. Si un host o una VM de firewall falla, la otra puede tomar el control.

- **VMs Nodos Master K3s - op-web-X:**

Para un plano de control K3s altamente disponible, se requieren tres VMs actuando como nodos master (o un número impar para consenso).

- Distribución:
 - op-web-1 (*Master*) en Host 1.
 - op-web-2 (*Master*) en Host 2.
 - op-web-3 (*Master*) en Host 3.
- Servicios Alojados (distribuidos por K3s entre estos masters/workers iniciales):
 - Traefik Ingress Controller
 - Cert-Manager
 - Keycloak

Un clúster K3s con múltiples masters asegura la disponibilidad y la orquestación incluso si un nodo master falla. K3s distribuirá las cargas de los componentes del sistema entre ellos.

- **VMs Nodos Worker K3s - op-api-X:**

Mínimo dos VMs para los nodos *worker* inicialmente, con la capacidad de añadir más según la carga.

- Distribución:
 - op-api-1 (*Worker*) en Host 1.
 - op-api-2 (*Worker*) en Host 2.
 - ...

- Servicios Alojados (distribuidos por K3s): Principalmente los pods del *Backend API* FastAPI y pods del *Frontend* Svelte.

Múltiples nodos *worker* permiten distribuir la carga de las aplicaciones, mejorar el rendimiento y asegurar la disponibilidad de las aplicaciones si un nodo *worker* falla (K3s reprogramaría los *pods* en otros nodos disponibles).

- **VMs Base de Datos (PostgreSQL) - op-db-X:**

Se desplegarán dos VMs PostgreSQL configuradas para replicación.

- Distribución:
 - op-db-primary en Host 1.
 - op-db-secondary (Réplica) en Host 2.

La replicación asegura la alta disponibilidad de la base de datos. Si la VM primaria o el host donde reside falla, se puede promover la réplica a primaria.

- **VMs Almacenamiento de Objetos (MinIO) - op-storage-X:**

Para un despliegue de MinIO distribuido y resiliente se necesitan un mínimo de tres VMs.

- Distribución:
 - op-storage-1 en Host 1.
 - op-storage-2 en Host 2.
 - op-storage-3 en Host 3.

El modo distribuido de MinIO protege contra la pérdida de datos incluso si una o más VMs/discos fallan. Distribuir las VMs de MinIO entre los hosts físicos maximiza esta resiliencia.

- **VMs Monitorización (Pila Prometheus) - op-monitoring-X:**

Idealmente, los componentes de monitorización también deberían ser redundantes. Podríamos tener dos VMs con una configuración en HA.

- Distribución:

- op-monitoring-A en Host 1.
- op-monitoring-B en Host 2.
- **VM Cliente/Administración - op-client-1:**
 - Despliegue: 1 VM.
 - Distribución: Puede residir en cualquiera de los hosts.

No es un servicio crítico para el funcionamiento de OpenPaDi para los usuarios finales, pero es útil para la administración. Su HA no es tan prioritaria como la de los otros servicios, aunque pueden conectarse tantos clientes como sean necesarios.

- **Resumen de la Distribución por Host:**

- **Host 1:**
 - op-router-fw-A (OPNsense)
 - op-web-1 (K3s *Master*)
 - op-api-1 (K3s *Worker*)
 - op-db-primary (PostgreSQL Primario)
 - op-storage-1 (MinIO Nodo 1)
 - op-monitoring-A (Monitorización)
- **Host 2:**
 - op-router-fw-B (OPNsense Secundario/Pasivo)
 - op-web-2 (K3s *Master*)
 - op-api-2 (K3s *Worker*)
 - op-db-secondary (PostgreSQL Réplica)
 - op-storage-2 (MinIO Nodo 2)
 - op-monitoring-B (Monitorización)
- **Host 3:**
 - op-web-3 (K3s *Master*)
 - (Opcional op-api-3 K3s *Worker*)
 - op-storage-3 (MinIO Nodo 3)
 - op-client-1 (Administración)

4.6.2. Estaciones de trabajo

La infraestructura hardware para las estaciones de trabajo del personal técnico de encargado del diseño, implementación y gestión de la PaaS OpenPaDi, se distribuye de la siguiente forma:

- **Configuración Hardware Típica (para Administradores/Desarrolladores de Infraestructura):**
 - Ordenadores portátiles de gama profesional o estaciones de trabajo de sobremesa.
 - CPU: Procesador multi-núcleo.
 - RAM: Mínimo 16 GB, recomendable 32 GB.
 - Almacenamiento: Disco SSD NVMe de 512 GB o superior.
 - Pantalla(s): Una o dos pantallas de alta resolución.
 - Conectividad: Gigabit Ethernet y Wi-Fi.

Esta configuración asegura el rendimiento necesario para ejecutar software de virtualización local (como VirtualBox para la simulación del entorno Proxmox y las VMs del proyecto), herramientas de desarrollo (IDEs, Docker, *kubectf*), múltiples instancias de navegador para acceder a interfaces de gestión web, y mantener una productividad eficiente durante las tareas de administración y desarrollo de la infraestructura.

- **Consideraciones para los usuarios finales:**

Los usuarios finales de la plataforma OpenPaDi solo requerirían una estación de trabajo estándar con un navegador web moderno.

4.7. Seguridad

El diseño de seguridad de la infraestructura PaaS para OpenPaDi se aborda desde múltiples capas para proteger la plataforma, los datos y los accesos.

- **Mecanismos de Vigilancia y Acceso Exterior/Interior:**
 - Acceso Exterior Controlado: El acceso a los servicios web (OpenPaDi, Keycloak) desde el exterior se canaliza a través de un *Ingress Controller* (Traefik) situado conceptualmente en una Zona

Desmilitarizada (DMZ) virtual. Todo el tráfico es cifrado mediante HTTPS (TLS).

- Acceso Administrativo: El acceso a la gestión de la infraestructura (Proxmox, OPNsense, nodos K3s) se restringe a redes de administración específicas y se realiza preferentemente mediante SSH con autenticación basada en claves y, donde aplique, VPNs seguras.
- Monitorización: Se prevé una pila de monitorización (Prometheus, Grafana, Loki) en entorno de producción, para supervisar el estado de la infraestructura y las aplicaciones, permitiendo la detección temprana de anomalías o actividad sospechosa.
- **Seguridad Perimetral:**
 - Firewall Virtualizado (OPNsense): Actúa como el firewall perimetral principal. Controla todo el tráfico entre Internet y las VLANs internas, así como el tráfico entre las propias VLANs. Se configuran reglas estrictas para permitir solo el tráfico necesario a los servicios expuestos (ej. HTTPS al *Ingress Controller*) y denegar el resto por defecto.
 - Terminación TLS en el Perímetro: Traefik gestiona los certificados TLS y termina las conexiones HTTPS, inspeccionando el tráfico antes de dirigirlo a los servicios internos.
- **Seguridad Interna: Lógica y de Accesos:**
 - Segmentación de Red (VLANs): La red interna se segmenta en múltiples VLANs (DMZ, Aplicaciones, Base de Datos, Almacenamiento, Gestión, Monitorización) gestionadas por OPNsense. Esto aísla los servicios y limita el impacto de una posible brecha de seguridad en un segmento.
 - Políticas de Firewall Inter-VLAN: OPNsense aplica reglas de firewall específicas para controlar qué VMs pueden comunicarse entre sí a través de diferentes VLANs y por qué puertos.
 - Seguridad en Kubernetes (K3s):

- *Network Policies* (Potencial): Aunque no detallado en la implementación actual, Kubernetes permite definir *Network Policies* para controlar el tráfico entre *pods* dentro del clúster.
- RBAC (Role-Based Access Control): K3s utiliza RBAC para controlar el acceso al API server de Kubernetes.
- Autenticación Centralizada: Keycloak gestiona la identidad de los usuarios y la autenticación para acceder a la aplicación OpenPaDi. La API valida tokens JWT para proteger sus *endpoints*.
- Mínimo Privilegio: Se configuran usuarios y permisos específicos para cada servicio (ej. usuarios dedicados en PostgreSQL para la API y Keycloak).
- **Copias de Seguridad:**
 - Se implementará una estrategia robusta y automatizada de copias de seguridad para garantizar la recuperación de datos y la continuidad del servicio OpenPaDi ante cualquier incidente.
 - Cómo:
 - Máquinas Virtuales (Proxmox VE): Se realizarán *snapshots* y *backups* completos/incrementales de todas las VMs críticas (*op-router-fw*, nodos K3s, *op-db-primary*, *op-storage-X*, etc.) utilizando Proxmox Backup Server.
 - Base de Datos (PostgreSQL): Se configurarán dumps lógicos diarios (*pg_dumpall*) de la base de datos *opadi_db* y *keycloak_db*.
 - Almacenamiento de Objetos (MinIO): Se establecerá una replicación continua (*mirroring*) del *bucket openpadi-documentos* hacia una segunda instancia de MinIO (idealmente en un servidor o almacenamiento distinto) o, en su defecto, *backups* diarios del contenido del *bucket*.
 - Configuraciones de K3s y Aplicaciones: Se respaldarán diariamente los manifiestos YAML de Kubernetes y la configuración del clúster K3s (*snapshot* de *etcd/SQLite*).
 - Dónde:

- Todas las copias de seguridad se almacenarán en un sistema de almacenamiento de *backup* dedicado y aislado de la infraestructura de producción. Para una recuperación ante desastres óptima, se considerará la replicación de estos *backups* a una ubicación geográfica externa o secundaria.
- Cuándo (Frecuencia y Retención):
 - Las copias de seguridad se programarán diariamente para los datos más críticos (bases de datos, configuraciones K3s) y semanalmente para *backups* completos de VMs. Se definirán políticas de retención para conservar múltiples puntos de recuperación (por ejemplo, diarios por 7 días, semanales por 4 semanas, mensuales por 6 meses). Se realizarán pruebas de restauración trimestrales para validar la integridad de los *backups* y el procedimiento de recuperación.

5. PRESUPUESTO DE LA SOLUCIÓN

A continuación, se presenta el presupuesto detallado para la implementación de la infraestructura PaaS OpenPaDi en un entorno de producción, considerando los recursos materiales y humanos necesarios. Este presupuesto refleja los costos estimados para una solución robusta y escalable.

5.1. Desglose de Costos:

- Estimación de Costos Recursos Materiales (Hardware):

Componente	Cantidad	Coste unitario (€)	Coste total (€)
Servidores Físicos (Hosts Proxmox VE)	3	6.000	18.000
Almacenamiento Centralizado (SAN/NAS)	1	5.500	5.500

Equipamiento de Red (Switches 10GbE, cableado)		2.500	2.500
Sistema de Alimentación Ininterrumpida (SAI/UPS)		1.000	1.000
Estaciones de Trabajo	2	1.500	3.000
SUBTOTAL			30.000

- **Estimación de Costos de Recursos Materiais (Software y Licencias)⁴⁰**

Componente	Tipo	Coste (€)
Proxmox VE, OPNsense, K3s, Docker, PostgreSQL, MinIO, Keycloak, Nginx, Svelte, FastAPI, Ubuntu Server LTS	Código abierto	0
SUBTOTAL		0

- **Estimación de Costos de Recursos Humanos**

Perfil Técnico	Nº Personas	Jornadas Estimadas/Persona	Tarifa Diaria Estimada (€)	Costo Total Estimado (€)
Líder Técnico/Arquitecto PaaS	1	11	400	4.400

⁴⁰ No se incluyen costes de suscripciones opcionales de soporte.

Técnico de Infraestructura y Soporte PaaS	1	11	400	4.400
SUBTOTAL				8.800

- **Presupuesto Total da Solución Propuesta**

Categoría	Costo Total Estimado (€)
Subtotal Hardware	30.000
Subtotal Software y Licencias	0
Subtotal Recursos Humanos	8.800
PRESUPUESTO TOTAL ESTIMADO	38.800

Este presupuesto se centra en la inversión inicial para la puesta en marcha. Gastos operativos recurrentes como el consumo energético de los servidores o la conectividad a Internet se considerarían adicionalmente en un plan financiero a largo plazo.

Será la Fundación Patrimonio Digital quien se encargue de financiar estos gastos, sin descartar la búsqueda activa de subvenciones y patrocinadores adicionales que puedan solventar la carga económica.

6. PROPUESTA DE MEJORAS

La solución PaaS diseñada para OpenPaDi establece una base sólida y robusta. No obstante, existen diversas áreas donde se podrían aplicar futuras ampliaciones y mejoras para potenciar aún más la plataforma.

6.1. Ampliación y Escalabilidad de la Infraestructura

- **Escalado Horizontal Completo del Clúster Proxmox VE y K3s:** Aunque el diseño contempla clústeres, la implementación actual se simuló con recursos limitados. Una futura mejora sería expandir físicamente el clúster Proxmox VE con más nodos host y, consecuentemente, el clúster K3s con más nodos *master* y *worker* para mejorar la resiliencia y capacidad de carga, tal como se

esbozó en la distribución de servicios para producción (apartado 4.6.1). Igualmente, sería conveniente utilizar Kubernetes en vez de la versión ligera K3S, que en este proyecto se utilizó debido a las limitaciones de hardware.

- Implementación de Almacenamiento Distribuido Avanzado para MinIO: Pasar del modo *standalone* de MinIO (utilizado en la simulación) a un despliegue distribuido con sobre múltiples nodos y discos, como se planeó para el entorno de producción.
- Alta Disponibilidad Avanzada para PostgreSQL: Implementar soluciones de clúster para PostgreSQL que vayan más allá de la replicación *streaming* básica, ofreciendo *failover* automático más robusto y balanceo de carga de lecturas.
- Redundancia Geográfica: Establecer un sitio de recuperación de desastres en una ubicación geográfica diferente, con replicación periódica de datos críticos para garantizar la continuidad del negocio ante fallos catastróficos en el sitio principal.
- Migración de estructura *on-premise* a estructura híbrida, aprovechando lo mejor de los servidores físicos con lo mejor de los servicios en la nube.

6.2. Incorporación de Nuevos Servicios y Funcionalidades

Aunque este proyecto no pretende desarrollar la aplicación para OpenPaDi, se indicarán también unas propuestas de mejora para la aplicación básica que se ha utilizado en la simulación de los escenarios de la infraestructura PaaS.

- Para la Aplicación OpenPaDi:
 - Búsqueda de Texto Avanzada: Integrar un motor de búsqueda como *Elasticsearch* u *OpenSearch* para permitir búsquedas textuales complejas dentro de las transcripciones, mejorando la capacidad de descubrimiento.
 - Asistencia por IA para Transcripción: Incorporar o facilitar la integración con herramientas de reconocimiento óptico de caracteres (OCR) para ayudar en el proceso de transcripción inicial.
 - Herramientas de Colaboración Avanzadas: Añadir funcionalidades como versionado de transcripciones, sistemas de anotación colaborativa y foros de discusión específicos por documento.

- Para la Plataforma PaaS:
 - Despliegue Automatizado de la Infraestructura (Infraestructura como Código - IaC): Implementar herramientas como Terraform para definir manera automatizada y reproducible toda la infraestructura de la PaaS, con el fin de agilizar, evitar errores humanos, y proveer de manera más eficiente la infraestructura a nuevos clientes.

6.3. Implantación de Tecnologías Emergentes

- Gestión de Infraestructura y Aplicaciones basada en *GitOps*: Utilizar herramientas como Flux para implementar un modelo *GitOps*, con el fin de automatizar y auditar los cambios.

6.4. Otras Mejoras y Elementos Pendientes

- Realizar pruebas exhaustivas de *failover* en todos los niveles (Proxmox, OPNsense HA, K3s, PostgreSQL, MinIO) y no solo simular la capa física de HA.
- Implementar y probar rigurosamente los planes de recuperación ante desastres, incluyendo la restauración en entornos de prueba aislados.
- Implementar *Network Policies* en K3s, realizar escaneos de vulnerabilidades periódicos, auditorías de seguridad y considerar pruebas de penetración.
- Realizar análisis más profundos para optimizar el uso de recursos y los costes operativos de la PaaS.
- Implementar los servicios de monitorización con servicios como Grafana, Prometheus y Loki.

7. CONCLUSIONES

El presente proyecto ha abordado con éxito el diseño y la implementación conceptual de una infraestructura de Plataforma como Servicio (PaaS) robusta y moderna, destinada a alojar la aplicación web OpenPaDi para la investigación en Humanidades. Se ha conseguido materializar una arquitectura que prioriza la

seguridad, la alta disponibilidad, la escalabilidad y la eficiencia operativa, utilizando principalmente software de código abierto y estándares consolidados.

7.1. Logros Principales:

- Se ha diseñado una arquitectura PaaS completa, desde la virtualización con Proxmox VE y la segmentación de red con OPNsense, hasta la orquestación de contenedores con K3s y la gestión de servicios de *backend* esenciales como PostgreSQL, MinIO y Keycloak.
- Se ha validado la funcionalidad de los componentes clave de la infraestructura y su correcta interacción, demostrando la viabilidad de la solución técnica propuesta para soportar una aplicación web compleja como OpenPaDi.
- La simulación del entorno, aun con limitaciones de hardware, ha permitido poner en práctica la configuración de servicios de red vitales (DHCP, DNS, Firewall), la gestión de identidades, el almacenamiento de objetos y bases de datos, y la exposición segura de aplicaciones mediante *Ingress* y gestión de certificados TLS.
- Se han abordado los requisitos específicos del proyecto OpenPaDi, ofreciendo una solución que facilitará el acceso y el intercambio de transcripciones digitalizadas, democratizando el acceso al patrimonio histórico escrito.

En la siguiente tabla podemos observar el cumplimiento de cada uno de los objetivos propuestos en el apartado 1.3. de la presente memoria:

OBJETIVO	¿SE CUMPLIÓ?	¿CÓMO?
OBJETIVO GENERAL		
Diseñar, implementar y gestionar infraestructura robusta y moderna	☑	Se diseñó una arquitectura PaaS completa que integra virtualización (Proxmox VE), segmentación de red (OPNsense), orquestación de contenedores (K3s) y servicios de backend esenciales, demostrando una solución moderna y funcional.
Sobre la cual se desplegará y operará OpenPaDi	☑	Toda la infraestructura se concibió y se validó para soportar las necesidades de la aplicación OpenPaDi, sirviendo como la

		plataforma base para su operación.
Ofrecer un entorno colaborativo, abierto, seguro y escalable	✓	Se logró mediante el uso de software de código abierto (fomentando apertura), la implementación de K3s y una arquitectura modular (escalabilidad), y múltiples capas de seguridad (firewall, TLS, autenticación centralizada) para un entorno seguro.
Facilitar búsqueda y análisis de fuentes históricas, democratizando el acceso	✓	La infraestructura diseñada y parcialmente implementada provee los servicios necesarios (bases de datos, almacenamiento, acceso web) que permitirán a OpenPaDi ofrecer estas funcionalidades a sus usuarios.
OBJETIVOS ESPECÍFICOS A: FUNCIONALES DE LA APLICACIÓN (SOPORTADOS POR INFRA.)		
A1. Gestión Integral de Transcripciones	✓	La infraestructura soporta la subida, almacenamiento (MinIO), consulta y descarga de documentos mediante la integración de un sistema de almacenamiento de objetos y una base de datos relacional (PostgreSQL) para los metadatos.
A2. Descubrimiento Eficiente de Fuentes	✓	Se implementó una base de datos PostgreSQL que permite almacenar y consultar metadatos descriptivos, facilitando así la búsqueda y filtrado eficiente de las transcripciones.
A3. Acceso Seguro mediante Autenticación	✓	Se integró Keycloak como sistema de gestión de identidades, controlando el acceso a la plataforma y sus recursos, asegurando que solo usuarios autorizados operen según sus permisos.
OBJETIVOS ESPECÍFICOS B: TÉCNICOS DE LA INFRAESTRUCTURA PAAS		

B1. Adopción de Estándares y Código Abierto	✓	La solución se construyó prioritariamente con software libre y tecnologías estándar (Proxmox VE, OPNsense, K3s, PostgreSQL, MinIO, Docker, etc.), asegurando transparencia, flexibilidad y sostenibilidad.
B2. Arquitectura Modular y Orientada a Servicios	✓	La plataforma se diseñó con componentes independientes y bien definidos (<i>frontend</i> , API, base de datos, almacenamiento, autenticación) desplegados como servicios/microservicios en K3s, facilitando el desarrollo, despliegue y escalado.
B3. Alta Disponibilidad y Resiliencia mediante Redundancia	✓	Se diseñó la redundancia en componentes críticos (firewall, nodos K3s, BBDD, almacenamiento) y se validó la resiliencia a nivel de servicios virtualizados. La HA física completa se simuló debido a limitaciones de hardware.
B4. Orquestación Automatizada de Contenedores	✓	Se utilizó K3s para la gestión automatizada del ciclo de vida de las aplicaciones contenerizadas (OpenPaDi Frontend, API, Keycloak), optimizando el uso de recursos, escalado y auto-reparación.
B5. Red Segura, Segmentada y Controlada	✓	Se estableció una infraestructura de red con segmentación lógica (VLANs gestionadas por Proxmox VE) y un perímetro de seguridad robusto (firewall OPNsense) para proteger activos y controlar el flujo de datos.
B6. Exposición Segura de Servicios Web	✓	El acceso externo a la aplicación se realiza a través de Traefik Ingress Controller, que gestiona el tráfico cifrado (TLS) con certificados de Cert-Manager,

		asegurando un punto de entrada único y seguro.
B7. Almacenamiento de Datos Persistente, Seguro y Fiable	✓	Se proveyeron soluciones de almacenamiento (PostgreSQL para metadatos, MinIO para archivos) que garantizan integridad, confidencialidad y disponibilidad, con capacidades de respaldo y redundancia consideradas en el diseño.
B8. Plan Efectivo de Continuidad del Negocio (Backup y Recuperación)	✓	Se definieron estrategias de copias de seguridad automatizadas y se consideró un plan de recuperación ante desastres. La implementación y prueba exhaustiva de estos planes se identificó como una mejora futura.
B10. Soporte Funcional a la Aplicación OpenPaDi	✓	La PaaS desarrollada demostró su capacidad para alojar y servir de manera eficiente la interfaz de usuario y las funcionalidades básicas de OpenPaDi, validando la idoneidad de la infraestructura mediante la interacción de los servicios.

7.2. Desafíos y Aprendizajes:

- La principal limitación ha sido la disponibilidad de hardware para replicar fielmente un entorno de producción con alta disponibilidad física. Esto ha llevado a simular ciertos aspectos y a centrar los esfuerzos en la resiliencia a nivel de servicios virtualizados. Fue necesario separar el escenario final completo en dos escenarios diferentes: el primero en Proxmox VE para replicar la segmentación de red y las reglas de firewall y el segundo en Virtual Box para comprobar la correcta integración de los servicios de la red. Además, Proxmox VE está pensado para instalarse directamente como hipervisor en los servidores físicos, sin añadir una capa extra más de abstracción como es Virtual Box; esto ha dado muchos problemas, especialmente en el diseño de red, a la

hora de entender cómo estaban interpretando las redes las diferentes capas de abstracción del escenario (Sistema Operativo anfitrión → Virtual Box → Proxmox VE → VM)

- La integración de múltiples componentes tecnológicos de diversa naturaleza ha supuesto un reto significativo, que ha requerido una comprensión profunda de cada uno de ellos y de sus interdependencias, en especial su comunicación a través de *Traefik* y la interacción entre los diferentes microservicios del clúster K3s.
- El proyecto ha permitido adquirir un conocimiento práctico sobre tecnologías de virtualización, segmentación de redes, orquestación de contenedores, gestión de identidades y seguridad en infraestructuras de TI, así como sobre la administración de sistemas informáticos en red.
- Se ampliaron conocimientos aprendidos en los dos años de ciclo a tecnologías más robustas y que a día de hoy son cada vez más usadas.

7.3. Impacto y Valor:

Este trabajo cumple con los objetivos académicos del ciclo formativo, asentando unas bases para una plataforma con potencial real de impacto en el ámbito de las Humanidades Digitales. La solución propuesta representa un ejemplo práctico de cómo se pueden construir infraestructuras tecnológicas complejas, seguras y escalables, utilizando conocimientos de los siguientes módulos del programa:

- Redes y Servicios de Red: diseño y topología de red física y lógica, segmentación de red, firewall, servicios DNS, DHCP, proxy inverso...
- Bases de Datos y SGBD: implementación y configuración de PostgreSQL e integración en el escenario final.
- Seguridad y Alta Disponibilidad: seguridad perimetral con OPNsense, gestión de certificados, segmentación de red, redundancia y diseño para HA, etc.
- Fundamentos de Hardware: planificación y diseño de la infraestructura física subyacente.

- Lenguajes de Marcas e Implantación de Aplicaciones Web: despliegue de un *frontend* útil, uso de contenedores y orquestación para el despliegue de la aplicación web.
- Implementación y Aplicación de Sistemas Operativos: instalación y configuración de los SO de cada máquina, así como la instalación y configuración de los servicios correspondientes a cada máquina.
- Empresa e Iniciativa Emprendedora: aunque en menor medida, sirvió para el análisis del contexto empresarial y la elaboración de un presupuesto para el proyecto.

En definitiva, el proyecto afronta la capacidad de diseñar, implementar y gestionar una infraestructura PaaS moderna, alineada con las necesidades actuales de despliegue de aplicaciones web.

Aunque se trata solo de un proyecto académico, las mejoras propuestas en el apartado anterior señalan un camino para su evolución y consolidación, asegurando su utilidad a largo plazo. El conocimiento y la experiencia adquiridos durante su realización son de gran valor para el futuro profesional en el ámbito de la administración de Sistemas y Redes.

8. BIBLIOGRAFÍA, ENLACES Y RECURSOS

A continuación, se presenta una lista de la documentación oficial de las tecnologías empleadas, los documentos de referencia utilizados para la estructuración y evaluación del proyecto, y otros recursos web relevantes.

8.1. Documentación Oficial de Tecnologías Utilizadas

Proxmox. “Proxmox Virtual Environment - Open-Source Server Virtualization”. [En línea]. Disponible en: <https://www.proxmox.com/en/proxmox-ve>

OPNsense. “OPNsense® a high-end open source firewall”. [En línea]. Disponible en: <https://opnsense.org/>

K3s. “Lightweight Kubernetes”. [En línea]. Disponible en: <https://k3s.io/>

Docker. “Docker: Accelerate how you build, share, and run applications”. [En línea]. Disponible en: <https://www.docker.com/>

PostgreSQL. “PostgreSQL: The World's Most Advanced Open Source Relational Database”. [En línea]. Disponible en: <https://www.postgresql.org/>

MinIO. “MinIO | High Performance, Kubernetes Native Object Storage”. [En línea]. Disponible en: <https://min.io/>

Keycloak. “Open Source Identity and Access Management”. [En línea]. Disponible en: <https://www.keycloak.org/>

Traefik Proxy. “Traefik Proxy - The Cloud Native Application Proxy”. [En línea]. Disponible en: <https://traefik.io/traefik/>

Svelte. “Cybernetically enhanced web apps”. [En línea]. Disponible en: <https://svelte.dev/>

FastAPI. “FastAPI framework, high performance, easy to learn, fast to code, ready for production”. [En línea]. Disponible en: <https://fastapi.tiangolo.com/>

Cert-Manager. “X.509 certificate management for Kubernetes”. [En línea]. Disponible en: <https://cert-manager.io/>

8.2. Documentación Académica y Guías de Referencia

Consellería de Cultura, Educación e Universidade, Xunta de Galicia, “Guía do módulo do proxecto fin de ciclo - Ciclo superior de Administración de Sistemas Informáticos en Rede”, IES Xulián Magariños, Documento Interno.

IES Xulián Magariños, “Anexo I: Rúbrica de corrección”, Documento de Evaluación Interno para el Ciclo Superior de ASIR.

8.3. Repositorio del Proyecto

Robles López, F. “OpenPaDi: Proyecto de Infraestructura PaaS para OpenPaDi”, Repositorio de Código Fuente en GitHub, 2025. [En línea]. Disponible en: <https://github.com/robleslf/OpenPaDi>